

Dependent Session Types for Verified Concurrent Programming (Companion Report)

ANONYMOUS AUTHOR(S)

Contents

1	Syntax	2
2	Auxiliary Operators/Judgments	2
3	Formal Typing Rules	4
3.1	Logical Level	4
3.2	Program Level	6
3.3	Process Level	7
4	Operational Semantics	8
4.1	Logical Level	8
4.2	Program Level	10
4.3	Process Level	11
5	Meta-Theory	12
5.1	Compatibility	12
5.2	Session Fidelity	21
5.3	Global Progress	27
5.4	Erasure Safety	41
	References	52

1 Syntax

The full syntax of TLL_C is presented below.

variables	x, y, z	
channels	c, d	
sorts	s, r, t	$::= \text{U} \mid \text{L}$
actions	ρ	$::= ! \mid ?$
terms	m, n, A, B, C	$::= x \mid c \mid s$ $\mid \Pi_t(x : A).B \mid \Pi_t\{x : A\}.B \mid \Sigma_t(x : A).B \mid \Sigma_t\{x : A\}.B$ $\mid \lambda_t(x : A).m \mid \lambda_t\{x : A\}.m \mid \langle m, n \rangle_t \mid \langle \{m\}, n \rangle_t$ $\mid m \ n \mid m \ \{n\} \mid R_{[z]A}^\Sigma(m, [x, y]n) \mid \mu(x : A).B$ $\mid \text{unit} \mid () \mid \text{bool} \mid \text{true} \mid \text{false} \mid R_{[z]A}^{\text{bool}}(m, n_1, n_2)$ $\mid C(A) \mid \text{return } m \mid \text{let } x \Leftarrow m \text{ in } n$ $\mid \text{proto} \mid \mathbf{1} \mid \rho(x : A). B \mid \rho\{x : A\}. B \mid \text{ch}\langle A \rangle \mid \text{hc}\langle A \rangle$ $\mid \text{fork}(x : A) \text{ with } m \mid \text{recv } m \mid \underline{\text{recv}} m \mid \text{send } m \mid \underline{\text{send}} m$ $\mid \text{close } m \mid \text{wait } m$
thunks	τ	$::= \text{fork}(x : A) \text{ with } m \mid \text{recv } c \mid \underline{\text{recv}} c$ $\mid \text{send } c \ v \mid \underline{\text{send}} c \ \{m\} \mid \text{close } c \mid \text{wait } c$ $\mid \text{let } x \Leftarrow \tau \text{ in } m$
values	u, v	$::= c \mid \lambda_t(x : A).m \mid \lambda_t\{x : A\}.m \mid \langle u, v \rangle_t \mid \langle \{m\}, v \rangle_t$ $\mid () \mid \text{true} \mid \text{false} \mid \text{return } v \mid \tau \mid \text{send } c \mid \underline{\text{send}} c$
process	O, P, Q	$\mid \langle m \rangle \mid (P \mid Q) \mid \text{vcd}.P$

2 Auxiliary Operators/Judgments

In this section, we define several auxiliary operators/judgments used in the formalization of TLL_C .

Sort Ordering. The sort ordering relation $\sqsubseteq$ is defined as follows:

$$(\text{ORD-U}) \quad \text{U} \sqsubseteq s \qquad (\text{ORD-L}) \quad \text{L} \sqsubseteq \text{L}$$

This relation is useful when defining the typing rules of dependent pairs by ensuring that pairs only contain values of a lower or equal sort.

Context Merge. The context merge operator $\cup$ is a partial function that combines two contexts into one by selective applying the contraction rule on unrestricted variables. The operator is undefined if both two contexts contain overlapping linear variables.

$$\begin{array}{c}
\text{MERGE-EMPTY} \\
\hline
\epsilon \cup \epsilon = \epsilon
\end{array}
\qquad
\begin{array}{c}
\text{MERGE-U} \\
\hline
\frac{\Delta_1 \cup \Delta_2 = \Delta \quad x \notin \Delta}{(\Delta_1, x :_{\text{U}} A) \cup (\Delta_2, x :_{\text{U}} A) = (\Delta, x :_{\text{U}} A)}
\end{array}$$

$$\begin{array}{c}
\text{MERGE-L}_1 \\
\hline
\frac{\Delta_1 \cup \Delta_2 = \Delta \quad x \notin \Delta}{(\Delta_1, x :_{\text{L}} A) \cup \Delta_2 = (\Delta, x :_{\text{L}} A)}
\end{array}
\qquad
\begin{array}{c}
\text{MERGE-L}_2 \\
\hline
\frac{\Delta_1 \cup \Delta_2 = \Delta \quad x \notin \Delta}{\Delta_1 \cup (\Delta_2, x :_{\text{L}} A) = (\Delta, x :_{\text{L}} A)}
\end{array}$$

Context Restriction. The context restriction operator $\triangleright$ is a predicate that is useful for defining the typing rules of λ -expressions. In particular, it prevents unrestricted functions from capturing linear variables in their closures.

$$\frac{}{\epsilon \triangleright s} (\text{RE-EMPTY}) \qquad \frac{\Delta \triangleright \text{U}}{\Delta, x :_{\text{U}} A \triangleright \text{U}} (\text{RE-U}) \qquad \frac{\Delta \triangleright \text{L}}{\Delta, x :_s A \triangleright \text{L}} (\text{RE-L})$$

Arity. For types A and X , we say that A is an *arity* ending on X if it is either X itself or a Π -type whose codomain is an arity ending on X . Formally, we define the judgment $A \text{ arity}(X)$ as follows:

$\frac{\text{ARITY-BASE}}{X \text{ arity}(X)}$	$\frac{\text{ARITY-IMPLICIT} \quad B \text{ arity}(X)}{\Pi_t \{x : A\}.B \text{ arity}(X)}$	$\frac{\text{ARITY-EXPLICIT} \quad B \text{ arity}(X)}{\Pi_t (x : A).B \text{ arity}(X)}$
--	---	---

This judgment is used for defining the typing rule of (parameterized) recursive protocols.

Guarded. For variable x and term m , we say that x is *guarded* in m if the judgment $m \text{ guard}(x)$ is derivable. Intuitively, this means that every occurrence of x in m appears under a protocol action. This is important for ensuring that recursive protocols do not unfold indefinitely without performing any actions. The judgment is defined as follows:

$\frac{\text{GUARD-VAR} \quad x \neq y}{y \text{ guard}(x)}$	$\frac{\text{GUARD-SORT}}{s \text{ guard}(x)}$	$\frac{\text{GUARD-IMPLICIT-FUN} \quad A \text{ guard}(x) \quad B \text{ guard}(x)}{\Pi_t \{x' : A\}.B \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-FUN} \quad A \text{ guard}(x) \quad B \text{ guard}(x)}{\Pi_t (x' : A).B \text{ guard}(x)}$
$\frac{\text{GUARD-IMPLICIT-LAM} \quad A \text{ guard}(x) \quad m \text{ guard}(x)}{\lambda_t \{x' : A\}.m \text{ guard}(x)}$		$\frac{\text{GUARD-EXPLICIT-LAM} \quad A \text{ guard}(x) \quad m \text{ guard}(x)}{\lambda_t (x' : A).m \text{ guard}(x)}$	$\frac{\text{GUARD-IMPLICIT-APP} \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{m \{n\} \text{ guard}(x)}$
$\frac{\text{GUARD-EXPLICIT-APP} \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{m \ n \text{ guard}(x)}$		$\frac{\text{GUARD-IMPLICIT-SUM} \quad A \text{ guard}(x) \quad B \text{ guard}(x)}{\Sigma_t \{x' : A\}.B \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-SUM} \quad A \text{ guard}(x) \quad B \text{ guard}(x)}{\Sigma_t (x' : A).B \text{ guard}(x)}$
$\frac{\text{GUARD-IMPLICIT-PAIR} \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{\langle \{m\}, n \rangle_t \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-PAIR} \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{\langle m, n \rangle \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-SUMELIM} \quad A \text{ guard}(x) \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{R_{[z]A}^\Sigma(m, [x', y']n)}$	
$\frac{\text{GUARD-UNIT}}{\text{unit guard}(x)}$	$\frac{\text{GUARD-UNITVAL}}{() \text{ guard}(x)}$	$\frac{\text{GUARD-BOOL}}{\text{bool guard}(x)}$	$\frac{\text{GUARD-TRUE}}{\text{true guard}(x)}$
			$\frac{\text{GUARD-FALSE}}{\text{false guard}(x)}$
$\frac{\text{GUARD-BOOLELIM} \quad A \text{ guard}(x) \quad m \text{ guard}(x) \quad n_1 \text{ guard}(x) \quad n_2 \text{ guard}(x)}{R_{[z]A}^{\text{bool}}(m, n_1, n_2) \text{ guard}(x)}$		$\frac{\text{GUARD-CTYPE} \quad A \text{ guard}(x)}{C(A) \text{ guard}(x)}$	$\frac{\text{GUARD-RETURN} \quad m \text{ guard}(x)}{\text{return } m \text{ guard}(x)}$
$\frac{\text{GUARD-BIND} \quad m \text{ guard}(x) \quad n \text{ guard}(x)}{\text{let } x' \leftarrow m \text{ in } n \text{ guard}(x)}$	$\frac{\text{GUARD-PROTO}}{\text{proto guard}(x)}$	$\frac{\text{GUARD-END}}{1 \text{ guard}(x)}$	$\frac{\text{GUARD-RECPROTO} \quad A \text{ guard}(x) \quad m \text{ guard}(x)}{\mu(x' : A).m \text{ guard}(x)}$
$\frac{\text{GUARD-IMPLICIT-ACTION} \quad A \text{ guard}(x)}{\rho \{x' : A\}. B \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-ACTION} \quad A \text{ guard}(x)}{\rho (x' : A). B \text{ guard}(x)}$	$\frac{\text{GUARD-CH} \quad A \text{ guard}(x)}{\text{ch}\langle A \rangle \text{ guard}(x)}$	$\frac{\text{GUARD-HC} \quad A \text{ guard}(x)}{\text{hc}\langle A \rangle \text{ guard}(x)}$
$\frac{\text{GUARD-CHANNEL}}{c \text{ guard}(x)}$	$\frac{\text{GUARD-FORK} \quad A \text{ guard}(x) \quad m \text{ guard}(x)}{\text{fork } (x' : A) \text{ with } m \text{ guard}(x)}$	$\frac{\text{GUARD-IMPLICIT-RECV} \quad m \text{ guard}(x)}{\text{recv } m \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-RECV} \quad m \text{ guard}(x)}{\text{recv } m \text{ guard}(x)}$
$\frac{\text{GUARD-IMPLICIT-SEND} \quad m \text{ guard}(x)}{\text{send } m \text{ guard}(x)}$	$\frac{\text{GUARD-EXPLICIT-SEND} \quad m \text{ guard}(x)}{\text{send } m \text{ guard}(x)}$	$\frac{\text{GUARD-WAIT} \quad m \text{ guard}(x)}{\text{wait } m \text{ guard}(x)}$	$\frac{\text{GUARD-CLOSE} \quad m \text{ guard}(x)}{\text{close } m \text{ guard}(x)}$

3 Formal Typing Rules

In this section, we present the full typing rules of TLL_C . We organize the typing rules into logical level, program level and process level.

3.1 Logical Level

The typing judgment for the logical level has the form $\Gamma \vdash m : A$. This judgment states that under the *logical context* Γ , term m has type A . The logical level is completely *structural*.

Logical Context. The logical context Γ is a sequence of variable bindings of the form $x_0 : A_0, x_1 : A_1, \dots, x_n : A_n$. Each variable x_i is bound to a type A_i . Variables in the logical context are unrestricted and can be used arbitrarily many times. The empty context is denoted by ϵ . To ensure the validity of types in the logical context, we define the context validity judgment $\Gamma \vdash$.

$$\begin{array}{c} \text{CTX-EMPTY} \\ \hline \epsilon \vdash \end{array} \quad \begin{array}{c} \text{CTX-VAR} \\ \hline \Gamma \vdash \quad \Gamma \vdash A : s \quad x \notin \Gamma \\ \hline \Gamma, x : A \vdash \end{array}$$

Note that the context validity judgment is *mutually inductively* defined with the typing judgment.

Core Typing. The core typing rules is responsible for the functional fragment of TLL_C . The convertibility relation $A \simeq B$ is used in the conversion rule to allow type equivalence up to β -reduction. We will present the definition of the convertibility relation in Section 4.1.

$$\begin{array}{c} \begin{array}{c} \text{SORT} \\ \hline \Gamma \vdash \\ \hline \Gamma \vdash s : U \end{array} \quad \begin{array}{c} \text{VAR} \\ \hline \Gamma, x : A \vdash \\ \hline \Gamma, x : A \vdash x : A \end{array} \quad \begin{array}{c} \text{CONVERSION} \\ \hline \Gamma \vdash B : s \quad \Gamma \vdash m : A \quad A \simeq B \\ \hline \Gamma \vdash m : B \end{array} \quad \begin{array}{c} \text{EXPLICIT-FUN} \\ \hline \Gamma \vdash A : s \quad \Gamma, x : A \vdash B : r \\ \hline \Gamma \vdash \Pi_t(x : A).B : t \end{array} \\ \\ \begin{array}{c} \text{IMPLICIT-FUN} \\ \hline \Gamma \vdash A : s \quad \Gamma, x : A \vdash B : r \\ \hline \Gamma \vdash \Pi_t\{x : A\}.B : t \end{array} \quad \begin{array}{c} \text{EXPLICIT-LAM} \\ \hline \Gamma, x : A \vdash m : B \\ \hline \Gamma \vdash \lambda_t(x : A).m : \Pi_t(x : A).B \end{array} \quad \begin{array}{c} \text{IMPLICIT-LAM} \\ \hline \Gamma, x : A \vdash m : B \\ \hline \Gamma \vdash \lambda_t\{x : A\}.m : \Pi_t\{x : A\}.B \end{array} \\ \\ \begin{array}{c} \text{EXPLICIT-APP} \\ \hline \Gamma \vdash m : \Pi_t(x : A).B \quad \Gamma \vdash n : A \\ \hline \Gamma \vdash m n : B[n/x] \end{array} \quad \begin{array}{c} \text{IMPLICIT-APP} \\ \hline \Gamma \vdash m : \Pi_t\{x : A\}.B \quad \Gamma \vdash n : A \\ \hline \Gamma \vdash m \{n\} : B[n/x] \end{array} \\ \\ \begin{array}{c} \text{EXPLICIT-SUM} \\ \hline s \sqsubseteq t \quad r \sqsubseteq t \quad \Gamma \vdash A : s \quad \Gamma, x : A \vdash B : r \\ \hline \Gamma \vdash \Sigma_t(x : A).B : t \end{array} \quad \begin{array}{c} \text{IMPLICIT-SUM} \\ \hline r \sqsubseteq t \quad \Gamma \vdash A : s \quad \Gamma, x : A \vdash B : r \\ \hline \Gamma \vdash \Sigma_t\{x : A\}.B : t \end{array} \\ \\ \begin{array}{c} \text{EXPLICIT-PAIR} \\ \hline \Gamma \vdash \Sigma_t(x : A).B : t \quad \Gamma \vdash m : A \quad \Gamma \vdash n : B[m/x] \\ \hline \Gamma \vdash \langle m, n \rangle_t : \Sigma_t(x : A).B \end{array} \quad \begin{array}{c} \text{IMPLICIT-PAIR} \\ \hline \Gamma \vdash \Sigma_t\{x : A\}.B : t \quad \Gamma \vdash m : A \quad \Gamma \vdash n : B[m/x] \\ \hline \Gamma \vdash \langle \{m\}, n \rangle_t : \Sigma_t\{x : A\}.B \end{array} \\ \\ \begin{array}{c} \text{EXPLICIT-SUMELIM} \\ \hline \Gamma, z : \Sigma_t(x : A).B \vdash C : s \quad \Gamma \vdash m : \Sigma_t(x : A).B \quad \Gamma, x : A, y : B \vdash n : C[\langle x, y \rangle_t/z] \\ \hline \Gamma \vdash R_{[z]C}^\Sigma(m, [x, y]n) : C[m/z] \end{array} \\ \\ \begin{array}{c} \text{IMPLICIT-SUMELIM} \\ \hline \Gamma, z : \Sigma_t\{x : A\}.B \vdash C : s \quad \Gamma \vdash m : \Sigma_t\{x : A\}.B \quad \Gamma, x : A, y : B \vdash n : C[\langle \{x\}, y \rangle_t/z] \\ \hline \Gamma \vdash R_{[z]C}^\Sigma(m, [x, y]n) : C[m/z] \end{array} \end{array}$$

Data Typing. The data typing rules govern the typing of base types such as the unit type and the boolean type. The rules are presented below.

$$\begin{array}{c}
\text{UNIT} \\
\frac{\Gamma \vdash}{\Gamma \vdash \text{unit} : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{UNITVAL} \\
\frac{\Gamma \vdash}{\Gamma \vdash () : \text{unit}}
\end{array}
\quad
\begin{array}{c}
\text{BOOL} \\
\frac{\Gamma \vdash}{\Gamma \vdash \text{bool} : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{TRUE} \\
\frac{\Gamma \vdash}{\Gamma \vdash \text{true} : \text{bool}}
\end{array}
\quad
\begin{array}{c}
\text{FALSE} \\
\frac{\Gamma \vdash}{\Gamma \vdash \text{false} : \text{bool}}
\end{array}$$

$$\begin{array}{c}
\text{BOOLELIM} \\
\frac{\Gamma, z : \text{bool} \vdash A : s \quad \Gamma \vdash m : \text{bool} \quad \Gamma \vdash n_1 : A[\text{true}/z] \quad \Gamma \vdash n_2 : A[\text{false}/z]}{\Gamma \vdash R_{[z]A}^{\text{bool}}(m, n_1, n_2) : A[m/z]}
\end{array}$$

Monadic Typing. The monadic typing rules govern the composition of monadic computations. The standard rules for monadic return and bind are presented below.

$$\begin{array}{c}
\text{CTYPE} \\
\frac{\Gamma \vdash A : s}{\Gamma \vdash C(A) : \mathbf{L}}
\end{array}
\quad
\begin{array}{c}
\text{RETURN} \\
\frac{\Gamma \vdash m : A}{\Gamma \vdash \text{return } m : C(A)}
\end{array}
\quad
\begin{array}{c}
\text{BIND} \\
\frac{\Gamma \vdash B : s \quad \Gamma \vdash m : C(A) \quad \Gamma, x : A \vdash n : C(B)}{\Gamma \vdash \text{let } x \Leftarrow m \text{ in } n : C(B)}
\end{array}$$

Session Typing. The session typing rules govern the typing of protocol, channels and concurrency primitives. The rules are presented below.

$$\begin{array}{c}
\text{PROTO} \\
\frac{\Gamma \vdash}{\Gamma \vdash \text{proto} : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{END} \\
\frac{\Gamma \vdash}{\Gamma \vdash \mathbf{1} : \text{proto}}
\end{array}
\quad
\begin{array}{c}
\text{EXPLICIT-ACTION} \\
\frac{\Gamma, x : A \vdash B : \text{proto}}{\Gamma \vdash \rho(x : A). B : \text{proto}}
\end{array}
\quad
\begin{array}{c}
\text{IMPLICIT-ACTION} \\
\frac{\Gamma, x : A \vdash B : \text{proto}}{\Gamma \vdash \rho\{x : A\}. B : \text{proto}}
\end{array}$$

$$\begin{array}{c}
\text{RECPROTO} \\
\frac{\Gamma, x : A \vdash m : A \quad A \text{ arity}(\text{proto}) \quad m \text{ guard}(x)}{\Gamma \vdash \mu(x : A). m : A}
\end{array}
\quad
\begin{array}{c}
\text{CHTYPE} \\
\frac{\Gamma \vdash A : \text{proto}}{\Gamma \vdash \text{ch}\langle A \rangle : \mathbf{L}}
\end{array}
\quad
\begin{array}{c}
\text{HCTYPE} \\
\frac{\Gamma \vdash A : \text{proto}}{\Gamma \vdash \text{hc}\langle A \rangle : \mathbf{L}}
\end{array}
\quad
\begin{array}{c}
\text{CHANNEL-CH} \\
\frac{\Gamma \vdash \epsilon \vdash A : \text{proto}}{\Gamma \vdash c : \text{ch}\langle A \rangle}
\end{array}$$

$$\begin{array}{c}
\text{CHANNEL-HC} \\
\frac{\Gamma \vdash \epsilon \vdash A : \text{proto}}{\Gamma \vdash c : \text{hc}\langle A \rangle}
\end{array}
\quad
\begin{array}{c}
\text{EXPLICIT-SEND-CH} \\
\frac{\Gamma \vdash m : \text{ch}\langle ! (x : A) \rangle. B}{\Gamma \vdash \text{send } m : \Pi_L(x : A). C(\text{ch}\langle B \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{EXPLICIT-SEND-HC} \\
\frac{\Gamma \vdash m : \text{hc}\langle ? (x : A) \rangle. B}{\Gamma \vdash \text{send } m : \Pi_L(x : A). C(\text{hc}\langle B \rangle)}
\end{array}$$

$$\begin{array}{c}
\text{IMPLICIT-SEND-CH} \\
\frac{\Gamma \vdash m : \text{ch}\langle ! \{x : A\} \rangle. B}{\Gamma \vdash \text{send } m : \Pi_L\{x : A\}. C(\text{ch}\langle B \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{IMPLICIT-SEND-HC} \\
\frac{\Gamma \vdash m : \text{hc}\langle ? \{x : A\} \rangle. B}{\Gamma \vdash \text{send } m : \Pi_L\{x : A\}. C(\text{hc}\langle B \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{EXPLICIT-RECV-CH} \\
\frac{\Gamma \vdash m : \text{ch}\langle ? (x : A) \rangle. B}{\Gamma \vdash \text{recv } m : C(\Sigma_L(x : A). \text{ch}\langle B \rangle)}
\end{array}$$

$$\begin{array}{c}
\text{EXPLICIT-RECV-HC} \\
\frac{\Gamma \vdash m : \text{hc}\langle ! (x : A) \rangle. B}{\Gamma \vdash \text{recv } m : C(\Sigma_L(x : A). \text{hc}\langle B \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{IMPLICIT-RECV-CH} \\
\frac{\Gamma \vdash m : \text{ch}\langle ? \{x : A\} \rangle. B}{\Gamma \vdash \text{recv } m : C(\Sigma_L\{x : A\}. \text{ch}\langle B \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{IMPLICIT-RECV-HC} \\
\frac{\Gamma \vdash m : \text{hc}\langle ! \{x : A\} \rangle. B}{\Gamma \vdash \text{recv } m : C(\Sigma_L\{x : A\}. \text{hc}\langle B \rangle)}
\end{array}$$

$$\begin{array}{c}
\text{FORK} \\
\frac{\Gamma, x : \text{ch}\langle A \rangle \vdash m : C(\text{unit})}{\Gamma \vdash \text{fork } (x : \text{ch}\langle A \rangle) \text{ with } m : C(\text{hc}\langle A \rangle)}
\end{array}
\quad
\begin{array}{c}
\text{CLOSE} \\
\frac{\Gamma \vdash m : \text{ch}\langle \mathbf{1} \rangle}{\Gamma \vdash \text{close } m : C(\text{unit})}
\end{array}
\quad
\begin{array}{c}
\text{WAIT} \\
\frac{\Gamma \vdash m : \text{hc}\langle \mathbf{1} \rangle}{\Gamma \vdash \text{wait } m : C(\text{unit})}
\end{array}$$

3.2 Program Level

The typing judgment for the program level has the form $\Theta; \Gamma; \Delta \vdash m : A$. This judgment states that under the channel context Θ , logical context Γ and the *program context* Δ , term m has type A . The program level is *substructural* as the usage of variables in the program context is tracked.

Program Context. The program context Δ is a sequence of variable bindings of the form $x_0 :_{s_0} A_0, x_1 :_{s_1} A_1, \dots, x_n :_{s_n} A_n$. Each variable x_i is bound to a type A_i with a sort annotation s_i . The variables in the program context are allowed to appear in computationally relevant positions inside m . To ensure that all types appear in the program context are well-formed, we define the program context validity judgment $\Gamma; \Delta \vdash$. The rules for this judgment are presented below.

$$\begin{array}{c} \text{CTX-EMPTY} \\ \hline \epsilon; \epsilon \vdash \end{array} \quad \begin{array}{c} \text{CTX-IMPLICIT-VAR} \\ \hline \Gamma; \Delta \vdash \quad \Gamma \vdash A : s \quad x \notin \Gamma \\ \hline \Gamma, x : A; \Delta \vdash \end{array} \quad \begin{array}{c} \text{CTX-EXPLICIT-VAR} \\ \hline \Gamma; \Delta \vdash \quad \Gamma \vdash A : s \quad x \notin \Gamma \\ \hline \Gamma, x : A; \Delta, x :_s A \vdash \end{array}$$

From these rules we can see that $\text{dom}(\Delta)$ is a subset of $\text{dom}(\Gamma)$. Additionally, the sort annotation s in each program context binding $x :_s A$ is the sort of the associated A type.

Core Typing. The core typing rules is responsible for the functional fragment of TLL_C .

$$\begin{array}{c} \text{VAR} \\ \hline \epsilon; \Gamma, x : A; \Delta, x :_s A \vdash \quad \Delta \triangleright U \\ \hline \epsilon; \Gamma, x : A; \Delta, x :_s A \vdash x : A \end{array} \quad \begin{array}{c} \text{CONVERSION} \\ \hline \Gamma \vdash B : s \quad \Theta; \Gamma; \Delta \vdash m : A \quad A \simeq B \\ \hline \Theta; \Gamma; \Delta \vdash m : B \end{array}$$

$$\begin{array}{c} \text{EXPLICIT-LAM} \\ \hline \Theta; \Gamma, x : A; \Delta, x :_s A \vdash m : B \quad \Theta \triangleright t \quad \Delta \triangleright t \\ \hline \Theta; \Gamma; \Delta \vdash \lambda_t(x : A).m : \Pi_t(x : A).B \end{array} \quad \begin{array}{c} \text{IMPLICIT-LAM} \\ \hline \Theta; \Gamma, x : A; \Delta \vdash m : B \quad \Theta \triangleright t \quad \Delta \triangleright t \\ \hline \Theta; \Gamma; \Delta \vdash \lambda_t\{x : A\}.m : \Pi_t\{x : A\}.B \end{array}$$

$$\begin{array}{c} \text{EXPLICIT-APP} \\ \hline \Theta_1; \Gamma; \Delta_1 \vdash m : \Pi_t(x : A).B \quad \Theta_2; \Gamma; \Delta_2 \vdash n : A \\ \hline \Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m \ n : B[n/x] \end{array} \quad \begin{array}{c} \text{IMPLICIT-APP} \\ \hline \Theta; \Gamma; \Delta \vdash m : \Pi_t\{x : A\}.B \quad \Gamma \vdash n : A \\ \hline \Theta; \Gamma; \Delta \vdash m \ \{n\} : B[n/x] \end{array}$$

$$\begin{array}{c} \text{EXPLICIT-PAIR} \\ \hline \Gamma \vdash \Sigma_t(x : A).B : t \\ \hline \Theta_1; \Gamma; \Delta_1 \vdash m : A \quad \Theta_2; \Gamma; \Delta_2 \vdash n : B[m/x] \\ \hline \Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash \langle m, n \rangle_t : \Sigma_t(x : A).B \end{array} \quad \begin{array}{c} \text{IMPLICIT-PAIR} \\ \hline \Gamma \vdash \Sigma_t\{x : A\}.B : t \\ \hline \Gamma \vdash m : A \quad \Theta; \Gamma; \Delta \vdash n : B[m/x] \\ \hline \Theta; \Gamma; \Delta \vdash \langle \{m\}, n \rangle_t : \Sigma_t\{x : A\}.B \end{array}$$

$$\begin{array}{c} \text{EXPLICIT-SUMELIM} \\ \hline \Gamma, z : \Sigma_t(x : A).B \vdash C : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m : \Sigma_t(x : A).B \\ \hline \Theta_2; \Gamma, x : A, y : B; \Delta_2, x :_{r1} A, y :_{r2} B \vdash n : C[\langle x, y \rangle_t/z] \\ \hline \Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m, [x, y]n) : C[m/z] \end{array} \quad \begin{array}{c} \text{IMPLICIT-SUMELIM} \\ \hline \Gamma, z : \Sigma_t\{x : A\}.B \vdash C : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m : \Sigma_t\{x : A\}.B \\ \hline \Theta_2; \Gamma, x : A, y : B; \Delta_2, y :_r B \vdash n : C[\langle \{x\}, y \rangle_t/z] \\ \hline \Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m, [x, y]n) : C[m/z] \end{array}$$

Data Typing. The data typing rules govern the typing of base types such as the unit type and the boolean type. The rules are presented below.

$$\begin{array}{c} \text{UNITVAL} \\ \hline \Gamma; \Delta \vdash \quad \Delta \triangleright U \\ \hline \epsilon; \Gamma; \Delta \vdash () : \text{unit} \end{array} \quad \begin{array}{c} \text{TRUE} \\ \hline \Gamma; \Delta \vdash \quad \Delta \triangleright U \\ \hline \epsilon; \Gamma; \Delta \vdash \text{true} : \text{bool} \end{array} \quad \begin{array}{c} \text{FALSE} \\ \hline \Gamma; \Delta \vdash \quad \Delta \triangleright U \\ \hline \epsilon; \Gamma; \Delta \vdash \text{false} : \text{bool} \end{array}$$

$$\begin{array}{c} \text{BOOLELIM} \\ \hline \Gamma, z : \text{bool} \vdash A : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m : \text{bool} \quad \Theta_2; \Gamma; \Delta_2 \vdash n_1 : A[\text{true}/z] \quad \Theta_3; \Gamma; \Delta_2 \vdash n_2 : A[\text{false}/z] \\ \hline \Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]A}^{\text{bool}}(m, n_1, n_2) : A[m/z] \end{array}$$

Monadic Typing. The monadic typing rules govern the composition of monadic computations. The standard rules for monadic return and bind are presented below.

$$\begin{array}{c}
 \text{RETURN} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : A}{\Theta; \Gamma; \Delta \vdash \text{return } m : C(A)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{BIND} \\
 \frac{\Gamma \vdash B : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m : C(A) \quad \Theta_2; \Gamma, x : A; \Delta_2, x :_r A \vdash n : C(B)}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash \text{let } x \leftarrow m \text{ in } n : C(B)}
 \end{array}$$

Session Typing. The session typing rules govern the typing of channels and concurrency primitives. The rules are presented below.

$$\begin{array}{c}
 \text{CHANNEL-CH} \\
 \frac{\Gamma; \Delta \vdash \epsilon \vdash A : \text{proto} \quad \Delta \triangleright U}{c :_{\text{L}} \text{ch}\langle A \rangle; \Gamma; \Delta \vdash c : \text{ch}\langle A \rangle}
 \end{array}
 \quad
 \begin{array}{c}
 \text{CHANNEL-HC} \\
 \frac{\Gamma; \Delta \vdash \epsilon \vdash A : \text{proto} \quad \Delta \triangleright U}{c :_{\text{L}} \text{hc}\langle A \rangle; \Gamma; \Delta \vdash c : \text{hc}\langle A \rangle}
 \end{array}$$

$$\begin{array}{c}
 \text{EXPLICIT-SEND-CH} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{ch}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{send } m : \Pi_{\text{L}}(x : A). C(\text{ch}\langle B \rangle)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{EXPLICIT-SEND-HC} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{hc}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{send } m : \Pi_{\text{L}}(x : A). C(\text{hc}\langle B \rangle)}
 \end{array}$$

$$\begin{array}{c}
 \text{IMPLICIT-SEND-CH} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{ch}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{send}} m : \Pi_{\text{L}}\{x : A\}. C(\text{ch}\langle B \rangle)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{IMPLICIT-SEND-HC} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{hc}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{send}} m : \Pi_{\text{L}}\{x : A\}. C(\text{hc}\langle B \rangle)}
 \end{array}$$

$$\begin{array}{c}
 \text{EXPLICIT-RECV-CH} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{ch}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{recv } m : C(\Sigma_{\text{L}}(x : A). \text{ch}\langle B \rangle)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{EXPLICIT-RECV-HC} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{hc}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{recv } m : C(\Sigma_{\text{L}}(x : A). \text{hc}\langle B \rangle)}
 \end{array}$$

$$\begin{array}{c}
 \text{IMPLICIT-RECV-CH} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{ch}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{recv}} m : C(\Sigma_{\text{L}}\{x : A\}. \text{ch}\langle B \rangle)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{IMPLICIT-RECV-HC} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{hc}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{recv}} m : C(\Sigma_{\text{L}}\{x : A\}. \text{hc}\langle B \rangle)}
 \end{array}$$

$$\begin{array}{c}
 \text{FORK} \\
 \frac{\Theta; \Gamma, x : \text{ch}\langle A \rangle; \Delta, x :_{\text{L}} \text{ch}\langle A \rangle \vdash m : C(\text{unit})}{\Theta; \Gamma; \Delta \vdash \text{fork } (x : \text{ch}\langle A \rangle) \text{ with } m : C(\text{hc}\langle A \rangle)}
 \end{array}
 \quad
 \begin{array}{c}
 \text{CLOSE} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{ch}\langle 1 \rangle}{\Theta; \Gamma; \Delta \vdash \text{close } m : C(\text{unit})}
 \end{array}
 \quad
 \begin{array}{c}
 \text{WAIT} \\
 \frac{\Theta; \Gamma; \Delta \vdash m : \text{hc}\langle 1 \rangle}{\Theta; \Gamma; \Delta \vdash \text{wait } m : C(\text{unit})}
 \end{array}$$

3.3 Process Level

The typing judgment for the process level has the form $\Theta \Vdash P$. This judgment states that under the channel context Θ , process P is well-typed. Unlike the logical and program levels which can type term that contain free variables, the process level only types processes whose terms are closed. Hence, there are no logical or program contexts in the process typing judgment.

$$\begin{array}{c}
 \text{EXPR} \\
 \frac{\Theta; \epsilon; \vdash m : C(\text{unit})}{\Theta \Vdash \langle m \rangle}
 \end{array}
 \quad
 \begin{array}{c}
 \text{PAR} \\
 \frac{\Theta_1 \Vdash P \quad \Theta_2 \Vdash Q}{\Theta_1 \cup \Theta_2 \Vdash P \mid Q}
 \end{array}
 \quad
 \begin{array}{c}
 \text{SCOPE} \\
 \frac{\Theta, c :_{\text{L}} \text{ch}\langle A \rangle, d :_{\text{L}} \text{hc}\langle A \rangle \Vdash P}{\Theta \Vdash \text{vcd}.P}
 \end{array}$$

4 Operational Semantics

In this section, we present the operational semantics of TLL_C . Similarly to the typing rules, we organize the presentation of the semantics into the logical level, program level, and process level.

4.1 Logical Level

The semantics of the logical level is defined in terms of the *parallel reduction* relation $m \Rightarrow m'$. This relation allows multiple redexes to be reduced simultaneously.

Core Reduction. The parallel reduction for core functional terms is defined as follows:

$\frac{}{x \Rightarrow x}$	$\frac{}{s \Rightarrow s}$	$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\Pi_s(x : A).B \Rightarrow \Pi_s(x : A').B'}$	$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\Pi_s\{x : A\}.B \Rightarrow \Pi_s\{x : A'\}.B'}$
$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\lambda_s(x : A).B \Rightarrow \lambda_s(x : A').B'}$	$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\lambda_s\{x : A\}.B \Rightarrow \lambda_s\{x : A'\}.B'}$	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{m n \Rightarrow m' n'}$	
$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{m \{n\} \Rightarrow m' \{n'\}}$	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{(\lambda_s(x : A).m) n \Rightarrow m' [n'/x]}$	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{(\lambda_s\{x : A\}.m) \{n\} \Rightarrow m' [n'/x]}$	
$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\Sigma_s(x : A).B \Rightarrow \Sigma_s(x : A').B'}$	$\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\Sigma_s\{x : A\}.B \Rightarrow \Sigma_s\{x : A'\}.B'}$	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{\langle m, n \rangle_s \Rightarrow \langle m', n' \rangle_s}$	
$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{\langle \{m\}, n \rangle_s \Rightarrow \langle \{m'\}, n' \rangle_s}$	$\frac{A \Rightarrow A' \quad m \Rightarrow m' \quad n \Rightarrow n'}{R_{[z]A}^\Sigma(m, [x, y]n) \Rightarrow R_{[z]A'}^\Sigma(m', [x, y]n')}$		
$\frac{m_1 \Rightarrow m'_1 \quad m_2 \Rightarrow m'_2 \quad n \Rightarrow n'}{R_{[z]A}^\Sigma(\langle m_1, m_2 \rangle_s, [x, y]n) \Rightarrow n' [m_1/x, m_2/y]}$	$\frac{m_1 \Rightarrow m'_1 \quad m_2 \Rightarrow m'_2 \quad n \Rightarrow n'}{R_{[z]A}^\Sigma(\langle \{m_1\}, m_2 \rangle_s, [x, y]n) \Rightarrow n' [m_1/x, m_2/y]}$		

Data Reduction. The parallel reduction for data terms is defined as follows:

$\frac{}{\text{unit} \Rightarrow \text{unit}}$	$\frac{}{() \Rightarrow ()}$	$\frac{}{\text{bool} \Rightarrow \text{bool}}$	$\frac{}{\text{true} \Rightarrow \text{true}}$	$\frac{}{\text{false} \Rightarrow \text{false}}$
$\frac{A \Rightarrow A' \quad m \Rightarrow m' \quad n_1 \Rightarrow n'_1 \quad n_2 \Rightarrow n'_2}{R_{[z]A}^{\text{bool}}(m, n_1, n_2) \Rightarrow R_{[z]A'}^{\text{bool}}(m', n'_1, n'_2)}$	$\frac{n_1 \Rightarrow n'_1}{R_{[z]A}^{\text{bool}}(\text{true}, n_1, n_2) \Rightarrow n'_1}$	$\frac{n_2 \Rightarrow n'_2}{R_{[z]A}^{\text{bool}}(\text{false}, n_1, n_2) \Rightarrow n'_2}$		

Monadic Reduction. The parallel reduction for monadic terms is defined as follows:

$\frac{A \Rightarrow A'}{C(A) \Rightarrow C(A')}$	$\frac{m \Rightarrow m'}{\text{return } m \Rightarrow \text{return } m'}$	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{\text{let } x \leftarrow m \text{ in } n \Rightarrow \text{let } x \leftarrow m' \text{ in } n'}$
	$\frac{m \Rightarrow m' \quad n \Rightarrow n'}{\text{let } x \leftarrow \text{return } m \text{ in } n \Rightarrow n' [m'/x]}$	

Session Reduction. The parallel reduction for protocols, channels and concurrency primitives are defined as follows:

$$\begin{array}{c}
\text{PSTEP-PROTO} \quad \text{PSTEP-END} \quad \text{PSTEP-RECPROTO} \quad \text{PSTEP-RECUNFOLD} \\
\frac{}{\text{proto} \Rightarrow \text{proto}} \quad \frac{}{\mathbf{1} \Rightarrow \mathbf{1}} \quad \frac{A \Rightarrow A' \quad m \Rightarrow m'}{\mu(x : A).m \Rightarrow \mu(x : A').m'} \quad \frac{A \Rightarrow A' \quad m \Rightarrow m'}{\mu(x : A).m \Rightarrow m'[(\mu(x : A').m')/x]} \\
\\
\text{PSTEP-EXPLICIT-ACTION} \quad \text{PSTEP-IMPLICIT-ACTION} \quad \text{PSTEP-CH} \quad \text{PSTEP-HC} \\
\frac{A \Rightarrow A' \quad B \Rightarrow B'}{\rho(x : A).B \Rightarrow \rho(x : A').B'} \quad \frac{A \Rightarrow A' \quad B \Rightarrow B'}{\rho\{x : A\}.B \Rightarrow \rho\{x : A'\}.B'} \quad \frac{A \Rightarrow A'}{\text{ch}\langle A \rangle \Rightarrow \text{ch}\langle A' \rangle} \quad \frac{A \Rightarrow A'}{\text{hc}\langle A \rangle \Rightarrow \text{hc}\langle A' \rangle} \\
\\
\text{PSTEP-CHANNEL} \quad \text{PSTEP-FORK} \quad \text{PSTEP-EXPLICIT-SEND} \\
\frac{}{c \Rightarrow d} \quad \frac{A \Rightarrow A' \quad m \Rightarrow m'}{\text{fork}(x : A) \text{ with } m \Rightarrow \text{fork}(x : A') \text{ with } m'} \quad \frac{m \Rightarrow m'}{\text{send } m \Rightarrow \text{send } m'} \\
\\
\text{PSTEP-IMPLICIT-SEND} \quad \text{PSTEP-EXPLICIT-RCV} \quad \text{PSTEP-IMPLICIT-RCV} \quad \text{PSTEP-CLOSE} \\
\frac{m \Rightarrow m'}{\text{send } m \Rightarrow \text{send } m'} \quad \frac{m \Rightarrow m'}{\text{recv } m \Rightarrow \text{recv } m'} \quad \frac{m \Rightarrow m'}{\text{recv } m \Rightarrow \text{recv } m'} \quad \frac{m \Rightarrow m'}{\text{close } m \Rightarrow \text{close } m'} \\
\\
\text{PSTEP-WAIT} \\
\frac{m \Rightarrow m'}{\text{wait } m \Rightarrow \text{wait } m'}
\end{array}$$

Convertibility Relation. The convertibility relation $A \simeq B$ is the reflexive, symmetric and transitive closure of the parallel reduction relation. It can be inductively defined as follows:

$$\begin{array}{c}
\text{CONV-REFL} \quad \text{CONV-PSTEP} \quad \text{CONV-PSTEP-REV} \\
\frac{}{A \simeq A} \quad \frac{A \simeq B \quad B \Rightarrow C}{A \simeq C} \quad \frac{A \simeq B \quad C \Rightarrow B}{A \simeq C}
\end{array}$$

Note that the program level CONVERSION rule (Section 3.2) also uses this convertibility relation.

4.2 Program Level

The semantics of the program level is defined in terms of a small-step reduction relation $m \rightsquigarrow m'$. Unlike the logical level which has a non-deterministic reduction strategy, the program level follows call-by-value evaluation. Arguments are fully evaluated before substitution into functions.

Core Reduction. The small-step reduction for core functional terms is defined as follows:

STEP-EXPLICIT-APP ₁ $\frac{m \rightsquigarrow m'}{m\ n \rightsquigarrow m'\ n}$	STEP-EXPLICIT-APP ₂ $\frac{n \rightsquigarrow n'}{m\ n \rightsquigarrow m\ n'}$	STEP-IMPLICIT-APP ₁ $\frac{m \rightsquigarrow m'}{m\ \{n\} \rightsquigarrow m'\ \{n\}}$	STEP-EXPLICIT- β $\frac{}{(\lambda_s(x : A).m)\ v \rightsquigarrow m[v/x]}$
STEP-IMPLICIT- β $\frac{}{(\lambda_s(x : A).m)\ \{n\} \rightsquigarrow m[n/x]}$	STEP-EXPLICIT-PAIR ₁ $\frac{m \rightsquigarrow m'}{\langle m, n \rangle_s \rightsquigarrow \langle m', n \rangle_s}$	STEP-EXPLICIT-PAIR ₂ $\frac{n \rightsquigarrow n'}{\langle m, n \rangle_s \rightsquigarrow \langle m, n' \rangle_s}$	
STEP-IMPLICIT-PAIR ₂ $\frac{n \rightsquigarrow n'}{\langle \{m\}, n \rangle_s \rightsquigarrow \langle \{m\}, n' \rangle_s}$	STEP-SUMELIM ₁ $\frac{m \rightsquigarrow m'}{R_{[z]A}^\Sigma(m, [x, y]n) \rightsquigarrow R_{[z]A}^\Sigma(m', [x, y]n)}$		
STEP-EXPLICIT-PAIRELIM $\frac{}{R_{[z]A}^\Sigma(\langle u, v \rangle_s, [x, y]n) \rightsquigarrow n[u/x, v/y]}$	STEP-IMPLICIT-PAIRELIM $\frac{}{R_{[z]A}^\Sigma(\langle \{m\}, v \rangle_s, [x, y]n) \rightsquigarrow n[m/x, v/y]}$		

Data Reduction. The small-step reduction for data terms is defined as follows:

STEP-BOOELIM $\frac{m \rightsquigarrow m'}{R_{[z]A}^{\text{bool}}(m, n_1, n_2) \rightsquigarrow R_{[z]A}^{\text{bool}}(m', n_1, n_2)}$	STEP-TRUEELIM $\frac{}{R_{[z]A}^{\text{bool}}(\text{true}, n_1, n_2) \rightsquigarrow n_1}$	STEP-FALSEELIM $\frac{}{R_{[z]A}^{\text{bool}}(\text{false}, n_1, n_2) \rightsquigarrow n_2}$
---	--	--

Monadic Reduction. The small-step reduction for monadic terms is defined as follows:

STEP-RETURN $\frac{m \rightsquigarrow m'}{\text{return } m \rightsquigarrow \text{return } m'}$	STEP-BIND $\frac{m \rightsquigarrow m'}{\text{let } x \leftarrow m \text{ in } n \rightsquigarrow \text{let } x \leftarrow m' \text{ in } n}$	STEP-RETURNBIND $\frac{}{\text{let } x \leftarrow \text{return } v \text{ in } n \rightsquigarrow n[v/x]}$
--	--	---

Session Reduction. The small-step reduction for session terms is defined as follows:

STEP-EXPLICIT-SEND $\frac{m \rightsquigarrow m'}{\text{send } m \rightsquigarrow \text{send } m'}$	STEP-IMPLICIT-SEND $\frac{m \rightsquigarrow m'}{\text{send } m \rightsquigarrow \text{send } m'}$	STEP-EXPLICIT-RECV $\frac{m \rightsquigarrow m'}{\text{recv } m \rightsquigarrow \text{recv } m'}$	STEP-IMPLICIT-RECV $\frac{m \rightsquigarrow m'}{\text{recv } m \rightsquigarrow \text{recv } m'}$
STEP-CLOSE $\frac{m \rightsquigarrow m'}{\text{close } m \rightsquigarrow \text{close } m'}$	STEP-WAIT $\frac{m \rightsquigarrow m'}{\text{wait } m \rightsquigarrow \text{wait } m'}$		

4.3 Process Level

The semantics of the process level is defined in terms of a small-step reduction relation $P \Rightarrow Q$. This relation is what gives TLL_C its concurrent behavior. Before we present the reduction rules, we first define the notion of *structural congruence* $\equiv$ which identifies processes that are the same up to reordering of parallel components and renaming of bound channels.

Structural Congruence. The structural congruence relation $\equiv$ is defined as follows:

$$\begin{aligned} P \mid Q &\equiv Q \mid P & O \mid (P \mid Q) &\equiv (O \mid P) \mid Q & P \mid \langle \text{return } () \rangle &\equiv P \\ vcd.P \mid Q &\equiv vcd.(P \mid Q) & vcd.P &\equiv vdc.P & vcd.vc'd'.P &\equiv vc'd'.vcd.P \end{aligned}$$

Process Reduction. The small-step reduction for processes is defined as follows:

Evaluation Contexts $M, N ::= [\cdot] \mid \text{let } x \leftarrow M \text{ in } m$

(PROC-FORK) $\langle N[\text{fork } (x : A) \text{ with } m] \rangle \Rightarrow vcd.(\langle N[\text{return } c] \rangle \mid \langle m[d/x] \rangle)$			
(PROC-END) $vcd.(\langle M[\text{close } c] \rangle \mid \langle N[\text{wait } d] \rangle) \Rightarrow \langle M[\text{return } ()] \rangle \mid \langle N[\text{return } ()] \rangle$			
(PROC-COM) $vcd.(\langle M[\text{send } c \ v] \rangle \mid \langle N[\text{recv } d] \rangle) \Rightarrow vcd.(\langle M[\text{return } c] \rangle \mid \langle N[\text{return } \langle v, d \rangle_L] \rangle)$			
(PROC-COM) $vcd.(\langle M[\text{send } c \ \{o\}] \rangle \mid \langle N[\text{recv } d] \rangle) \Rightarrow vcd.(\langle M[\text{return } c] \rangle \mid \langle N[\text{return } \langle \{o\}, d \rangle_L] \rangle)$			
(PROC-EXPR) $\frac{m \rightsquigarrow m'}{\langle m \rangle \Rightarrow \langle m' \rangle}$	(PROC-PAR) $\frac{P \Rightarrow Q}{O \mid P \Rightarrow O \mid Q}$	(PROC-SCOPE) $\frac{P \Rightarrow Q}{vcd.P \Rightarrow vcd.Q}$	(PROC-CONGR) $\frac{P \equiv P' \quad P' \Rightarrow Q' \quad Q' \equiv Q}{P \Rightarrow Q}$

5 Meta-Theory

In this section, we study the meta-theory of TLL_C . The results are classified into 4 categories.

- (1) **Compatibility**: the extensions of TLL_C are compatible with the underlying TLL theory.
- (2) **Session Fidelity**: processes follow the protocols specified by their session types.
- (3) **Global Progress**: well-typed reachable processes do not deadlock.
- (4) **Erasure Safety**: evaluation of erased terms and processes is safe.

5.1 Compatibility

To show that the TLL_C extensions made are compatible with the theory of TLL, we prove the TLL_C terms enjoy the same properties as TLL ones.

Confluence. We show that the logical reduction relation is confluent. This property is important as it ensures that type convertibility can be checked regardless of the order in which reductions are applied. Confluence is easy to prove here as the parallel reduction satisfies the diamond property.

LEMMA 5.1 (DIAMOND PROPERTY). *If $m \Rightarrow m_1$ and $m \Rightarrow m_2$, then there exists m' such that $m_1 \Rightarrow m'$ and $m_2 \Rightarrow m'$.*

PROOF. By induction on the structure of the parallel reduction. □

LEMMA 5.2. *If $m \Rightarrow m_1$ and $m \Rightarrow^* m_2$, then there exists m' such that $m_1 \Rightarrow^* m'$ and $m_2 \Rightarrow m'$.*

PROOF. By induction on the derivation of $m \Rightarrow^* m_2$ and Lemma 5.1. □

THEOREM 5.3 (CONFLUENCE). *If $m \Rightarrow^* m_1$ and $m \Rightarrow^* m_2$, then there exists m' such that $m_1 \Rightarrow^* m'$ and $m_2 \Rightarrow^* m'$.*

PROOF. By induction on the derivation of $m \Rightarrow^* m_2$ and Lemma 5.2. □

The validity of the confluence property allows us to prove the injectivity of the convertibility relation for types.

COROLLARY 5.4. $s_1 \simeq s_2$ implies $s_1 = s_2$.

COROLLARY 5.5. $\Pi_s\{x : A\}.B \simeq \Pi_{s'}\{x : A'\}.B'$ implies $s = s'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.6. $\Pi_s(x : A).B \simeq \Pi_{s'}(x : A').B'$ implies $s = s'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.7. $\Sigma_s\{x : A\}.B \simeq \Sigma_{s'}\{x : A'\}.B'$ implies $s = s'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.8. $\Sigma_s(x : A).B \simeq \Sigma_{s'}(x : A').B'$ implies $s = s'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.9. $C(A) \simeq C(B)$ implies $A \simeq B$.

COROLLARY 5.10. $\rho\{x : A\}.B \simeq \rho'\{x : A'\}.B'$ implies $\rho = \rho'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.11. $\rho(x : A).B \simeq \rho'(x : A').B'$ implies $\rho = \rho'$, $A \simeq A'$, and $B \simeq B'$.

COROLLARY 5.12. $\mathbf{ch}\langle A \rangle \simeq \mathbf{ch}\langle B \rangle$ implies $A \simeq B$.

COROLLARY 5.13. $\mathbf{hc}\langle A \rangle \simeq \mathbf{hc}\langle B \rangle$ implies $A \simeq B$.

Weakening. Weakening allows for the addition of unused variables to a typing context. The logical level type system allows weakening as it is a fully structural type system. On the other hand, the program level type system only allows weakening of unrestricted variables, i.e. variables whose types inhabit U .

LEMMA 5.14 (RENAMING ARITY). *Given renaming ξ , if there is A arity(**proto**), then there is $A[\xi]$ arity(**proto**).*

PROOF. By induction on the structure of A . □

LEMMA 5.15 (RENAMING GUARDED). *Given renaming ξ , if there is $\forall x, y, \xi(x) = \xi(y) \implies x = y$, then given variable x and A guard(x), there is $A[\xi]$ guard($\xi(x)$).*

PROOF. By induction on the structure of A . □

LEMMA 5.16 (LOGICAL WEAKENING). *If $\Gamma \vdash m : A$ and $\Gamma \vdash B : s$, then $\Gamma, x : B \vdash m : A$ where $x \notin \Gamma$.*

PROOF. By induction on the derivation of $\Gamma \vdash m : A$. For more details, see file `sta_weak.v` of our Rocq development which uses a De Bruijn indices representation for variables. □

LEMMA 5.17 (PROGRAM WEAKENING (EXPLICIT)). *If $\Theta; \Gamma; \Delta \vdash m : A$ and $\Gamma \vdash B : U$, then $\Theta; \Gamma, x : B; \Delta, x :_U B \vdash m : A$ where $x \notin \Gamma$.*

PROOF. By induction on the derivation of $\Theta; \Gamma; \Delta \vdash m : A$. For more details, see file `dyn_weak.v` of our Rocq development which uses a De Bruijn indices representation for variables. □

LEMMA 5.18 (PROGRAM WEAKENING (IMPLICIT)). *If $\Theta; \Gamma; \Delta \vdash m : A$ and $\Delta \vdash B : L$, then $\Theta; \Gamma, x : B; \Delta \vdash m : A$ where $x \notin \Delta$.*

PROOF. By induction on the derivation of $\Theta; \Gamma; \Delta \vdash m : A$. For more details, see file `dyn_weak.v` of our Rocq development which uses a De Bruijn indices representation for variables. □

Substitution. The substitution lemma at the logical level is standard as the logical type system is completely structural. The substitution lemma at the program level is more involved as it needs to track linear variables in the program context.

LEMMA 5.19 (SUBSTITUTION ARITY). *Given substitution σ , if there is A arity(**proto**), then there is $A[\sigma]$ arity(**proto**).*

PROOF. By induction on the structure of A . □

LEMMA 5.20 (SUBSTITUTION GUARDED). *Given substitution σ and variables x, y and term A , if there is $\forall z, x \neq z \implies (\sigma z)$ guard(y) and A guard(x), then $A[\sigma]$ guard(y).*

PROOF. By induction on the structure of A . □

LEMMA 5.21 (LOGICAL SUBSTITUTION). *If $\Gamma, x : B \vdash m : A$ and $\Gamma \vdash n : B$, then $\Gamma \vdash m[n/x] : A[n/x]$.*

PROOF. This lemma is proved through a more general lemma involving simultaneous substitutions. For more details, see file `sta_subst.v` of our Rocq development. □

COROLLARY 5.22. *If $\Gamma, x : A \vdash m : C$ and $A \simeq B$, then $\Gamma, x : B \vdash m : C$.*

LEMMA 5.23 (PROGRAM SUBSTITUTION (EXPLICIT)). *If $\Theta_1; \Gamma, x : B; \Delta_1, x :_s B \vdash m : A$ and $\Theta_2; \Gamma; \Delta_2 \vdash n : B$ and $\Theta_2 \triangleright s$ and $\Delta_2 \triangleright s$, then $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m[n/x] : A[n/x]$.*

PROOF. This lemma is proved through a more general lemma involving simultaneous substitutions. For more details, see file `dyn_subst.v` of our Rocq development. □

COROLLARY 5.24. *If $\Theta; \Gamma, x : A; \Delta \vdash m : C$ and $\Gamma \vdash B : s$ and $A \simeq B$, then $\Theta; \Gamma, x : B; \Delta \vdash m : C$.*

LEMMA 5.25 (PROGRAM SUBSTITUTION (IMPLICIT)). *If $\Theta; \Gamma, x : B; \Delta \vdash m : A$ and $\Gamma \vdash n : B$, then $\Theta; \Gamma; \Delta \vdash m[n/x] : A[n/x]$.*

PROOF. This lemma is proved through a more general lemma involving simultaneous substitutions. For more details, see file `dyn_subst.v` of our Rocq development. $\square$

COROLLARY 5.26. *If $\Theta; \Gamma, x : A; \Delta, x :_s A \vdash m : C$ and $A \simeq B$, then $\Theta; \Gamma, x : B; \Delta, x :_s B \vdash m : C$.*

Sort Uniqueness. Due to the fact that TLL_C utilizes the sort of types to determine (sub)structural properties of their inhabitants, it is important for types to have unique sorts. If a type could have multiple sorts, then it would be ambiguous whether its inhabitants are linear or unrestricted.

One of the main challenges in proving sort uniqueness is that there is no uniqueness of types in general. In particular, dependent pairs like $\langle \{m\}, n \rangle_s$ do not have unique typing. For this reason, we prove a weaker property of *type similarity* instead of type uniqueness. Then from type similarity we can derive sort uniqueness. We begin by defining the *head similarity* relation as follows:

$$\begin{array}{c}
\frac{}{\text{headSim}(x, x)} \quad \frac{}{\text{headSim}(s, s)} \quad \frac{\text{headSim}(B_1, B_2)}{\text{headSim}(\Pi_s(x : A_1).B_1, \Pi_s(x : A_2).B_2)} \\
\frac{\text{headSim}(B_1, B_2)}{\text{headSim}(\Pi_s\{x : A_1\}.B_1, \Pi_s\{x : A_2\}.B_2)} \quad \frac{}{\text{headSim}(\lambda_s(x : A).m, \lambda_s(x : A).m)} \\
\frac{}{\text{headSim}(\lambda_s\{x : A\}.m, \lambda_s\{x : A\}.m)} \quad \frac{}{\text{headSim}(m \ n, m \ n)} \quad \frac{}{\text{headSim}(m \ \{n\}, m \ \{n\})} \\
\frac{}{\text{headSim}(\Sigma_s(x : A_1).B_1, \Sigma_s(x : A_2).B_2)} \quad \frac{}{\text{headSim}(\Sigma_s\{x : A_1\}.B_1, \Sigma_s\{x : A_2\}.B_2)} \\
\frac{}{\text{headSim}(\langle m, n \rangle_s, \langle m, n \rangle_s)} \quad \frac{}{\text{headSim}(\langle \{m\}, n \rangle_s, \langle \{m\}, n \rangle_s)} \\
\frac{}{\text{headSim}(R_{[z]A}^\Sigma(m, [x, y]n), R_{[z]A}^\Sigma(m, [x, y]n))} \quad \frac{}{\text{headSim}(\mu(x : A).m, \mu(x : A).m)} \quad \frac{}{\text{headSim}(\text{unit}, \text{unit})} \\
\frac{}{\text{headSim}((), ())} \quad \frac{}{\text{headSim}(\text{bool}, \text{bool})} \quad \frac{}{\text{headSim}(\text{true}, \text{true})} \quad \frac{}{\text{headSim}(\text{false}, \text{false})} \\
\frac{}{\text{headSim}(R_{[z]A}^{\text{bool}}(m, n_1, n_2), R_{[z]A}^{\text{bool}}(m, n_1, n_2))} \quad \frac{}{\text{headSim}(C(A), C(B))} \quad \frac{}{\text{headSim}(\text{return } m, \text{return } m)} \\
\frac{}{\text{headSim}(\text{let } m \leftarrow x \text{ in } n, \text{let } m \leftarrow x \text{ in } n)} \quad \frac{}{\text{headSim}(\text{proto}, \text{proto})} \quad \frac{}{\text{headSim}(\mathbf{1}, \mathbf{1})} \\
\frac{}{\text{headSim}(\rho(x : A).B, \rho(x : A).B)} \quad \frac{}{\text{headSim}(\rho\{x : A\}.B, \rho\{x : A\}.B)} \quad \frac{}{\text{headSim}(\text{ch}\langle A \rangle, \text{ch}\langle B \rangle)} \\
\frac{}{\text{headSim}(\text{hc}\langle A \rangle, \text{hc}\langle B \rangle)} \quad \frac{}{\text{headSim}(c, c)} \quad \frac{}{\text{headSim}(\text{fork } (x : A) \text{ with } m, \text{fork } (x : A) \text{ with } m)} \\
\frac{}{\text{headSim}(\text{recv } m, \text{recv } m)} \quad \frac{}{\text{headSim}(\text{recv } m, \text{recv } m)} \quad \frac{}{\text{headSim}(\text{send } m, \text{send } m)} \\
\frac{}{\text{headSim}(\text{send } m, \text{send } m)} \quad \frac{}{\text{headSim}(\text{close } m, \text{close } m)} \quad \frac{}{\text{headSim}(\text{wait } m, \text{wait } m)}
\end{array}$$

We then define the *type similarity* relation as follows:

$$\text{sim}(A, B) \triangleq \exists A', B', A \simeq A' \wedge B \simeq B' \wedge \text{headSim}(A', B')$$

The similarity relation is naturally extended to typing contexts as follows:

$$\frac{}{\text{sim}(\epsilon, \epsilon)} \qquad \frac{\text{sim}(A, B) \quad \text{sim}(\Gamma_1, \Gamma_2)}{\text{sim}((\Gamma_1, x : A), (\Gamma_2, x : B))}$$

The (head) similarity relation enjoys the following properties.

LEMMA 5.27 (HEADSIM REFLEXIVE). *For any term A , there is $\text{headSim}(A, A)$.*

PROOF. By induction on the structure of A . □

LEMMA 5.28 (HEADSIM SYMMETRIC). *For any $\text{headSim}(A, B)$, there is $\text{headSim}(B, A)$.*

PROOF. By induction on the derivation of $\text{headSim}(A, B)$. □

LEMMA 5.29 (HEADSIM SUBSTITUTION). *Given substitution σ , if there is $\text{headSim}(A, B)$, then there is $\text{headSim}(A[\sigma], B[\sigma])$.*

PROOF. By induction on the derivation of $\text{headSim}(A, B)$. □

LEMMA 5.30 (SIM REFLEXIVE). *For any term A , there is $\text{sim}(A, A)$.*

PROOF. By the reflexivity of $\simeq$ and Lemma 5.27. □

LEMMA 5.31 (SIM TRANSITIVE LEFT). *For any $\text{sim}(A, B)$ and $B \simeq C$, there is $\text{sim}(A, C)$.*

PROOF. By the transitivity of $\simeq$. □

LEMMA 5.32 (SIM TRANSITIVE RIGHT). *For any $\text{sim}(A, B)$ and $A \simeq C$, there is $\text{sim}(C, B)$.*

PROOF. By the transitivity of $\simeq$. □

LEMMA 5.33 (SIM SYMMETRIC). *For any $\text{sim}(A, B)$, there is $\text{sim}(B, A)$.*

PROOF. By the symmetry of $\simeq$ and Lemma 5.28. □

LEMMA 5.34 (SIM SUBSTITUTION). *Given substitution σ , if there is $\text{sim}(A, B)$, then there is $\text{sim}(A[\sigma], B[\sigma])$.*

PROOF. By the substitutivity of $\simeq$ and Lemma 5.29. □

LEMMA 5.35 (SIM SORT INJECTIVE). *If $\text{sim}(s_1, s_2)$, then $s_1 = s_2$.*

PROOF. By the definition of similarity and Corollary 5.4. □

LEMMA 5.36 (TYPE SIMILARITY). *Given $\Gamma_1 \vdash m : A$ and $\Gamma_2 \vdash m : B$ and $\text{sim}(\Gamma_1, \Gamma_2)$, then $\text{sim}(A, B)$.*

PROOF. By induction on the derivation of $\Gamma_1 \vdash m : A$. □

THEOREM 5.37 (SORT UNIQUENESS). *Given $\Gamma \vdash m : s$ and $\Gamma \vdash m : t$, then $s = t$.*

PROOF. From Lemma 5.27 we have $\text{sim}(\Gamma, \Gamma)$. Then from Lemma 5.36 we have $\text{sim}(s, t)$. Finally from Lemma 5.35 we have $s = t$. □

Inversion. Due to the presence of type conversion, inversion lemmas are necessary to reason about typing derivations.

LEMMA 5.38. *If $\Gamma \vdash \Pi_s(x : A).B : C$, then there exists t such that $\Gamma, x : A \vdash B : t$ and $C \simeq s$.*

LEMMA 5.39. *If $\Gamma \vdash \Pi_s\{x : A\}.B : C$, then there exists t such that $\Gamma, x : A \vdash B : t$ and $C \simeq s$.*

LEMMA 5.40. *If $\Gamma \vdash \lambda_{s_1}(x : A_1).m : \Pi_{s_2}(x : A_2).B$, then $\Gamma, x : A_1 \vdash m : B$.*

LEMMA 5.41. *If $\Gamma \vdash \lambda_{s_1}\{x : A_1\}.m : \Pi_{s_2}\{x : A_2\}.B$, then $\Gamma, x : A_1 \vdash m : B$.*

LEMMA 5.42. *If $\Gamma \vdash \Sigma_t(x : A).B : C$, then there exists s, r such that $s \sqsubseteq t$ and $r \sqsubseteq t$ and $\Gamma \vdash A : s$ and $\Gamma, x : A \vdash B : r$ and $C \simeq t$.*

LEMMA 5.43. *If $\Gamma \vdash \Sigma_t\{x : A\}.B : C$, then there exists s, r such that $r \sqsubseteq t$ and $\Gamma \vdash A : s$ and $\Gamma, x : A \vdash B : r$ and $C \simeq t$.*

LEMMA 5.44. *If $\Gamma \vdash C(A) : B$, then there exists s such that $\Gamma \vdash A : s$ and $B \simeq L$.*

LEMMA 5.45. *If $\Gamma \vdash \mathbf{ch}\langle A \rangle : B$, then $\Gamma \vdash A : \mathbf{proto}$ and $B \simeq L$.*

LEMMA 5.46. *If $\Gamma \vdash \mathbf{hc}\langle A \rangle : B$, then $\Gamma \vdash A : \mathbf{proto}$ and $B \simeq L$.*

LEMMA 5.47. *If $\Gamma \vdash \rho(x : A).B : C$, then $\Gamma, x : A \vdash B : \mathbf{proto}$.*

LEMMA 5.48. *If $\Gamma \vdash \rho\{x : A\}.B : C$, then $\Gamma, x : A \vdash B : \mathbf{proto}$.*

LEMMA 5.49. *If $\Theta; \Gamma; \Delta \vdash \lambda_s(x : A_2).m : \Pi_t(x : A_1).B$, then $\Theta; \Gamma, x : A_1; \Delta, x :_r A_1 \vdash m : B$.*

LEMMA 5.50. *If $\Theta; \Gamma; \Delta \vdash m n : C$, then there exists $A, B, s, \Theta_1, \Theta_2, \Delta_1, \Delta_2$ such that $\Theta_1; \Gamma; \Delta_1 \vdash m : \Pi_s(x : A).B$ and $\Theta_2; \Gamma; \Delta_2 \vdash n : A$ and $\Theta = \Theta_1 \cup \Theta_2$ and $\Delta = \Delta_1 \cup \Delta_2$ and $C \simeq B[n/x]$.*

LEMMA 5.51. *If $\Theta; \Gamma; \Delta \vdash m \{n\} : C$, then there exists A, B, s such that $\Theta; \Gamma; \Delta \vdash m : \Pi_s\{x : A\}.B$ and $\Gamma \vdash n : A$ and $C \simeq B[n/x]$.*

LEMMA 5.52. *If $\Theta; \Gamma; \Delta \vdash \langle m, n \rangle_s : \Sigma_t(x : A).B$, then there exists $\Theta_1, \Theta_2, \Delta_1, \Delta_2$ such that $\Theta_1; \Gamma; \Delta_1 \vdash m : A$ and $\Theta_2; \Gamma; \Delta_2 \vdash n : B[m/x]$ and $\Theta = \Theta_1 \cup \Theta_2$ and $\Delta = \Delta_1 \cup \Delta_2$ and $s = t$.*

LEMMA 5.53. *If $\Theta; \Gamma; \Delta \vdash () : \text{unit}$, then $\Theta = \epsilon$ and $\Delta \triangleright U$.*

LEMMA 5.54. *If $\Theta; \Gamma; \Delta \vdash \text{true} : \text{bool}$, then $\Theta = \epsilon$ and $\Delta \triangleright U$.*

LEMMA 5.55. *If $\Theta; \Gamma; \Delta \vdash \text{false} : \text{bool}$, then $\Theta = \epsilon$ and $\Delta \triangleright U$.*

LEMMA 5.56. *If $\Theta; \Gamma; \Delta \vdash \mathbf{return} m : C(A)$, then $\Theta; \Gamma; \Delta \vdash m : A$.*

LEMMA 5.57. *If $\Theta; \Gamma; \Delta \vdash \mathbf{let} m \leftarrow x \mathbf{in} n : C(B)$, then there exists $A, \Theta_1, \Theta_2, \Delta_1, \Delta_2$ such that $\Theta_1; \Gamma; \Delta_1 \vdash m : C(A)$ and $\Theta_2; \Gamma, x : A; \Delta_2, x :_s A \vdash n : C(B)$ and $\Theta = \Theta_1 \cup \Theta_2$ and $\Delta = \Delta_1 \cup \Delta_2$ and $x \notin FV(B)$.*

LEMMA 5.58. *If $\Theta; \Gamma; \Delta \vdash \mathbf{fork} (x : A) \mathbf{with} m : B$, then there exists A' such that $\Gamma \vdash A' : \mathbf{proto}$ and $A = \mathbf{ch}\langle A' \rangle$ and $B \simeq C(\mathbf{hc}\langle A' \rangle)$ and $\Theta; \Gamma, x : \mathbf{ch}\langle A' \rangle; \Delta, x :_L \mathbf{ch}\langle A' \rangle \vdash m : C(\text{unit})$.*

LEMMA 5.59. *If $\Theta; \Gamma; \Delta \vdash \mathbf{send} m : C$, then there exists A, B such that either $C \simeq \Pi_L(x : A).C(\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{ch}\langle!(x : A). B \rangle$ or $C \simeq \Pi_L(x : A).C(\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{hc}\langle?(x : A). B \rangle$.*

LEMMA 5.60. *If $\Theta; \Gamma; \Delta \vdash \mathbf{send} m : C$, then there exists A, B such that either $C \simeq \Pi_L\{x : A\}.C(\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{ch}\langle!\{x : A\}. B \rangle$ or $C \simeq \Pi_L\{x : A\}.C(\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{hc}\langle?\{x : A\}. B \rangle$.*

LEMMA 5.61. *If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} m : C$, then there exists A, B such that either $C \simeq C(\Sigma_L(x : A).\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{ch}\langle?(x : A). B \rangle$ or $C \simeq C(\Sigma_L(x : A).\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{hc}\langle!(x : A). B \rangle$.*

LEMMA 5.62. *If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} \, m : C$, then there exists A, B such that either $C \simeq C(\Sigma_L\{x : A\}.\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{ch}\langle ?\{x : A\}. B \rangle$ or $C \simeq C(\Sigma_L\{x : A\}.\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m : \mathbf{hc}\langle !\{x : A\}. B \rangle$.*

LEMMA 5.63. *If $\Theta; \Gamma; \Delta \vdash \mathbf{close} \, m : A$, then $\Theta; \Gamma; \Delta \vdash m : \mathbf{ch}\langle \mathbf{1} \rangle$ and $A \simeq C(\mathbf{unit})$.*

LEMMA 5.64. *If $\Theta; \Gamma; \Delta \vdash \mathbf{wait} \, m : A$, then $\Theta; \Gamma; \Delta \vdash m : \mathbf{hc}\langle \mathbf{1} \rangle$ and $A \simeq C(\mathbf{unit})$.*

Type Validity. We show that all types appearing in typing judgments are valid, i.e. they are well-sorted at the logical level.

THEOREM 5.65 (LOGICAL TYPE VALIDITY). *If $\Gamma \vdash m : A$, then there exists s such that $\Gamma \vdash A : s$.*

PROOF. By induction on the derivation of $\Gamma \vdash m : A$. We will show some representative cases.

Case (VAR): From the premise we have $\Gamma, x : A \vdash$ which implies $\Gamma \vdash A : s$ for some s .

Case (EXPLICIT-LAM): From the induction hypothesis we have $\Gamma, x : A \vdash B : r$ for some r . The validity of context $\Gamma, x : A \vdash$ implies $\Gamma \vdash A : s$ for some s . Then from EXPLICIT-FUN we have $\Gamma \vdash \Pi_s(x : A).B : s$ which concludes this case.

Case (EXPLICIT-APP): From the induction hypothesis we have $\Gamma \vdash \Pi_s(x : A).B : r$ for some r . From Lemma 5.38 we have $\Gamma, x : A \vdash B : t$ for some t . By Lemma 5.21 we have $\Gamma \vdash B[n/x] : t$ which concludes this case.

Case (EXPLICIT-RECV-CH): From the induction hypothesis we have $\Gamma \vdash \mathbf{ch}\langle ?(x : A). B \rangle : s$ for some s . By Lemma 5.45 we have $\Gamma \vdash ?(x : A). B : \mathbf{proto}$. By Lemma 5.47 we have $\Gamma, x : A \vdash B : \mathbf{proto}$. From the validity of context $\Gamma, x : A \vdash$ we have $\Gamma \vdash A : t$ for some t . Applying CHTYPE, we have $\Gamma, x : A \vdash \mathbf{ch}\langle B \rangle : L$. Applying EXPLICIT-SUM, we have $\Gamma \vdash \Sigma_L(x : A).\mathbf{ch}\langle B \rangle : L$. Applying CTYPE, we have $\Gamma \vdash C(\Sigma_L(x : A).\mathbf{ch}\langle B \rangle) : L$ which concludes this case. $\square$

To show that the types appearing in program level typing judgments are valid, we first prove the lifting theorem which allows us to lift programs to the logical level.

THEOREM 5.66 (LIFTING). *If $\Theta; \Gamma; \Delta \vdash m : A$, then $\Gamma \vdash m : A$.*

PROOF. By induction on the derivation of $\Theta; \Gamma; \Delta \vdash m : A$. $\square$

THEOREM 5.67 (PROGRAM TYPE VALIDITY). *If $\Theta; \Gamma; \Delta \vdash m : A$, then there exists s such that $\Gamma \vdash A : s$.*

PROOF. Immediate from Theorem 5.66 and Theorem 5.65. $\square$

Subject Reduction. We show that both the logical and program level type systems enjoy subject reduction under logical and program reductions respectively.

LEMMA 5.68 (ARITY PRESERVATION). *If $A \Rightarrow A'$ and A arity(**proto**), then A' arity(**proto**).*

PROOF. By induction on the derivation of $A \Rightarrow A'$. $\square$

LEMMA 5.69 (GUARD PRESERVATION). *If $A \Rightarrow A'$ and A guard(x), then A' guard(x).*

PROOF. By induction on the derivation of $A \Rightarrow A'$ and appealing to Lemma 5.20. $\square$

THEOREM 5.70 (LOGICAL SUBJECT REDUCTION). *If $\Gamma \vdash m : A$ and $m \Rightarrow m'$, then $\Gamma \vdash m' : A$.*

PROOF. By induction on the derivation of $\Gamma \vdash m : A$ and case analysis on the reduction $m \Rightarrow m'$. We present the following representative cases.

Case (EXPLICIT-LAM): From case analysis on the reduction, we have $A \Rightarrow A'$ and $m \Rightarrow m'$. From the induction hypothesis we have $\Gamma, x : A \vdash m' : B$ and $\Gamma \vdash A' : s$ for some s . By definition of convertibility, we have $A \simeq A'$. By Corollary 5.22 we have $\Gamma, x : A' \vdash m' : B$. By EXPLICIT-LAM we have $\Gamma \vdash \lambda_s(x : A').m' : \Pi_s(x : A').B$. By CONVERSION we have $\Gamma \vdash \lambda_s(x : A').m' : \Pi_s(x : A).B$ which concludes this case.

Case (EXPLICIT-APP): From case analysis on the reduction we have two sub-cases: (1) PSTEP-EXPLICIT-APP and (2) PSTEP-EXPLICIT- β .

In sub-case (1) PSTEP-EXPLICIT-APP, we have $m \Rightarrow m'$ and $n \Rightarrow n'$. From the induction hypothesis we have $\Gamma \vdash m' : \Pi_s(x : A).B$ and $\Gamma \vdash n' : A$. By EXPLICIT-APP we have $\Gamma \vdash m' n' : B[n'/x]$. By definition of convertibility, we have $B[n/x] \simeq B[n'/x]$. By validity we have $\Gamma \vdash \Pi_s(x : A).B : t$ for some t . By Lemma 5.38 we have $\Gamma, x : A \vdash B : r$ for some r . By Lemma 5.21 we have $\Gamma \vdash B[n/x] : r$. By CONVERSION we have $\Gamma \vdash m' n' : B[n/x]$ which concludes this sub-case.

In sub-case (2) PSTEP-EXPLICIT- β , we have $m = \lambda_s(x : A).m_0$ for some m_0 and $m_0 \Rightarrow m'_0$ and $n \Rightarrow n'$. By Lemma 5.40 we have $\Gamma, x : A \vdash m_0 : B$. By the induction hypothesis we have $\Gamma, x : A \vdash m'_0 : B$ and $\Gamma \vdash n' : A$. By Lemma 5.21 we have $\Gamma \vdash m'_0[n'/x] : B[n'/x]$. By definition of convertibility, we have $B[n/x] \simeq B[n'/x]$. By validity we have $\Gamma \vdash \Pi_s(x : A).B : t$ for some t . By Lemma 5.38 we have $\Gamma, x : A \vdash B : r$ for some r . By Lemma 5.21 we have $\Gamma \vdash B[n/x] : r$. By CONVERSION we have $\Gamma \vdash m'_0[n'/x] : B[n/x]$ which concludes this sub-case.

Case (BOOLELIM) From case analysis on the reduction we have three sub-cases: (1) PSTEP-BOOLELIM, (2) PSTEP-TRUEELIM, and (3) PSTEP-FALSEELIM.

In sub-case (1) PSTEP-BOOLELIM, we have $A \Rightarrow A'$, $m \Rightarrow m'$, $n_1 \Rightarrow n'_1$, and $n_2 \Rightarrow n'_2$. By the induction hypothesis we have $\Gamma, z : \text{bool} \vdash A' : s$, $\Gamma \vdash m' : \text{bool}$, $\Gamma \vdash n'_1 : A[\text{true}/z]$, and $\Gamma \vdash n'_2 : A[\text{false}/z]$. By definition of convertibility, we have $A \simeq A'$. By Lemma 5.21 we have $\Gamma \vdash A'[\text{true}/z] : s$ and $\Gamma \vdash A'[\text{false}/z] : s$. By CONVERSION we have $\Gamma \vdash n'_1 : A'[\text{true}/z]$ and $\Gamma \vdash n'_2 : A'[\text{false}/z]$. By BOOLELIM we have $\Gamma \vdash R_{[z]A'}^{\text{bool}}(m', n'_1, n'_2) : A'[m'/z]$. By definition of convertibility, we have $A[m/z] \simeq A'[m'/z]$. By Lemma 5.21 we have $\Gamma \vdash A[m/z] : s$. By CONVERSION we have $\Gamma \vdash R_{[z]A'}^{\text{bool}}(m', n'_1, n'_2) : A[m/z]$ which concludes this sub-case.

Case (RECPROTO): From case analysis on the reduction we have two sub-cases: (1) PSTEP-RECPROTO and (2) PSTEP-RECUNFOLD.

In sub-case (1) PSTEP-RECPROTO, we have $A \Rightarrow A'$ and $m \Rightarrow m'$. By validity of context $\Gamma, x : A \vdash$ we have $\Gamma \vdash A : s$ for some s . By the induction hypothesis we have $\Gamma, x : A \vdash m' : A$ and $\Gamma \vdash A' : s$. By definition of convertibility, we have $A \simeq A'$. By Corollary 5.22 we have $\Gamma, x : A' \vdash m' : A$. By CONVERSION we have $\Gamma, x : A' \vdash m' : A'$. By Lemma 5.68 we have A' arity(**proto**). By Lemma 5.69 we have m' guard(x). By RECPROTO we have $\Gamma \vdash \mu(x : A').m' : A'$. By CONVERSION we have $\Gamma \vdash \mu(x : A').m' : A$ which concludes this sub-case.

In sub-case (2) PSTEP-RECUNFOLD, we have $A \Rightarrow A'$ and $m \Rightarrow m'$. By validity of context $\Gamma, x : A \vdash$ we have $\Gamma \vdash A : s$ for some s . By the induction hypothesis we have $\Gamma, x : A \vdash m' : A$ and $\Gamma \vdash A' : s$. By definition of convertibility, we have $A \simeq A'$. By Corollary 5.22 we have $\Gamma, x : A' \vdash m' : A$. By CONVERSION we have $\Gamma, x : A' \vdash m' : A'$. By Lemma 5.68 we have A' arity(**proto**). By Lemma 5.69 we have m' guard(x). By RECPROTO we have $\Gamma \vdash \mu(x : A').m' : A'$. By Lemma 5.21 we have $\Gamma \vdash m'[\mu(x : A').m'/x] : A'$. By CONVERSION we have $\Gamma \vdash m'[\mu(x : A').m'/x] : A$ which concludes this sub-case. $\square$

In order to show subject reduction at the program level, we need to show that reduction of redexes in dependent positions preserves typing. To do so, we prove the following lemma which lifts program reductions into convertibility at the logical level.

LEMMA 5.71 (PROGRAM STEP CONVERTIBLE). *If $\Theta; \epsilon \vdash m : A$ and $m \rightsquigarrow n$, then $m \simeq n$.*

The program level substitution lemma (Lemma 5.23) requires context restrictions $\Theta_2 \triangleright s$ and $\Delta_2 \triangleright s$ for the substituted term n . To ensure that these restrictions are satisfied for *values*, we prove the following context bound lemma.

LEMMA 5.72 (PROGRAM CONTEXT BOUND). *Given $\Theta; \Gamma; \Delta \vdash v : A$ and $\Gamma \vdash A : s$, then $\Theta \triangleright s$ and $\Delta \triangleright s$.*

PROOF. By induction on the derivation of $\Theta; \Gamma; \Delta \vdash v : A$ where v is a value. We present the following representative cases.

Case (EXPLICIT-LAM): From the premise we have $\Theta \triangleright t$ and $\Delta \triangleright t$. By Lemma 5.38 we have $t \simeq s$. By injectivity of sorts (Corollary 5.4) we have $t = s$ which concludes this case.

Case (EXPLICIT-PAIR): From the assumption that the pair is a value, we have $v = \langle v_1, v_2 \rangle_t$ for some t . Additionally, we have $\Theta_1; \Gamma; \Delta_1 \vdash v_1 : A$ and $\Theta_2; \Gamma; \Delta_2 \vdash v_2 : B[v_1/x]$ and $\Gamma \vdash \Sigma_t(x : A).B : t$. By Theorem 5.37 we have $s = t$. By Lemma 5.42 we have $\Gamma \vdash A : s$ and $\Gamma, x : A \vdash B : r$ and $s \sqsubseteq t$ and $r \sqsubseteq t$. By the induction hypothesis we have $\Theta_1 \triangleright s$ and $\Delta_1 \triangleright s$ and $\Theta_2 \triangleright r$ and $\Delta_2 \triangleright r$. These context restrictions can then be weakened to $\Theta_1 \triangleright t$ and $\Delta_1 \triangleright t$ and $\Theta_2 \triangleright t$ and $\Delta_2 \triangleright t$. The merged contexts now satisfy $\Theta_1 \cup \Theta_2 \triangleright t$ and $\Delta_1 \cup \Delta_2 \triangleright t$ which concludes this case.

Case (RETURN): From the premise we have $\Theta; \Gamma; \Delta \vdash m : A$ and $\Gamma \vdash C(A) : s$. By Theorem 5.37 we have $s = L$. The context restrictions $\Theta \triangleright L$ and $\Delta \triangleright L$ hold trivially which concludes this case.

Case (CHANNEL-CH) From the premise we have $\Theta \triangleright U$. From ORD-U we have $U \sqsubseteq s$ which allows us to weaken the restriction into $\Theta \triangleright s$ and concluding this case.

Case (EXPLICIT-SEND-CH): From the premise we have $\Gamma \vdash \Pi_L(x : A).C(\mathbf{ch}\langle B \rangle) : s$. By Theorem 5.37 we have $s = L$. The context restrictions $\Theta \triangleright L$ and $\Delta \triangleright L$ hold trivially. $\square$

THEOREM 5.73 (PROGRAM SUBJECT REDUCTION). *If $\Theta; \epsilon; \epsilon \vdash m : A$ and $m \rightsquigarrow m'$, then $\Theta; \epsilon; \epsilon \vdash m' : A$.*

PROOF. By induction on the derivation of $\Theta; \epsilon; \epsilon \vdash m : A$ and case analysis on the reduction $m \rightsquigarrow m'$. We present the following representative cases.

Case (EXPLICIT-APP): From case analysis on the reduction we have three sub-cases: (1) STEP-EXPLICIT-APP₁, (2) STEP-EXPLICIT-APP₂, and (3) STEP-EXPLICIT- β .

In sub-case (1) STEP-EXPLICIT-APP₁, we have $m \rightsquigarrow m'$. By the induction hypothesis we have $\Theta_1; \Gamma; \Delta_1 \vdash m' : \Pi_t(x : A).B$. By EXPLICIT-APP we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m' n : B[n/x]$ which concludes this sub-case.

In sub-case (2) STEP-EXPLICIT-APP₂, we have $n \rightsquigarrow n'$. By the induction hypothesis we have $\Theta_2; \Gamma; \Delta_2 \vdash n' : A$. By EXPLICIT-APP we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m n' : B[n'/x]$. By Lemma 5.71 we have $n \simeq n'$ and $B[n/x] \simeq B[n'/x]$. By Theorem 5.66 we have $\Gamma \vdash n : A$. Applying Theorem 5.67 on $\Theta_1; \Gamma; \Delta_1 \vdash m : \Pi_t(x : A).B$ we have $\Gamma \vdash \Pi_t(x : A).B : s$ for some s . By Lemma 5.38 we have $\Gamma, x : A \vdash B : r$ for some r . By Lemma 5.21 we have $\Gamma \vdash B[n/x] : r$. By CONVERSION we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m n' : B[n/x]$ which concludes this sub-case.

In sub-case (3) STEP-EXPLICIT- β , we have $m = \lambda_t(x : A_0).m_0$ for some A_0 and m_0 . By Lemma 5.49 we have $\Theta_1; \Gamma, x : A; \Delta_1, x : r \vdash m_0 : B$. From the validity of context $\Theta_1; \Gamma, x : A; \Delta_1, x : r \vdash$ we have $\Gamma \vdash A : r$. By Lemma 5.72 and $\Theta_2; \Gamma; \Delta_2 \vdash v : A$ and $\Gamma \vdash A : r$ we have $\Theta_2 \triangleright r$. By Lemma 5.23 we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m_0[v/x] : B[v/x]$ which concludes this sub-case.

Case (EXPLICIT-SUMELIM): From case analysis on the reduction we have two sub-cases: (1) STEP-SUMELIM₁ and (2) STEP-EXPLICIT-PAIRELIM.

In sub-case (1) STEP-SUMELIM₁, we have $m \rightsquigarrow m'$. By the induction hypothesis we have $\Theta_1; \Gamma; \Delta_1 \vdash m' : \Sigma_t(x : A).B$. By EXPLICIT-SUMELIM we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m', [x, y]n) : C[m'/z]$. By Lemma 5.71 we have $m \simeq m'$ and $C[m/z] \simeq C[m'/z]$. By Theorem 5.66 we have $\Gamma \vdash m : \Sigma_t(x : A).B$. Applying Lemma 5.21 on assumption $\Gamma, z : \Sigma_t(x : A).B \vdash C : s$ and $\Gamma \vdash m : \Sigma_t(x : A).B$ we have $\Gamma \vdash C[m/z] : s$. By CONVERSION we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m', [x, y]n) : C[m/z]$ which concludes this sub-case.

In sub-case (2) STEP-EXPLICIT-PAIRELIM, we have $m = \langle u, v \rangle_t$ for some u, v and t . By Lemma 5.52 we have $\Theta_{11}; \Gamma; \Delta_{11} \vdash u : A$ and $\Theta_{12}; \Gamma; \Delta_{12} \vdash v : B[u/x]$ and $\Theta_1 = \Theta_{11} \cup \Theta_{12}$ and $\Delta_1 = \Delta_{11} \cup \Delta_{12}$ and $s = t$. From the validity of context $\Theta_2; \Gamma, x : A, y : B; \Delta_2, x : r_1 \vdash$ we have $\Gamma \vdash A : r_1$ and $\Gamma, x : A \vdash B : r_2$. Applying Theorem 5.66 to $\Theta_{11}; \Gamma; \Delta_{11} \vdash u : A$ we have $\Gamma \vdash u : A$. Applying Lemma 5.21 we have $\Gamma \vdash B[u/x] : r_2$. By Lemma 5.72 and $\Theta_{11}; \Gamma; \Delta_{11} \vdash u : A$ and $\Gamma \vdash A : r_1$ we have $\Theta_{11} \triangleright r_1$. By

Lemma 5.72 and $\Theta_{12}; \Gamma; \Delta_{12} \vdash v : B[u/x]$ and $\Gamma \vdash B[u/x] : r_2$ we have $\Theta_{12} \triangleright r_2$. By Lemma 5.23 we have $\Theta_{11} \cup \Theta_{12} \cup \Theta_2; \Gamma; \Delta_{11} \cup \Delta_{12} \cup \Delta_2 \vdash n[u/x, v/y] : C[\langle u, v \rangle_t/z]$ which concludes this sub-case.

Case (BOOLELIM): By case analysis on the reduction we have three sub-cases: (1) STEP-BOOLELIM₁, (2) STEP-TRUEELIM, and (3) STEP-FALSEELIM.

In sub-case (1) STEP-BOOLELIM₁, we have $m \rightsquigarrow m'$. By the induction hypothesis we have $\Theta_1; \Gamma; \Delta_1 \vdash m' : \text{bool}$. By BOOLELIM we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^{\text{bool}}(m', n_1, n_2) : C[m'/z]$. By Lemma 5.71 we have $m \simeq m'$ and $C[m/z] \simeq C[m'/z]$. By Theorem 5.66 we have $\Gamma \vdash m : \text{bool}$. Applying Lemma 5.21 on $\Gamma, z : \text{bool} \vdash C : s$ and $\Gamma \vdash m : \text{bool}$ we have $\Gamma \vdash C[m/z] : s$. By CONVERSION we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^{\text{bool}}(m', n_1, n_2) : C[m/z]$ which concludes this sub-case.

In sub-case (2) STEP-TRUEELIM, we have $m = \text{true}$. By Lemma 5.54 we have $\Theta_1 = \epsilon$ and $\Delta_1 \triangleright U$. Thus we have $\Theta_1 \cup \Theta_2 = \Theta_2$ and $\Delta_1 \cup \Delta_2 = \Delta_2$. The assumption $\Theta_2; \Gamma; \Delta_2 \vdash n_1 : C[\text{true}/z]$ gives us the desired result which concludes this sub-case.

In sub-case (3) STEP-FALSEELIM, we have $m = \text{false}$. By Lemma 5.55 we have $\Theta_1 = \epsilon$ and $\Delta_1 \triangleright U$. Thus we have $\Theta_1 \cup \Theta_2 = \Theta_2$ and $\Delta_1 \cup \Delta_2 = \Delta_2$. The assumption $\Theta_2; \Gamma; \Delta_2 \vdash n_2 : C[\text{false}/z]$ gives us the desired result which concludes this sub-case.

Case (BIND): By case analysis on the reduction we have two sub-cases: (1) STEP-BIND and (2) STEP-RETURNBIND.

In sub-case (1) STEP-BIND, we have $m \rightsquigarrow m'$. By the induction hypothesis we have $\Theta_1; \Gamma; \Delta_1 \vdash m' : C(A)$. By BIND we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash \text{let } x \leftarrow m' \text{ in } n : C$ which concludes this sub-case.

In sub-case (2) STEP-RETURNBIND, we have $m = \text{return } v$ for some value v . By Lemma 5.56 we have $\Theta_1; \Gamma; \Delta_1 \vdash v : A$. From the validity of context $\Theta_2; \Gamma, x : A; \Delta_2, x :_r A \vdash$ we have $\Gamma \vdash A : r$. By Lemma 5.72 and $\Theta_1; \Gamma; \Delta_1 \vdash v : A$ and $\Gamma \vdash A : r$ we have $\Theta_1 \triangleright r$. By Lemma 5.23 we have $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash n[v/x] : C(B[v/x])$. From assumption $\Gamma \vdash B : s$ we know that $x \notin \text{FV}(B)$, thus $B[v/x] = B$ and $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash n[v/x] : C(B)$. which concludes this sub-case. $\square$

Progress. Due to the presence of concurrency primitives, the values that the program level terms can reduce to are not necessarily canonical forms. They can also be thunked monadic computations. These thunked computations will eventually be reduced by the semantics of the process level.

The following canonical forms lemmas are used to prove program progress (Theorem 5.81). They are proved by induction on the typing derivation of the value.

LEMMA 5.74. *If $\Theta; \epsilon; \epsilon \vdash v : \Pi_s(x : A).B$ then $v = \lambda_s(x : A).m$ or $v = \text{send } u$.*

LEMMA 5.75. *If $\Theta; \epsilon; \epsilon \vdash v : \Pi_s\{x : A\}.B$ then $v = \lambda_s\{x : A\}.m$ or $v = \text{send } u$.*

LEMMA 5.76. *If $\Theta; \epsilon; \epsilon \vdash v : \Sigma_s(x : A).B$ then $v = \langle v_1, v_2 \rangle_s$.*

LEMMA 5.77. *If $\Theta; \epsilon; \epsilon \vdash v : \Sigma_s\{x : A\}.B$ then $v = \langle \{m\}, v_2 \rangle_s$.*

LEMMA 5.78. *If $\Theta; \epsilon; \epsilon \vdash v : \text{unit}$ then $v = ()$.*

LEMMA 5.79. *If $\Theta; \epsilon; \epsilon \vdash v : \text{bool}$ then $v = \text{true}$ or $v = \text{false}$.*

LEMMA 5.80. *If $\Theta; \epsilon; \epsilon \vdash v : C(A)$ then $v = \text{return } u$ or v is a thunk.*

THEOREM 5.81 (PROGRAM PROGRESS). *If $\Theta; \epsilon; \epsilon \vdash m : A$, then m is a value or there exists m' such that $m \rightsquigarrow m'$.*

PROOF. By induction on the derivation of $\Theta; \epsilon; \epsilon \vdash m : A$. We present the following cases.

Case (VAR): Impossible since the context is empty.

Case (EXPLICIT-LAM): Trivial since $\lambda_t(x : A).m_0$ is a value.

Case (EXPLICIT-APP): By the induction hypothesis we have that either m is a value or there exists m' such that $m \rightsquigarrow m'$. If $m \rightsquigarrow m'$, then we are done by STEP-EXPLICIT-APP₁. If m is a value, by Lemma 5.74 we have two sub-cases: (1) $m = \lambda_t(x : A).m_0$ and (2) $m = \text{send } u$.

In sub-case (1) $m = \lambda_t(x : A).m_0$, by the induction hypothesis on n we have that either n is a value or there exists n' such that $n \rightsquigarrow n'$. If $n \rightsquigarrow n'$, then we are done by STEP-EXPLICIT-APP₂. If n is a value, then we are done by STEP-EXPLICIT- β .

In sub-case (2) $m = \mathbf{send} \ u$, by the induction hypothesis on n we have that either n is a value or there exists n' such that $n \rightsquigarrow n'$. If $n \rightsquigarrow n'$, then we are done by STEP-EXPLICIT-APP₂. If n is a value v , then we are done as $\mathbf{send} \ u \ v$ is a value.

Case (EXPLICIT-PAIR): By assumption we have $\Theta_1; \Gamma; \Delta_1 \vdash m_1 : A$ and $\Theta_2; \Gamma; \Delta_2 \vdash m_2 : B[m_1/x]$. From the induction hypothesis we have that either m_1 is a value or there exists m'_1 such that $m_1 \rightsquigarrow m'_1$. If $m_1 \rightsquigarrow m'_1$, then we are done by STEP-EXPLICIT-PAIR₁. If m_1 is a value u , then by the induction hypothesis on m_2 we have that either m_2 is a value or there exists m'_2 such that $m_2 \rightsquigarrow m'_2$. If $m_2 \rightsquigarrow m'_2$, then we are done by STEP-EXPLICIT-PAIR₂. If m_2 is a value v , then we are done since $\langle u, v \rangle_t$ is a value.

Case (EXPLICIT-SUMELIM): By the induction hypothesis we have that either m is a value or there exists m' such that $m \rightsquigarrow m'$. If $m \rightsquigarrow m'$, then we are done by STEP-SUMELIM₁. If m is a value, by Lemma 5.76 we have $m = \langle u, v \rangle_t$ for some u, v and t . We are done by STEP-EXPLICIT-PAIRELIM.

Case (BOOLELIM): By the induction hypothesis we have that either m is a value or there exists m' such that $m \rightsquigarrow m'$. If $m \rightsquigarrow m'$, then we are done by STEP-BOOLELIM₁. If m is a value, by Lemma 5.79 we have two sub-cases: (1) $m = \mathbf{true}$ and (2) $m = \mathbf{false}$. In sub-case (1) $m = \mathbf{true}$, we are done by STEP-TRUEELIM. In sub-case (2) $m = \mathbf{false}$, we are done by STEP-FALSEELIM.

Case (RETURN): By the induction hypothesis we have that either m is a value or there exists m' such that $m \rightsquigarrow m'$. If $m \rightsquigarrow m'$, then we are done by STEP-RETURN. If m is a value, then $\mathbf{return} \ m$ is a value.

Case (BIND): By the induction hypothesis we have that either m is a value or there exists m' such that $m \rightsquigarrow m'$. If $m \rightsquigarrow m'$, then we are done by STEP-BIND. If m is a value, then $\mathbf{let} \ x \leftarrow m \ \mathbf{in} \ n$ is a value.

For the session typing rules, the term m is a thunked computation and thus a value. $\square$

5.2 Session Fidelity

The session fidelity property ensures that well-typed processes will adhere to the protocols specified by their session types during execution. To prove this property, we first must prove that structural congruence preserves typing.

LEMMA 5.82 (CONGRUENCE). *Given $\Theta \Vdash P$ and $P \equiv Q$, then $\Theta \Vdash Q$.*

PROOF. By induction on the derivation of $\Theta \Vdash P$ and case analysis on the congruence relation.

Case (EXPR): It suffices to consider the case $\langle m \rangle \equiv (\langle m \rangle \mid \langle \mathbf{return} \ () \rangle)$, i.e. $Q = \langle m \rangle \mid \langle \mathbf{return} \ () \rangle$. Since $\Theta \Vdash \langle m \rangle$ and $\epsilon \Vdash \langle \mathbf{return} \ () \rangle$, we have $\Theta \cup \epsilon \Vdash \langle m \rangle \mid \langle \mathbf{return} \ () \rangle$ by PAR.

Case (PAR):

$$\frac{\text{PAR} \quad \Theta_1 \Vdash P \quad \Theta_2 \Vdash Q}{\Theta_1 \cup \Theta_2 \Vdash P \mid Q}$$

By case analysis on the congruence relation we have the following sub-cases:

- (1) $P \mid Q \equiv Q \mid P$
- (2) $P \mid (Q_1 \mid Q_2) \equiv (P \mid Q_1) \mid Q_2$
- (3) $P \mid \langle \mathbf{return} \ () \rangle \equiv P$
- (4) $\mathbf{vcd}.P \mid Q \equiv \mathbf{vcd}.(P \mid Q)$

In sub-case (1), by PAR we have $\Theta_2 \cup \Theta_1 \Vdash Q \mid P$. By the commutativity of $\cup$, we have $\Theta_1 \cup \Theta_2 \Vdash Q \mid P$ which concludes this sub-case.

In sub-case (2), we have $\Theta_2 \Vdash Q_1 \mid Q_2$. By inversion on its typing derivation, we have $\Theta_{21} \Vdash Q_1$ and $\Theta_{22} \Vdash Q_2$ such that $\Theta_2 = \Theta_{21} \cup \Theta_{22}$. By PAR we have $\Theta_1 \cup \Theta_{21} \Vdash P \mid Q_1$. By PAR again we have $(\Theta_1 \cup \Theta_{21}) \cup \Theta_{22} \Vdash (P \mid Q_1) \mid Q_2$. By the associativity of $\cup$, we have $\Theta_1 \cup (\Theta_{21} \cup \Theta_{22}) \Vdash (P \mid Q_1) \mid Q_2$. By substituting Θ_2 , we have $\Theta_1 \cup \Theta_2 \Vdash (P \mid Q_1) \mid Q_2$ which concludes this sub-case.

In sub-case (3), we have $Q = \langle \text{return} () \rangle$. By assumption we have $\Theta_2 \Vdash \langle \text{return} () \rangle$. By inversion on its typing derivation, we have $\Theta_2; \epsilon; \epsilon \vdash \text{return} () : C(\text{unit})$. By Lemma 5.56 we have $\Theta_2; \epsilon; \epsilon \vdash () : \text{unit}$. By Lemma 5.53 we have $\Theta_2 = \epsilon$ and $\epsilon \triangleright U$. Thus we have $\Theta_1 \cup \Theta_2 \Vdash P$.

In sub-case (4), we have $\Theta_1 \Vdash \text{vcd}.P$ and $\Theta_2 \Vdash Q$ by assumption. By inversion on the typing derivation of $\Theta_1 \Vdash \text{vcd}.P$, we have $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P$ for some protocol A . By PAR we have $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \cup \Theta_2 \Vdash P \mid Q$. Since c and d are not in Θ_2 , we have $(\Theta_1 \cup \Theta_2), c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \mid Q$. By SCOPE we have $\Theta_1 \cup \Theta_2 \Vdash \text{vcd}.(P \mid Q)$ which concludes this sub-case.

Case (SCOPE):

$$\frac{\text{SCOPE} \quad \Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P}{\Theta \Vdash \text{vcd}.P}$$

By case analysis on the congruence relation we have the following sub-cases:

- (1) $\text{vcd}.(P \mid Q) \equiv \text{vcd}.P \mid Q$
- (2) $\text{vcd}.P \equiv \text{vdc}.P$
- (3) $\text{vcd}.vc'd'.P \equiv vc'd'.\text{vcd}.P$

In sub-case (1), we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \mid Q$ by assumption. By inversion on its typing derivation, there exists Θ_1 and Θ_2 such that $\Theta = \Theta_1 \cup \Theta_2$ and $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P$ and $\Theta_2 \Vdash Q$. Channels c and d must be distributed the typing judgment of P as they are linear and do not appear in Q . Applying SCOPE to $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P$ we have $\Theta_1 \Vdash \text{vcd}.P$. By PAR we have $\Theta_1 \cup \Theta_2 \Vdash \text{vcd}.P \mid Q$ which concludes this sub-case.

In sub-case (2), we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P$ by assumption. By exchange of the context, we have $\Theta, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash P$. By SCOPE we have $\Theta \Vdash \text{vdc}.P$ which concludes this sub-case.

In sub-case (3), we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle, c' :_{\mathbb{L}} \mathbf{ch}\langle A' \rangle, d' :_{\mathbb{L}} \mathbf{hc}\langle A' \rangle \Vdash P$ by assumption. By exchange, we have $\Theta, c' :_{\mathbb{L}} \mathbf{ch}\langle A' \rangle, d' :_{\mathbb{L}} \mathbf{hc}\langle A' \rangle, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P$. Applying SCOPE twice we have $\Theta \Vdash vc'd'.\text{vcd}.P$ which concludes this sub-case. $\square$

To pull terms out of evaluation contexts, we need the following lemma.

LEMMA 5.83. *If $\Theta; \epsilon; \epsilon \vdash \mathcal{M}[m] : C(B)$, then there exists Θ_1, Θ_2, A such that $\Theta_1; \epsilon; \epsilon \vdash m : C(A)$ and $\Theta = \Theta_1 \cup \Theta_2$. For any Θ_3, n such that $\Theta_3; \epsilon; \epsilon \vdash n : C(A)$ and $\Theta_2 \cup \Theta_3$ is well-defined, we have $\Theta_2 \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{M}[n] : C(B)$.*

PROOF. By induction on the structure of $\mathcal{M}$.

Case ($\mathcal{M} = [\cdot]$): Trivial by choosing $\Theta_1 = \Theta$, $\Theta_2 = \epsilon$ and $A = B$.

Case ($\mathcal{M} = \text{let } x \Leftarrow \mathcal{N} \text{ in } n$): We have the typing judgment

$$\Theta; \epsilon; \epsilon \vdash \text{let } x \Leftarrow \mathcal{N}[m] \text{ in } n : C(B)$$

By Lemma 5.57 there exists Θ'_1, Θ'_2, A' such that

$$\begin{aligned} \Theta'_1; \epsilon; \epsilon \vdash \mathcal{N}[m] : C(A') \\ \Theta'_2; x : A'; x :_r A' \vdash n : C(B) \\ \Theta = \Theta'_1 \cup \Theta'_2 \quad x \notin \text{FV}(B) \end{aligned}$$

By the induction hypothesis on $\Theta'_1; \epsilon; \epsilon \vdash \mathcal{N}[m] : C(A')$, there exists $\Theta'_{11}, \Theta'_{12}, A''$ such that

$$\begin{aligned}\Theta'_{11}; \epsilon; \epsilon \vdash m : A'' \\ \Theta'_1 = \Theta'_{11} \cup \Theta'_{12}\end{aligned}$$

and for any Θ_3, n' such that $\Theta_3; \epsilon; \epsilon \vdash n' : A''$ and $\Theta'_{12} \cup \Theta_3$ is well-defined, we have

$$\Theta'_{12} \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{N}[n'] : C(A')$$

By choosing $\Theta_1 = \Theta'_{11}$, $\Theta_2 = \Theta'_{12} \cup \Theta'_2$ and $A = A''$, we have $\Theta_1; \epsilon; \epsilon \vdash m : A$ and $\Theta = \Theta_1 \cup \Theta_2$.

For any Θ_3, n' such that $\Theta_3; \epsilon; \epsilon \vdash n' : A$ and $\Theta_2 \cup \Theta_3$ is well-defined, we know that $\Theta'_{12} \cup \Theta_3$ is well-defined, which means that

$$\Theta'_{12} \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{N}[n'] : C(A')$$

Now applying BIND we have

$$\Theta_2 \cup \Theta_3; \epsilon; \epsilon \vdash \mathbf{let } x \leftarrow \mathcal{N}[n'] \mathbf{ in } n : C(B)$$

which concludes this case. $\square$

THEOREM 5.84 (SESSION FIDELITY). *If $\Theta \Vdash P$ and $P \Rightarrow Q$, then $\Theta \Vdash Q$.*

PROOF. By induction on the derivation of $P \Rightarrow Q$ and case analysis on the typing judgment.

Case (PROC-FORK):

$$\langle \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with } m] \rangle \Rightarrow vcd.(\langle \mathcal{N}[\mathbf{return } c] \rangle \mid \langle m[d/x] \rangle)$$

By inversion on $\Theta \Vdash \langle \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with } m] \rangle$ we have

$$\Theta; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with } m] : C(\mathbf{unit})$$

By Lemma 5.83 we have Θ_1, Θ_2, B such that

$$\Theta_1; \epsilon; \epsilon \vdash \mathbf{fork}(x : A) \mathbf{with } m : C(B)$$

and $\Theta = \Theta_1 \cup \Theta_2$ and for any Θ_3, n such that $\Theta_3; \epsilon; \epsilon \vdash n : C(B)$ and $\Theta_2 \cup \Theta_3$ is well-defined, we have $\Theta_2 \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{N}[n] : C(\mathbf{unit})$.

Applying Lemma 5.58 to $\Theta_1; \epsilon; \epsilon \vdash \mathbf{fork}(x : A) \mathbf{with } m : C(B)$ we have

$$\Theta_1; x : \mathbf{ch}\langle A' \rangle; x :_t \mathbf{ch}\langle A' \rangle \vdash m : C(\mathbf{unit})$$

and $A = \mathbf{ch}\langle A' \rangle$ and $C(B) \simeq C(\mathbf{hc}\langle A' \rangle)$. By the validity of context $\Theta_2; x : \mathbf{ch}\langle A' \rangle; x :_t \mathbf{ch}\langle A' \rangle \vdash$ we have $\epsilon \vdash \mathbf{ch}\langle A' \rangle : t$. By CHTYPE and Theorem 5.37 we have $t = L$. Applying Corollary 5.9 to $C(B) \simeq C(\mathbf{hc}\langle A' \rangle)$ we have $B \simeq \mathbf{hc}\langle A' \rangle$.

By CHANNEL-HC, for some fresh channel c we have $c :_L \mathbf{hc}\langle A' \rangle; \epsilon; \epsilon \vdash c : \mathbf{hc}\langle A' \rangle$.

By CHANNEL-HC, for some fresh channel d we have $d :_L \mathbf{ch}\langle A' \rangle; \epsilon; \epsilon \vdash d : \mathbf{ch}\langle A' \rangle$.

By RETURN we have $c :_L \mathbf{hc}\langle A' \rangle; \epsilon; \epsilon \vdash \mathbf{return } c : C(\mathbf{hc}\langle A' \rangle)$.

Since c is fresh, we have $\Theta_2, c :_L \mathbf{hc}\langle A' \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return } c] : C(\mathbf{unit})$.

By Lemma 5.23 we have $\Theta_1, d :_L \mathbf{ch}\langle A' \rangle; \epsilon; \epsilon \vdash m[d/x] : C(\mathbf{unit})$.

By EXPR we have $\Theta_1, d :_L \mathbf{ch}\langle A' \rangle \Vdash \langle m[d/x] \rangle$ and $\Theta_2, c :_L \mathbf{hc}\langle A' \rangle \Vdash \langle \mathcal{N}[\mathbf{return } c] \rangle$.

By PAR we have $\Theta_1, d :_L \mathbf{ch}\langle A' \rangle \cup \Theta_2, c :_L \mathbf{hc}\langle A' \rangle \Vdash \langle \mathcal{N}[\mathbf{return } c] \rangle \mid \langle m[d/x] \rangle$.

By SCOPE we have $\Theta_1 \cup \Theta_2 \Vdash vcd.(\langle \mathcal{N}[\mathbf{return } c] \rangle \mid \langle m[d/x] \rangle)$ which concludes this case.

Case (PROC-END):

$$vcd.(\langle \mathcal{M}[\mathbf{close } c] \rangle \mid \langle \mathcal{N}[\mathbf{wait } d] \rangle) \Rightarrow \langle \mathcal{M}[\mathbf{return } ()] \rangle \mid \langle \mathcal{N}[\mathbf{return } ()] \rangle$$

By inversion on $\Theta \Vdash vcd.(\langle \mathcal{M}[\mathbf{send } c v] \rangle \mid \langle \mathcal{N}[\mathbf{recv } d] \rangle)$ we have either

- (1) $\Theta, c :_L \mathbf{ch}\langle A \rangle, d :_L \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close } c] \rangle \mid \langle \mathcal{N}[\mathbf{wait } d] \rangle$
- (2) $\Theta, c :_L \mathbf{hc}\langle A \rangle, d :_L \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close } c] \rangle \mid \langle \mathcal{N}[\mathbf{wait } d] \rangle$

In sub-case (2), by inversion on its typing derivation we have $\Theta_1, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close} \ c] \rangle$ and $\Theta_2, d :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{N}[\mathbf{wait} \ d] \rangle$ such that $\Theta = \Theta_1 \cup \Theta_2$.

By inversion on $\Theta_1, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close} \ c] \rangle$ we have

$$\Theta_1, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{close} \ c] : C(\text{unit})$$

By Lemma 5.83 we have $\Theta_{11}, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{close} \ c : C(B)$.

By Lemma 5.63 we have $\Theta_{11}, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash c : \mathbf{ch}\langle 1 \rangle$ which is a contradiction since c cannot be $\mathbf{ch}\langle 1 \rangle$ in this context. Thus this sub-case is impossible.

In sub-case (1), by inversion on its typing derivation we have $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close} \ c] \rangle$ and $\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{N}[\mathbf{wait} \ d] \rangle$ such that $\Theta = \Theta_1 \cup \Theta_2$.

By inversion on $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{close} \ c] \rangle$ we have

$$\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{close} \ c] : C(\text{unit})$$

By inversion on $\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{N}[\mathbf{wait} \ d] \rangle$ we have

$$\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{wait} \ d] : C(\text{unit})$$

By Lemma 5.57 we have

$$\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{close} \ c : C(B_1) \text{ and } \Theta_1 = \Theta_{11} \cup \Theta_{12}$$

By Lemma 5.57 we have

$$\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{wait} \ d : C(B_2) \text{ and } \Theta_2 = \Theta_{21} \cup \Theta_{22}$$

Applying Lemma 5.63 on $\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{close} \ c : C(B_1)$ gives us

$$\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash c : \mathbf{ch}\langle 1 \rangle \quad \text{and} \quad C(B_1) \simeq C(\text{unit})$$

which implies $A \simeq 1$, $B_1 \simeq \text{unit}$, and $\Theta_{11} = \epsilon$.

Applying Lemma 5.64 on $\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{wait} \ d : C(B_2)$ gives us

$$\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash d : \mathbf{hc}\langle 1 \rangle \quad \text{and} \quad C(B_2) \simeq C(\text{unit})$$

which implies $A \simeq 1$, $B_2 \simeq \text{unit}$ and $\Theta_{21} = \epsilon$.

Since $\Theta_{11} = \epsilon$ and $\Theta_{21} = \epsilon$, we have

$$\Theta_{11}; \epsilon; \epsilon \vdash \mathbf{return} \ () : C(\text{unit}) \quad \text{and} \quad \Theta_{21}; \epsilon; \epsilon \vdash \mathbf{return} \ () : C(\text{unit})$$

Applying the evaluation contexts $\mathcal{M}$ and $\mathcal{N}$, we have

$$\Theta_{11} \cup \Theta_{12}; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{return} \ ()] : C(\text{unit})$$

and

$$\Theta_{21} \cup \Theta_{22}; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return} \ ()] : C(\text{unit})$$

Applying EXPR and PAR, we have

$$(\Theta_{11} \cup \Theta_{12}) \cup (\Theta_{21} \cup \Theta_{22}) \Vdash \langle \mathcal{M}[\mathbf{return} \ ()] \rangle \mid \langle \mathcal{N}[\mathbf{return} \ ()] \rangle$$

which concludes this case since $(\Theta_{11} \cup \Theta_{12}) \cup (\Theta_{21} \cup \Theta_{22}) = \Theta$.

Case (PROC-COM):

$$vcd.(\langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle) \Rightarrow vcd.(\langle \mathcal{M}[\mathbf{return} \ c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \ \langle v, d \rangle_{\mathbb{L}}] \rangle)$$

By inversion on $\Theta \Vdash vcd.(\langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle)$ we have either

- (1) $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle$
- (2) $\Theta, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle, d :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle$

In sub-case (1), by inversion on its typing derivation we have $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle$ and $\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{N}[\mathbf{recv} \ d] \rangle$ such that $\Theta = \Theta_1 \cup \Theta_2$.

By inversion on $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle \Vdash \langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle$ we have

$$\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{send} \ c \ v] : C(\text{unit})$$

By inversion on $\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash \langle \mathcal{N}[\mathbf{recv} \ d] \rangle$ we have

$$\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{recv} \ d] : C(\text{unit})$$

Applying Lemma 5.83 to $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{send} \ c \ v] : C(\text{unit})$ we have

$$\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \ c \ v : C(B_1) \text{ and } \Theta_1 = \Theta_{11} \cup \Theta_{12}$$

Applying Lemma 5.83 to $\Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{recv} \ d] : C(\text{unit})$ we have

$$\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{recv} \ d : C(B_2) \text{ and } \Theta_2 = \Theta_{21} \cup \Theta_{22}$$

Applying Lemma 5.50 on $\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \ c \ v : C(B_1)$ then there exists A', B'_1 such that

$$\begin{aligned} \Theta_{111}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \ c : \Pi_t(x : A'). B'_1 \\ \Theta_{112}; \epsilon; \epsilon \vdash v : A' \\ C(B_1) \simeq B'_1[v/x] \quad \text{and} \quad \Theta_{11} = \Theta_{111} \cup \Theta_{112} \end{aligned}$$

Applying Lemma 5.59 on $\Theta_{111}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \ c : \Pi_t(x : A'). B'_1$ gives us either

- (a) $\Pi_t(x : A'). B'_1 \simeq \Pi_L(A''). C(\mathbf{ch}\langle B'_1 \rangle)$ and $\Theta_{111}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash c : \mathbf{ch}\langle !(x : A''). B'_1 \rangle$.
- (b) $\Pi_t(x : A'). B'_1 \simeq \Pi_L(A''). C(\mathbf{hc}\langle B'_1 \rangle)$ and $\Theta_{111}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash c : \mathbf{hc}\langle ?(x : A''). B'_1 \rangle$.

In (b), we have a contradiction since c cannot be $\mathbf{hc}\langle ?(x : A''). B'_1 \rangle$ in this context. Thus (b) is impossible. In (a), we have $A' \simeq A''$ and $B'_1 \simeq C(\mathbf{ch}\langle B'_1 \rangle)$ by Corollary 5.6. Additionally, we have $A \simeq !(x : A''). B'_1$ and $B_1 \simeq \mathbf{ch}\langle B'_1[v/x] \rangle$ and $\Theta_{111} = \epsilon$.

Applying Lemma 5.61 on $\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{recv} \ d : C(B_2)$ gives us either

- (a) $C(B_2) \simeq C(\Sigma_L(x : A'''). \mathbf{ch}\langle B'_2 \rangle)$ and $\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash d : \mathbf{ch}\langle ?(x : A'''). B'_2 \rangle$.
- (b) $C(B_2) \simeq C(\Sigma_L(x : A'''). \mathbf{hc}\langle B'_2 \rangle)$ and $\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash d : \mathbf{hc}\langle !(x : A'''). B'_2 \rangle$.

In (a), we have a contradiction since d cannot be $\mathbf{ch}\langle ?(x : A'''). \mathbf{ch}\langle B'_2 \rangle \rangle$ in this context. Thus (a) is impossible. In (b), we have $B_2 \simeq \Sigma_L(x : A'''). \mathbf{hc}\langle B'_2 \rangle$ by Corollary 5.9. Additionally, we have $A \simeq !(x : A'''). B'_2$ and $\Theta_{21} = \epsilon$.

Since $A \simeq !(x : A''). B'_1$ and $A \simeq !(x : A'''). B'_2$, by transitivity of $\simeq$ we have

$$!(x : A''). B'_1 \simeq !(x : A'''). B'_2$$

By Corollary 5.10 we have $A' \simeq A'' \simeq A'''$ and $B'_1 \simeq B'_2$.

By CHANNEL-CH, we have $c :_{\mathbb{L}} \mathbf{ch}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash c : \mathbf{ch}\langle B'_1[v/x] \rangle$.

By CHANNEL-HC, we have $d :_{\mathbb{L}} \mathbf{hc}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash d : \mathbf{hc}\langle B'_1[v/x] \rangle$.

Applying RETURN to $c :_{\mathbb{L}} \mathbf{ch}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash c : \mathbf{ch}\langle B'_1[v/x] \rangle$ we have

$$c :_{\mathbb{L}} \mathbf{ch}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \mathbf{return} \ c : C(\mathbf{ch}\langle B'_1[v/x] \rangle)$$

Since there is $B_1 \simeq \mathbf{ch}\langle B'_1[v/x] \rangle$, we can apply evaluation context $\mathcal{M}$ to create

$$\Theta_{12}, c :_{\mathbb{L}} \mathbf{ch}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{return} \ c] : C(\text{unit})$$

Pairing with v with d using EXPLICIT-PAIR we have

$$\Theta_{112}, d :_{\mathbb{L}} \mathbf{hc}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \langle v, d \rangle_{\mathbb{L}} : \Sigma_L(x : A'). \mathbf{hc}\langle B'_1 \rangle$$

Apply RETURN to $\Theta_{112}, d :_{\mathbb{L}} \mathbf{hc}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \langle v, d \rangle_{\mathbb{L}} : \Sigma_L(x : A'). \mathbf{hc}\langle B'_1 \rangle$ we have

$$\Theta_{112}, d :_{\mathbb{L}} \mathbf{hc}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \mathbf{return} \ \langle v, d \rangle_{\mathbb{L}} : C(\Sigma_L(x : A'). \mathbf{hc}\langle B'_1 \rangle)$$

Since $B_2 \simeq \Sigma_L(x : A''').\mathbf{hc}\langle B'_2 \rangle \simeq \Sigma_L(x : A').\mathbf{hc}\langle B'_1 \rangle$, we have

$$\Theta_{112} \cup \Theta_{22}, d :_L \mathbf{hc}\langle B'_1[v/x] \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return} \langle v, d \rangle_L] : C(\text{unit})$$

By **EXPR** we have

$$\Theta_{12}, c :_L \mathbf{ch}\langle B'_1[v/x] \rangle \Vdash \langle \mathcal{M}[\mathbf{return} c] \rangle$$

and

$$\Theta_{112} \cup \Theta_{22}, d :_L \mathbf{hc}\langle B'_1[v/x] \rangle \Vdash \langle \mathcal{N}[\mathbf{return} \langle v, d \rangle_L] \rangle$$

By **PAR** we have

$$(\Theta_{112} \cup \Theta_{12} \cup \Theta_{22}), c :_L \mathbf{ch}\langle B'_1[v/x] \rangle, d :_L \mathbf{hc}\langle B'_1[v/x] \rangle \Vdash \langle \mathcal{M}[\mathbf{return} c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \langle v, d \rangle_L] \rangle$$

By **SCOPE** we have

$$\Theta_{112} \cup \Theta_{12} \cup \Theta_{22} \Vdash \text{vcd}.(\langle \mathcal{M}[\mathbf{return} c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \langle v, d \rangle_L] \rangle)$$

Since $\Theta_{111} = \epsilon$, $\Theta_{21} = \epsilon$, we have $\Theta = ((\Theta_{111} \cup \Theta_{112}) \cup \Theta_{12}) \cup (\Theta_{21} \cup \Theta_{22})$ which means

$$\Theta \Vdash \text{vcd}.(\langle \mathcal{M}[\mathbf{return} c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \langle v, d \rangle_L] \rangle)$$

thus concluding this sub-case.

For sub-case (2), the proof is similar to sub-case (1). The only difference is that the $\mathbf{ch}\langle \cdot \rangle$ and $\mathbf{hc}\langle \cdot \rangle$ types are swapped.

Case (PROC-COM):

$$\text{vcd}.(\langle \mathcal{M}[\mathbf{send} c \{o\}] \rangle \mid \langle \mathcal{N}[\mathbf{recv} d] \rangle) \Rightarrow \text{vcd}.(\langle \mathcal{M}[\mathbf{return} c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \langle \{o\}, d \rangle_L] \rangle)$$

The proof is similar to the previous case (**PROC-COM**) with the only difference being that explicit applications are replaced with implicit applications.

Case (PROC-EXPR):

$$\frac{m \rightsquigarrow m'}{\langle m \rangle \Rightarrow \langle m' \rangle}$$

By inversion on $\Theta \Vdash \langle m \rangle$ we have $\Theta; \epsilon; \epsilon \vdash m : C(\text{unit})$. By Theorem 5.73 we have $\Theta; \epsilon; \epsilon \vdash m' : C(\text{unit})$. By **EXPR** we have $\Theta \Vdash \langle m' \rangle$ which concludes this case.

Case (PROC-PAR):

$$\frac{P \Rightarrow Q}{O \mid P \Rightarrow O \mid Q}$$

By inversion on $\Theta \Vdash O \mid P$ we have $\Theta_1 \Vdash O$ and $\Theta_2 \Vdash P$ such that $\Theta = \Theta_1 \cup \Theta_2$. By the induction hypothesis we have $\Theta_2 \Vdash Q$. By **PAR** we have $\Theta_1 \cup \Theta_2 \Vdash O \mid Q$ which concludes this case.

Case (PROC-SCOPE):

$$\frac{P \Rightarrow Q}{\text{vcd}.P \Rightarrow \text{vcd}.Q}$$

By inversion on $\Theta \Vdash \text{vcd}.P$ we have either

- (1) $\Theta, c :_L \mathbf{ch}\langle A \rangle, d :_L \mathbf{hc}\langle A \rangle \Vdash P$
- (2) $\Theta, c :_L \mathbf{hc}\langle A \rangle, d :_L \mathbf{ch}\langle A \rangle \Vdash P$

In case (1), by the induction hypothesis we have $\Theta, c :_L \mathbf{ch}\langle A \rangle, d :_L \mathbf{hc}\langle A \rangle \Vdash Q$. By **SCOPE** we have $\Theta \Vdash \text{vcd}.Q$ which concludes this sub-case.

In case (2), by the induction hypothesis we have $\Theta, c :_L \mathbf{hc}\langle A \rangle, d :_L \mathbf{ch}\langle A \rangle \Vdash Q$. By **SCOPE** we have $\Theta \Vdash \text{vcd}.Q$ which concludes this sub-case.

Case (PROC-CONGR):

$$\frac{P \equiv P' \quad P' \Rightarrow Q' \quad Q' \equiv Q}{P \Rightarrow Q}$$

By Lemma 5.82 we have $\Theta \Vdash P'$. By the induction hypothesis we have $\Theta \Vdash Q'$. By Lemma 5.82 we have $\Theta \Vdash Q$ which concludes this case. $\square$

5.3 Global Progress

The process level type system of TLL_C is insufficient to ensure that arbitrary process configurations enjoy global progress. This is because cyclic channel topologies are also considered to be well-typed. However, we can still prove a weaker form of progress for a class of configurations we call *reachable configurations*. Intuitively, a reachable configuration is one that can be reached from a well-typed singleton process through **fork**-operations.

Formally, we define the structure of *spawning trees* to capture the spawning relationships between parent-to-children processes. This formalism is inspired by the *nested-multiverse semantics* for reasoning about probabilistic session types [1, 2] and dominator trees from graph theory.

$$\begin{aligned} \text{spawning tree } \mathcal{P}, \mathcal{Q} &::= \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ &\quad | \quad \text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{aligned}$$

Each tree is associated with a term m that performs computation and a set of children processes $\{(c_i, \mathcal{P}_i)\}_{i \in I}$ where m communicates with each child process $\mathcal{P}_i$ through channel c_i . It also contains a set of subtrees $\{\mathcal{Q}_j\}_{j \in \mathcal{J}}$ that are no longer in communication with m (i.e. they have been detached through **close/wait**-operations). In the case of internal nodes, d is the channel which m uses to communicate with its parent process.

The crucial ideal behind the spawning tree structure is that we are going to define an alternative process semantics that operates on spawning trees. We will show that this alternative semantics can be simulated by the original process semantics. Moreover, we will show that the spawning tree semantics enjoys global progress. By defining reachable configurations as those that can be derived from well-typed spawning trees, we can then prove that reachable configurations enjoy global progress (induced by simulation).

To make the typing rules of spawning trees easier to define, we first introduce the following notations for channel types and $\kappa \in \{+, -\}$:

$$\begin{aligned} \text{ch}^+ \langle A \rangle &= \mathbf{ch} \langle A \rangle & \neg \text{ch}^+ \langle A \rangle &= \text{ch}^- \langle A \rangle \\ \text{ch}^- \langle A \rangle &= \mathbf{hc} \langle A \rangle & \neg \text{ch}^- \langle A \rangle &= \text{ch}^+ \langle A \rangle \end{aligned}$$

We define the typing rules for spawning trees as follows:

$$\frac{c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle; \epsilon; \epsilon \vdash m : C(\text{unit}) \quad \forall i \in I, \neg \text{ch}^{\kappa_i} \langle A_i \rangle \Vdash \mathcal{P}_i \quad \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j}{\Vdash \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})} \text{(VALID-ROOT)}$$

$$\frac{c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle, d :_{\mathbb{L}} \text{ch}^{\kappa} \langle A \rangle; \epsilon; \epsilon \vdash m : C(\text{unit}) \quad \forall i \in I, \neg \text{ch}^{\kappa_i} \langle A_i \rangle \Vdash \mathcal{P}_i \quad \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j}{\text{ch}^{\kappa} \langle A \rangle \Vdash \text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})} \text{(VALID-NODE)}$$

For the root node, we require that the term m be well-typed in channel context $c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle$ comprised of $\text{ch}^{\kappa_i} \langle A_i \rangle$ channels connecting to its children $\mathcal{P}_i$. The dual of each channel $\neg \text{ch}^{\kappa_i} \langle A_i \rangle$ is propagated to type the corresponding child $\neg \text{ch}^{\kappa_i} \langle A_i \rangle \Vdash \mathcal{P}_i$. When typing an internal node, we require that m be well-typed in a channel context that also includes $d :_{\mathbb{L}} \text{ch}^{\kappa} \langle A \rangle$, i.e. the channel connecting to its parent process.

We define the *flattening* operation $|\mathcal{P}|$ that converts a spawning tree into a standard process configuration. The operation is defined as follows:

$$\begin{array}{c} \text{FLATTEN-ROOT} \\ \frac{\forall i \in \mathcal{I}, |\mathcal{P}_i| = (d_i, P_i) \quad \forall j \in \mathcal{J}, |\mathcal{Q}_j| = Q_j}{|\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})| = \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i}) \mid \overline{Q_j}} \\ \\ \text{FLATTEN-NODE} \\ \frac{\forall i \in \mathcal{I}, |\mathcal{P}_i| = (d_i, P_i) \quad \forall j \in \mathcal{J}, |\mathcal{Q}_j| = Q_j}{|\text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})| = (d, \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i}) \mid \overline{Q_j})} \end{array}$$

The flattening operation recursively flattens each child process $\mathcal{P}_i$ into a channel-process pair (d_i, P_i) and each subtree $\mathcal{Q}_j$ into a process configuration Q_j . It then composes m with all the sub-processes in parallel. The channel pairs c_i, d_i are restricted to ensure proper channel scoping.

We now connect the validity of spawning trees to the well-typedness of flattened process configurations through Lemma 5.85.

LEMMA 5.85 (FLATTEN VALID). *If $\Vdash \mathcal{P}$ and $|\mathcal{P}| = P$, then $\Vdash P$ and if $\text{ch}^K \langle A \rangle \Vdash \mathcal{P}$ and $|\mathcal{P}| = (d, P)$, then $d :_{\mathcal{L}} \text{ch}^K \langle A \rangle \Vdash P$.*

PROOF. By mutual induction on the derivation of $\Vdash \mathcal{P}$ and $A \Vdash P$.

Case (VALID-ROOT): From FLATTEN-ROOT, we have $|\mathcal{P}_i| = (d_i, P_i)$ for each child process $\mathcal{P}_i$ and $|\mathcal{Q}_j| = Q_j$ for each subtree $\mathcal{Q}_j$. By the induction hypothesis, we have $d_i :_{\mathcal{L}} \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash P_i$ for each $i \in \mathcal{I}$ and $\Vdash Q_j$ for each $j \in \mathcal{J}$.

From the premise of VALID-ROOT, we have $\overline{c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle}; \epsilon; \epsilon \vdash m : C(\text{unit})$.

By EXPR, we have $c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle \Vdash \langle m \rangle$.

By applying PAR repeated, we have $\overline{c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle}, d_i :_{\mathcal{L}} \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash (\langle m \rangle \mid \overline{P_i})$.

By applying SCOPE repeatedly, we have $\Vdash \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i})$.

By applying PAR repeatedly, we have $\Vdash \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i}) \mid \overline{Q_j}$ which concludes this case.

Case (VALID-NODE): From FLATTEN-NODE, we have for each child process $\mathcal{P}_i$, $|\mathcal{P}_i| = (d_i, P_i)$ and for each subtree $\mathcal{Q}_j$, $|\mathcal{Q}_j| = Q_j$. By the induction hypothesis, we have $d_i :_{\mathcal{L}} \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash P_i$ for each $i \in \mathcal{I}$ and $\Vdash Q_j$ for each $j \in \mathcal{J}$.

From the premise of VALID-NODE, we have $\overline{c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle}, d :_{\mathcal{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash m : C(\text{unit})$.

By EXPR, we have $c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle, d :_{\mathcal{L}} \text{ch}^K \langle A \rangle \Vdash \langle m \rangle$.

By applying PAR repeated, we have $\overline{c_i :_{\mathcal{L}} \text{ch}^{K_i} \langle A_i \rangle}, d_i :_{\mathcal{L}} \neg \text{ch}^{K_i} \langle A_i \rangle, d :_{\mathcal{L}} \text{ch}^K \langle A \rangle \Vdash (\langle m \rangle \mid \overline{P_i})$.

By applying SCOPE repeatedly, we have $d :_{\mathcal{L}} \text{ch}^K \langle A \rangle \Vdash \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i})$.

By applying PAR repeatedly, we have $d :_{\mathcal{L}} \text{ch}^K \langle A \rangle \Vdash \overline{vc_i d_i}.(\langle m \rangle \mid \overline{P_i}) \mid \overline{Q_j}$ which concludes this case. $\square$

We now define the spawning tree semantics through the following reduction rules:

ROOT-FORK

$$\begin{array}{c} \mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(m)\} \\ \hline \text{Root}(\mathcal{N}[\text{fork } (x : A) \text{ with } m], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Root}(\mathcal{N}[\text{return } c], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \mathcal{I}'} \cup \{(c, \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in \mathcal{I}'}, \emptyset))\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{array}$$

NODE-FORK

$$\begin{array}{c} \mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(m)\} \\ \hline \text{Node}(d, \mathcal{N}[\text{fork } (x : A) \text{ with } m], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Node}(d, \mathcal{N}[\text{return } c], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \mathcal{I}'} \cup \{(c, \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in \mathcal{I}'}, \emptyset))\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{array}$$

ROOT-WAIT

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{close } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{Q}_k = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\text{wait } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}})
\end{array}$$

NODE-WAIT

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{close } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{Q}_k = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\text{wait } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}})
\end{array}$$

ROOT-CLOSE

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{wait } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{Q}_k = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\text{close } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}})
\end{array}$$

NODE-CLOSE

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{wait } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{Q}_k = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\text{close } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}})
\end{array}$$

ROOT-SEND

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(v)\} \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } \langle v, d_k \rangle_{\mathcal{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-SEND

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \quad \mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(v)\} \\
d \notin \text{FC}(v) \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } \langle v, d_k \rangle_{\mathcal{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

ROOT-RECV

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{send } d_k v], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{I}' = \{i \in \mathcal{I}_k \mid c_i \in \text{FC}(v)\} \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \setminus \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\text{recv } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\text{return } \langle v, c_k \rangle_{\mathcal{L}}], \{(c_i, \mathcal{P}_i)\}_{i \in (\mathcal{I} \setminus \{k\}) \cup \mathcal{I}'} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-RECV

$$\begin{array}{c}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{send } d_k v], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{I}' = \{i \in \mathcal{I}_k \mid c_i \in \text{FC}(v)\} \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \setminus \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\text{recv } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\text{return } \langle v, c_k \rangle_{\mathcal{L}}], \{(c_i, \mathcal{P}_i)\}_{i \in (\mathcal{I} \setminus \{k\}) \cup \mathcal{I}'} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

ROOT-SEND

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{recv}} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{return}} \langle \{o\}, d_k \rangle_{\mathcal{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\underline{\text{send}} c_k \{o\}], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\underline{\text{return}} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-SEND

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{recv}} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{return}} \langle \{o\}, d_k \rangle_{\mathcal{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\underline{\text{send}} c_k \{o\}], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\underline{\text{return}} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

ROOT-RECV

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{send}} d_k \{o\}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{return}} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Root}(\mathcal{M}[\underline{\text{recv}} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Root}(\mathcal{M}[\underline{\text{return}} \langle \{o\}, c_k \rangle_{\mathcal{L}}], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-RECV

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{send}} d_k \{o\}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{return}} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\hline
\text{Node}(d, \mathcal{M}[\underline{\text{recv}} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{M}[\underline{\text{return}} \langle \{o\}, c_k \rangle_{\mathcal{L}}], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-FORWARD

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\underline{\text{recv}} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\
\mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(v)\} \quad d \in \text{FC}(v) \quad \mathcal{P}'_k = \text{Node}(c_k, \mathcal{M}[\underline{\text{return}} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')} , \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\hline
\text{Node}(d, \mathcal{M}[\underline{\text{send}} c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\
\Rightarrow \text{Node}(d, \mathcal{N}[\underline{\text{return}} \langle v, d_k \rangle_{\mathcal{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'} \cup \{(d_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})
\end{array}$$

ROOT-CHILD

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k \Rightarrow \mathcal{P}'_k \\
\hline
\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

NODE-CHILD

$$\begin{array}{l}
k \in \mathcal{I} \quad \mathcal{P}_k \Rightarrow \mathcal{P}'_k \\
\hline
\text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})
\end{array}$$

ROOT-SUBTREE

$$\begin{array}{l}
k \in \mathcal{J} \quad \mathcal{Q}_k \Rightarrow \mathcal{Q}'_k \\
\hline
\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \setminus \{k\}} \cup \{\mathcal{Q}'_k\})
\end{array}$$

NODE-SUBTREE

$$\begin{array}{l}
k \in \mathcal{J} \quad \mathcal{Q}_k \Rightarrow \mathcal{Q}'_k \\
\hline
\text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \setminus \{k\}} \cup \{\mathcal{Q}'_k\})
\end{array}$$

$$\begin{array}{c}
\text{ROOT-EXPR} \\
\hline
m \rightsquigarrow m' \\
\hline
\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J}) \Rightarrow \text{Root}(m', \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J})
\end{array}$$

$$\begin{array}{c}
\text{NODE-EXPR} \\
\hline
m \rightsquigarrow m' \\
\hline
\text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J}) \Rightarrow \text{Node}(d, m', \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J})
\end{array}$$

In the rules above, the **ROOT-FORK** and **NODE-FORK** rules describe how a **fork**-operation spawns a new child process and adds it to the set of children processes. The new child process is represented as an internal node. The channel connecting the child to its parent is fresh. Child processes that are connected to channels in $\text{FC}(m)$ (the free channels in the spawning expression) are moved to be children of the newly spawned process.

The **ROOT-WAIT**, **NODE-WAIT**, **ROOT-CLOSE** and **NODE-CLOSE** rules describe how **close/wait**-operations detach a child from its parent. The detached process is moved to the set of subtrees.

The **ROOT-SEND** and **NODE-SEND** rules describe how a **send**-operation sends a value to a child process. The child process must be waiting to receive a value through a **recv**-operation. The sent value may contain channels that are connected to other child processes. It is important to note that **NODE-SEND** only applies when the sent value *does not* contain the channel d connecting to the parent, i.e. the side condition $d \notin \text{FC}(v)$. When a value is sent, any child processes connected to channels in $\text{FC}(v)$ are moved to be children of the receiving process.

The **ROOT-RECV** and **NODE-RECV** rules describe how a **recv**-operation receives a value from a child process. The child process must be waiting to send a value through a **send**-operation. The received value may contain channels that are connected to other child processes. When a value is received, any child processes connected to channels in $\text{FC}(v)$ are moved to be children of the receiving process. Note that, due to linearity, the channel d_k connecting the sending child to its parent cannot be in $\text{FC}(v)$. Thus, there is no side condition in **NODE-RECV**. Moreover, this means that cyclic channel dependencies cannot arise here.

The **ROOT-SEND**, **NODE-SEND**, **ROOT-RECV**, and **NODE-RECV** rules describe the sending and receiving of ghost messages through **send** and **recv** operations. Due to the fact that ghost messages do not contain channels, there are no side conditions or changes to the spawning tree structure.

The **NODE-FORWARD** rule describes how a **send**-operation can forward a parent channel d to a child process. The child process must be waiting to receive a value. The sent value must contain the parent channel d , i.e. the side condition $d \in \text{FC}(v)$. When this happens, the child process takes over the parent channel d and tree is restructured so that the child process becomes the new parent and the sending process (the original parent) becomes one of its child processes. Other child processes that are connected to channels in $\text{FC}(v)$ are also moved to be children of the receiving process.

The **ROOT-CHILD**, **NODE-CHILD**, **ROOT-SUBTREE**, and **NODE-SUBTREE** rules describe how a child process or subtree can take a reduction step.

The **ROOT-EXPR** and **NODE-EXPR** rules describe how the expression in a node can reduce.

The **ROOT-UNIT** and **NODE-UNIT** rules describe how a subtree that has finished (i.e. its expression is **return** ()) and it has no children or subtrees) can be removed from the spawning tree.

We now state the simulation theorem between the spawning tree semantics and the standard semantics (Section 4.3). With slight abuse of notation, we write $|\mathcal{P}|$ to denote just the process P obtained by flattening the spawning tree $\mathcal{P}$.

LEMMA 5.86 (SPAWNING TREE SIMULATION). *If $\Vdash \mathcal{P}$ or $A \Vdash \mathcal{P}$, then given $\mathcal{P} \Rightarrow \mathcal{P}'$ there is $|\mathcal{P}| \Rightarrow |\mathcal{P}'|$.*

PROOF. By induction on the derivation of $\mathcal{P} \Rightarrow \mathcal{P}'$.

Case (ROOT-FORK):

ROOT-FORK

$$\frac{I' = \{i \in I \mid c_i \in \text{FC}(m)\}}{\text{Root}(\mathcal{N}[\text{fork}(x : A) \text{ with } m], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(\mathcal{N}[\text{return } c], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus I'} \cup \{(c, \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in I'}, \emptyset))\}, \{Q_j\}_{j \in \mathcal{J}})}$$

Flattening the LHS, we have:

$$\overline{vc_i d_i}.(\langle \mathcal{N}[\text{fork}(x : A) \text{ with } m] \rangle \mid \overline{P_i}) \mid \overline{Q_j}$$

Repeated application of PROC-PAR and PROC-SCOPE and then PROC-FORK on the LHS gives us the reduced configuration

$$\overline{vc_i d_i}_{i \in I} . (\overline{vcd}.(\langle \mathcal{N}[\text{return } c] \rangle \mid \langle m[d/x] \rangle) \mid \overline{P_i}) \mid \overline{Q_j}$$

By linearity of channels, we know that $\{c_{i'}, d_{i'}\}_{i' \in I'}$ do not appear in $\mathcal{N}[\text{return } c]$. Thus, we can apply structural congruence to rearrange the scoping to obtain

$$\overline{vc_i d_i}_{i \in I \setminus I'} . (\overline{vcd}.(\langle \mathcal{N}[\text{return } c] \rangle \mid \overline{vc_{i'} d_{i'}}_{i' \in I'} . (\langle m[d/x] \rangle \mid \overline{P_{i'}}_{i' \in I'})) \mid \overline{P_i}) \mid \overline{Q_j}$$

which is the flattened RHS.

Case (NODE-FORK): Similar to the ROOT-FORK case.

Case (ROOT-WAIT):

ROOT-WAIT

$$\frac{k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{close } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{Q_{j_k}\}_{j_k \in \mathcal{J}_k}) \quad Q_{j_k} = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{Q_{j_k}\}_{j_k \in \mathcal{J}_k})}{\text{Root}(\mathcal{M}[\text{wait } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(\mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus \{k\}}, \{Q_j\}_{j \in \mathcal{J} \cup \{k\}})}$$

Flattening both sides, we have:

$$\overline{vc_i d_i}.(\langle \mathcal{M}[\text{wait } c_k] \rangle \mid \overline{P_{i(i \in I \setminus \{k\})}} \mid (\overline{vc_{i_k} d_{i_k}}.(\langle \mathcal{N}[\text{close } d_k] \rangle \mid \overline{P_{i_k}} \mid \overline{Q_{j_k}})) \mid \overline{Q_j}) \\ \overline{vc_i d_i}_{i \in I \setminus \{k\}} . (\langle \mathcal{M}[\text{return } ()] \rangle \mid \overline{P_{i(i \in I \setminus \{k\})}} \mid \overline{Q_j} \mid (\overline{vc_{i_k} d_{i_k}}.(\langle \mathcal{N}[\text{return } ()] \rangle \mid \overline{P_{i_k}} \mid \overline{Q_{j_k}}))$$

Apply PROC-CONGR to the LHS to rearrange the processes, then apply PROC-SCOPE and PROC-PAR repeatedly to isolate the sub-configuration $\overline{vc_k d_k}.(\langle \mathcal{M}[\text{wait } c_k] \rangle \mid \langle \mathcal{N}[\text{close } d_k] \rangle)$. Finally, apply PROC-WAIT to this sub-configuration to obtain the reduced configuration, which is structurally congruent to the RHS.

Case (NODE-WAIT): Similar to the ROOT-WAIT case.

Case (ROOT-CLOSE): Similar to the ROOT-WAIT case.

Case (NODE-CLOSE): Similar to the ROOT-WAIT case.

Case (ROOT-SEND):

ROOT-SEND

$$\frac{k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{Q_{j_k}\}_{j_k \in \mathcal{J}_k}) \quad I' = \{i \in I \mid c_i \in \text{FC}(v)\} \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } \langle v, d_k \rangle_L], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k \cup I'}, \{Q_{j_k}\}_{j_k \in \mathcal{J}_k})}{\text{Root}(\mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(\mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus (\{k\} \cup I')} \cup \{(c_k, \mathcal{P}'_k)\}, \{Q_j\}_{j \in \mathcal{J}})}$$

Flattening both sides, we have:

$$\overline{vc_i d_i}.(\langle \mathcal{M}[\text{send } c_k v] \rangle \mid \overline{P_{i(i \in I \setminus \{k\})}} \mid (\overline{vc_{i_k} d_{i_k}}.(\langle \mathcal{N}[\text{recv } d_k] \rangle \mid \overline{P_{i_k}} \mid \overline{Q_{j_k}})) \mid \overline{Q_j}) \\ \overline{vc_i d_i}_{i \in (I \setminus (\{k\} \cup I')) \cup \{k\}} . (\langle \mathcal{M}[\text{return } c_k] \rangle \mid \overline{P_{i(i \in I \setminus (\{k\} \cup I'))}} \mid \overline{vc_{i_k} d_{i_k}} . \overline{vc_i d_i}_{i \in I'} . (\langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle \mid \overline{P_{i_k}} \mid \overline{P_{i(i \in I')}} \mid \overline{Q_{j_k}} \mid \overline{Q_j})$$

Apply PROC-CONGR to the LHS to rearrange the processes, then apply PROC-SCOPE and PROC-PAR repeatedly to isolate the sub-configuration $vc_k d_k.(\langle \mathcal{M}[\text{send } c_k v] \rangle \mid \langle \mathcal{N}[\text{recv } d_k] \rangle)$. Now, apply PROC-SEND to this sub-configuration to obtain the reduced configuration

$$vc_k d_k.(\langle \mathcal{M}[\text{return } c_k] \rangle \mid \langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle)$$

Note that, by linearity, the channels $\{c_i, d_i\}_{(i \in I')}$ do not occur in $\mathcal{M}[\text{return } c_k]$. Thus, structural congruence can be applied to move the scope of these channels to

$$\overline{vc_i d_i}_{(i \in I')}.(\langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle \mid \overline{P_i}_{(i \in I')})$$

which gives us the desired result.

Case (NODE-SEND, ROOT-RECV NODE-RECV): Similar to the ROOT-SEND case.

Case (ROOT-SEND NODE-SEND ROOT-RECV NODE-RECV): Similar to the ROOT-SEND case. The only difference is that scope restriction does not need to be applied to move any channels since the ghost message o does not contain channels.

Case (NODE-FORWARD):

NODE-FORWARD

$$\frac{\begin{array}{l} k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{Q_{j_k}\}_{j_k \in J_k}) \\ I' = \{i \in I \mid c_i \in \text{FC}(v)\} \quad d \in \text{FC}(v) \quad \mathcal{P}'_k = \text{Node}(c_k, \mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus (\{k\} \cup I')}, \{Q_j\}_{j \in J}) \end{array}}{\begin{array}{l} \text{Node}(d, \mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J}) \\ \Rightarrow \text{Node}(d, \mathcal{N}[\text{return } \langle v, d_k \rangle_L], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k \cup I'} \cup \{(d_k, \mathcal{P}'_k)\}, \{Q_{j_k}\}_{j_k \in J_k}) \end{array}}$$

Flattening both sides, we have:

$$\begin{aligned} & \overline{vc_i d_i}.(\langle \mathcal{M}[\text{send } c_k v] \rangle \mid \overline{P_i}_{(i \in I \setminus \{k\})} \mid (\overline{vc_{i_k} d_{i_k}}.(\langle \mathcal{N}[\text{recv } d_k] \rangle \mid \overline{P_{i_k}} \mid \overline{Q_{j_k}})) \mid \overline{Q_j}) \\ & \overline{vc_{i_k} d_{i_k}}_{(i_k \in I_k \cup I' \cup \{k\})}.(\langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle \mid \overline{P_{i_k}}_{(i_k \in I_k \cup I')} \mid \\ & \quad \mid \overline{vc_i d_i}_{(i \in I \setminus (\{k\} \cup I'))}.(\langle \mathcal{M}[\text{return } c_k] \rangle \mid \overline{P_i}_{(i \in I \setminus (\{k\} \cup I'))}) \mid \overline{Q_j}) \mid \overline{Q_{j_k}} \end{aligned}$$

Apply PROC-CONGR to the LHS to rearrange the processes, then apply PROC-SCOPE and PROC-PAR repeatedly to isolate the sub-configuration $vc_k d_k.(\langle \mathcal{M}[\text{send } c_k v] \rangle \mid \langle \mathcal{N}[\text{recv } d_k] \rangle)$. Now, apply PROC-SEND to this sub-configuration to obtain the reduced configuration

$$vc_k d_k.(\langle \mathcal{M}[\text{return } c_k] \rangle \mid \langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle)$$

By symmetry of structural congruence, we have

$$vc_k d_k.(\langle \mathcal{N}[\text{return } \langle v, d_k \rangle_L] \rangle \mid \langle \mathcal{M}[\text{return } c_k] \rangle)$$

Since $\{c_i, d_i\}_{(i \in I \setminus (\{k\} \cup I'))}$ do not occur in $\mathcal{N}[\text{return } \langle v, d_k \rangle_L]$, we can apply structural congruence to move the scope of these channels to

$$\overline{vc_i d_i}_{(i \in I \setminus (\{k\} \cup I'))}.(\langle \mathcal{M}[\text{return } c_k] \rangle \mid \overline{P_i}_{(i \in I \setminus (\{k\} \cup I'))})$$

which gives us the desired result.

Case (ROOT-CHILD, NODE-CHILD, ROOT-SUBTREE, NODE-SUBTREE): By the induction hypothesis, we have $|\mathcal{P}_k| \Rightarrow |\mathcal{P}'_k|$ or $|Q_k| \Rightarrow |Q'_k|$. Repeated application of PROC-PAR and PROC-SCOPE gives us the desired result.

Case (ROOT-EXPR, NODE-EXPR): By the assumption, we have $m \rightsquigarrow m'$. Repeated application of PROC-PAR, PROC-SCOPE and then PROC-EXPR gives us the desired result. $\square$

In order to show that spawning trees are an adequate characterization of reachability, we prove the following fidelity theorem. This theorem states that if a spawning tree is well-typed and it takes a reduction step, then the resulting spawning tree is also well-typed. Thus, starting from a well-typed singleton $\text{Root}(m, \emptyset, \emptyset)$, the spawning trees reachable from it are all well-typed.

LEMMA 5.87 (SPAWNING TREE FIDELITY). *If $\Vdash \mathcal{P}$ or $\text{ch}^K \langle A \rangle \Vdash \mathcal{P}$, then given $\mathcal{P} \Rightarrow \mathcal{Q}$ there is $\Vdash \mathcal{Q}$ or $\text{ch}^K \langle A \rangle \Vdash \mathcal{Q}$ respectively.*

PROOF. By induction on the derivation of $\mathcal{P} \Rightarrow \mathcal{Q}$ and by case analysis on the derivation of the typing judgment $\Vdash \mathcal{P}$ or $\text{ch}^K \langle A \rangle \Vdash \mathcal{P}$.

Case (ROOT-FORK):

ROOT-FORK

$$\frac{I' = \{i \in I \mid c_i \in \text{FC}(m)\}}{\begin{array}{l} \text{Root}(\mathcal{N}[\text{fork}(x : A) \text{ with } m], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Root}(\mathcal{N}[\text{return } c], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus I'} \cup \{(c, \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in I'}, \emptyset))\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{array}}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(\mathcal{N}[\text{fork}(x : A) \text{ with } m], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{array}{l} \overline{c_i :_{\text{L}} \text{ch}^{K_i} \langle A_i \rangle}; \epsilon; \epsilon \vdash \mathcal{N}[\text{fork}(x : A) \text{ with } m] : C(\text{unit}) \\ \forall i \in I, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{array}$$

Similarly to the reasoning in Theorem 5.84 for the PROC-FORK case, by inversion on the term typing judgment we know that $A = \text{ch} \langle A' \rangle$ and the following hold:

$$\begin{array}{l} \overline{c_i :_{\text{L}} \text{ch}^{K_i} \langle A_i \rangle}_{(i \in I')}, d :_{\text{L}} \text{hc} \langle A' \rangle; \epsilon; \epsilon \vdash m[d/x] : C(\text{unit}) \\ \overline{c_i :_{\text{L}} \text{ch}^{K_i} \langle A_i \rangle}_{(i \in I \setminus I')}, c :_{\text{L}} \text{ch} \langle A' \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\text{return } c] : C(\text{unit}) \end{array}$$

We can partition $\{(c_i, \mathcal{P}_i)\}_{i \in I}$ into $\{(c_i, \mathcal{P}_i)\}_{i \in I \setminus I'}$ and $\{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in I'}$.

We now have well-typed node

$$\text{ch}^- \langle A' \rangle \Vdash \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in I'}, \emptyset)$$

which in turn gives us the well-typed root

$$\Vdash \text{Root}(\mathcal{N}[\text{return } c], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus I'} \cup \{(c, \text{Node}(d, m[d/x], \{(c_{i'}, \mathcal{P}_{i'})\}_{i' \in I'}, \emptyset))\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

and concludes this case.

Case (NODE-FORK): Similar to the ROOT-FORK case.

Case (ROOT-WAIT):

ROOT-WAIT

$$\begin{array}{l} k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{close } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \mathcal{Q}_k = \text{Root}(\mathcal{N}[\text{return } ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \hline \text{Root}(\mathcal{M}[\text{wait } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Root}(\mathcal{M}[\text{return } ()], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}}) \end{array}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(\mathcal{M}[\text{wait } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{array}{l} \overline{c_i :_{\text{L}} \text{ch}^{K_i} \langle A_i \rangle}; \epsilon; \epsilon \vdash \mathcal{M}[\text{wait } c_k] : C(\text{unit}) \\ \forall i \in I, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{array}$$

Since $k \in I$, we know

$$\neg \text{ch}^{K_k} \langle A_k \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\text{close } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

and by inversion on this typing judgment, we have

$$\begin{array}{c} \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle, d_k :_{\mathbb{L}} \neg \text{ch}^{\kappa_k} \langle A_k \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{close} d_k] : C(\text{unit})} \\ \forall i_k \in \mathcal{I}_k, \neg \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle \Vdash \mathcal{P}_{i_k} \\ \forall j_k \in \mathcal{J}_k, \Vdash \mathcal{Q}_{j_k} \end{array}$$

Similarly to the reasoning in Theorem 5.84 for the PROC-END case, we know $A_k \simeq \mathbf{1}$ and

$$\begin{array}{c} \overline{c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle_{(i \in \mathcal{I} \setminus \{k\})}; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{return} ()] : C(\text{unit})} \\ \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return} ()] : C(\text{unit})} \end{array}$$

This gives us the well-typed root

$$\Vdash \text{Root}(\mathcal{N}[\mathbf{return} ()], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

which in turn gives us the well-typed root

$$\Vdash \text{Root}(\mathcal{M}[\mathbf{return} ()], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J} \cup \{k\}})$$

and concludes this case.

Case (NODE-WAIT, ROOT-CLOSE, NODE-CLOSE): Similar to the ROOT-WAIT case.

Case (ROOT-SEND):

ROOT-SEND

$$\begin{array}{c} k \in \mathcal{I} \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\mathbf{recv} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \mathcal{I}' = \{i \in \mathcal{I} \mid c_i \in \text{FC}(v)\} \quad \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\mathbf{return} \langle v, d_k \rangle_{\mathbb{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \hline \text{Root}(\mathcal{M}[\mathbf{send} c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Root}(\mathcal{M}[\mathbf{return} c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{array}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(\mathcal{M}[\mathbf{send} c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{array}{c} \overline{c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{send} c_k v] : C(\text{unit})} \\ \forall i \in \mathcal{I}, \neg \text{ch}^{\kappa_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{array}$$

From $k \in \mathcal{I}$, we know

$$\neg \text{ch}^{\kappa_k} \langle A_k \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\mathbf{recv} d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

and by inversion on this typing judgment, we have

$$\begin{array}{c} \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle, d_k :_{\mathbb{L}} \neg \text{ch}^{\kappa_k} \langle A_k \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{recv} d_k] : C(\text{unit})} \\ \forall i_k \in \mathcal{I}_k, \neg \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle \Vdash \mathcal{P}_{i_k} \\ \forall j_k \in \mathcal{J}_k, \Vdash \mathcal{Q}_{j_k} \end{array}$$

Similarly to the reasoning in Theorem 5.84 for the PROC-COMM case, there exists B'' such that

$$\begin{array}{c} \overline{c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle_{(i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}'))}, c_k :_{\mathbb{L}} \text{ch}^{\kappa_k} \langle B''[v/x] \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{return} c_k] : C(\text{unit})} \\ \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{\kappa_{i_k}} \langle A_{i_k} \rangle, c_i :_{\mathbb{L}} \text{ch}^{\kappa_i} \langle A_i \rangle_{(i \in \mathcal{I}')} , d_k :_{\mathbb{L}} \neg \text{ch}^{\kappa_k} \langle B''[v/x] \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return} \langle v, d_k \rangle_{\mathbb{L}}] : C(\text{unit})} \end{array}$$

We can partition $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}$ into $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')}$ and $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}'}$.

This gives us the well-typed node

$$\neg \text{ch}^{\kappa_k} \langle B''[v/x] \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\mathbf{return} \langle v, d_k \rangle_{\mathbb{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

which in turn gives us the well-typed root

$$\Vdash \text{Root}(\mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus (\{k\} \cup I')} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

and concludes this case.

Case (NODE-SEND, ROOT-RECV, NODE-RECV): Similar to the ROOT-SEND case.

Case (ROOT-SEND):

$$\begin{array}{c} \text{ROOT-SEND} \\ k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \mathcal{P}'_k = \text{Node}(d_k, \mathcal{N}[\text{return } \langle \{o\}, d_k \rangle_L], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ \hline \text{Root}(\mathcal{M}[\text{send } c_k \{o\}], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Root}(\mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \end{array}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(\mathcal{M}[\text{send } c_k \{o\}], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{array}{l} \overline{c_i :_L \text{ch}^{k_i} \langle A_i \rangle}; \epsilon; \epsilon \vdash \mathcal{M}[\text{send } c_k \{o\}] : C(\text{unit}) \\ \forall i \in I, \neg \text{ch}^{k_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{array}$$

From $k \in I$, we know

$$\neg \text{ch}^{k_k} \langle A_k \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

and by inversion on this typing judgment, we have

$$\begin{array}{l} \overline{c_{i_k} :_L \text{ch}^{k_{i_k}} \langle A_{i_k} \rangle}, d_k :_L \neg \text{ch}^{k_k} \langle A_k \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\text{recv } d_k] : C(\text{unit}) \\ \forall i_k \in I_k, \neg \text{ch}^{k_{i_k}} \langle A_{i_k} \rangle \Vdash \mathcal{P}_{i_k} \\ \forall j_k \in \mathcal{J}_k, \Vdash \mathcal{Q}_{j_k} \end{array}$$

Similarly to the reasoning in Theorem 5.84 for the PROC-COMM case, there exists B'' such that

$$\begin{array}{l} \overline{c_i :_L \text{ch}^{k_i} \langle A_i \rangle}_{(i \in I \setminus \{k\})}, c_k :_L \text{ch}^{k_k} \langle B''[o/x] \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\text{return } c_k] : C(\text{unit}) \\ \overline{c_{i_k} :_L \text{ch}^{k_{i_k}} \langle A_{i_k} \rangle}, d_k :_L \neg \text{ch}^{k_k} \langle B''[o/x] \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\text{return } \langle \{o\}, d_k \rangle_L] : C(\text{unit}) \end{array}$$

This gives us the well-typed node

$$\neg \text{ch}^{k_k} \langle B''[o/x] \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\text{return } \langle \{o\}, d_k \rangle_L], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

which in turn gives us the well-typed root

$$\Vdash \text{Root}(\mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

and concludes this case.

Case (NODE-SEND, ROOT-RECV, NODE-RECV): Similar to the ROOT-SEND case.

Case (NODE-FORWARD):

NODE-FORWARD

$$\begin{array}{c} k \in I \quad \mathcal{P}_k = \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \\ I' = \{i \in I \mid c_i \in \text{FC}(v)\} \quad d \in \text{FC}(v) \quad \mathcal{P}'_k = \text{Node}(c_k, \mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in I \setminus (\{k\} \cup I')}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \hline \text{Node}(d, \mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \\ \Rightarrow \text{Node}(d, \mathcal{N}[\text{return } \langle v, d_k \rangle_L], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in I_k \cup I'} \cup \{(d_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k}) \end{array}$$

By inversion on the typing judgment

$$\text{ch}^k \langle A \rangle \Vdash \text{Node}(d, \mathcal{M}[\text{send } c_k v], \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{aligned} & \overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle}, d :_{\mathbb{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\text{send } c_k \ v] : C(\text{unit}) \\ & \forall i \in \mathcal{I}, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ & \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{aligned}$$

From $k \in \mathcal{I}$, we know

$$\neg \text{ch}^{K_k} \langle A_k \rangle \Vdash \text{Node}(d_k, \mathcal{N}[\text{recv } d_k], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

and by inversion on this typing judgment, we have

$$\begin{aligned} & \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{K_{i_k}} \langle A_{i_k} \rangle}, d_k :_{\mathbb{L}} \neg \text{ch}^{K_k} \langle A_k \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\text{recv } d_k] : C(\text{unit}) \\ & \forall i_k \in \mathcal{I}_k, \neg \text{ch}^{K_{i_k}} \langle A_{i_k} \rangle \Vdash \mathcal{P}_{i_k} \\ & \forall j_k \in \mathcal{J}_k, \Vdash \mathcal{Q}_{j_k} \end{aligned}$$

Similarly to the reasoning in Theorem 5.84 for the PROC-COMM case, there exists B'' such that

$$\begin{aligned} & \overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle}_{(i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}'))}, c_k :_{\mathbb{L}} \text{ch}^{K_k} \langle B''[v/x] \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\text{return } c_k] : C(\text{unit}) \\ & \overline{c_{i_k} :_{\mathbb{L}} \text{ch}^{K_{i_k}} \langle A_{i_k} \rangle}, \overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle}_{(i \in \mathcal{I}')}, d_k :_{\mathbb{L}} \neg \text{ch}^{K_k} \langle B''[v/x] \rangle, d :_{\mathbb{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\text{return } \langle v, d_k \rangle_{\mathbb{L}}] : C(\text{unit}) \end{aligned}$$

We can partition $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}}$ into $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')}$ and $\{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}'}$.

This gives us the well-typed node

$$\text{ch}^{K_k} \langle B''[v/x] \rangle \Vdash \text{Node}(c_k, \mathcal{M}[\text{return } c_k], \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus (\{k\} \cup \mathcal{I}')} , \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

which in turn gives us the well-typed node

$$\text{ch}^K \langle A \rangle \Vdash \text{Node}(d, \mathcal{N}[\text{return } \langle v, d_k \rangle_{\mathbb{L}}], \{(c_{i_k}, \mathcal{P}_{i_k})\}_{i_k \in \mathcal{I}_k \cup \mathcal{I}'} \cup \{(d_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_{j_k}\}_{j_k \in \mathcal{J}_k})$$

and concludes this case.

Case (ROOT-CHILD):

ROOT-CHILD

$$\frac{k \in \mathcal{I} \quad \mathcal{P}_k \Rightarrow \mathcal{P}'_k}{\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{aligned} & \overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle}; \epsilon; \epsilon \vdash m : C(\text{unit}) \\ & \forall i \in \mathcal{I}, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ & \forall j \in \mathcal{J}, \Vdash \mathcal{Q}_j \end{aligned}$$

From $k \in \mathcal{I}$ and the typing judgment $\neg \text{ch}^{K_k} \langle A_k \rangle \Vdash \mathcal{P}_k$,

By the induction hypothesis, we have the well-typed process $\neg \text{ch}^{K_k} \langle A_k \rangle \Vdash \mathcal{P}'_k$.

This gives us the well-typed root

$$\Vdash \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I} \setminus \{k\}} \cup \{(c_k, \mathcal{P}'_k)\}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})$$

and concludes this case.

Case (NODE-CHILD, ROOT-SUBTREE, NODE-SUBTREE): Similar to the ROOT-CHILD case.

Case (ROOT-EXPR):

ROOT-EXPR

$$\frac{m \rightsquigarrow m'}{\text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}}) \Rightarrow \text{Root}(m', \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{\mathcal{Q}_j\}_{j \in \mathcal{J}})}$$

By inversion on the typing judgment

$$\Vdash \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{Q_j\}_{j \in \mathcal{J}})$$

we have

$$\begin{array}{c} \overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle; \epsilon; \epsilon \vdash m : C(\text{unit})} \\ \forall i \in \mathcal{I}, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \\ \forall j \in \mathcal{J}, \Vdash Q_j \end{array}$$

By Theorem 5.73 we have

$$\overline{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle; \epsilon; \epsilon \vdash m' : C(\text{unit})}$$

which gives us the well-typed root

$$\Vdash \text{Root}(m', \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{Q_j\}_{j \in \mathcal{J}})$$

and concludes this case.

Case (NODE-EXPR): Similar to the ROOT-EXPR case. $\square$

Now we can define the reachability of a configuration through spawning trees.

Definition 5.88 (Reachability). A configuration P is *reachable* if there exists a spawning tree $\mathcal{P}$ such that $\Vdash \mathcal{P}$ and $|\mathcal{P}| = P$.

Now to prove the main progress theorem, we first need to define a few auxiliary judgments that will help us characterize the state of a spawning tree.

The first judgment characterizes when a spawning tree is *terminal*, i.e., it has no further reductions. The expression in the root of the spawning tree must be a **return** () expression. The set of child processes should be empty and all sub-trees must be terminal.

$$\frac{\forall j \in \mathcal{J}, \quad Q_j \text{ Terminal}}{\text{Root}(\text{return} (), \emptyset, \{Q_j\}_{j \in \mathcal{J}}) \text{ Terminal}} (\text{TERMINAL-ROOT})$$

LEMMA 5.89 (TERMINAL). If $\mathcal{P}$ Terminal, then $|\mathcal{P}| \equiv \langle \text{return} () \rangle$.

PROOF. By induction on the derivation of $\mathcal{P}$ Terminal and definition of structural congruence. $\square$

The second judgment characterizes when a process is *poised* to communicate on a channel d . These poised expressions correspond to thunked concurrency primitives that are waiting to be executed. The **fork**-operation is not include here because it does not rely on any channel.

POISED-EXPLICIT-RECV	POISED-IMPLICIT-RECV	POISED-EXPLICIT-SEND	POISED-IMPLICIT-SEND
$\overline{\text{recv } d \text{ Poised}(d)}$	$\overline{\text{recv } d \text{ Poised}(d)}$	$\overline{\text{send } d \ v \text{ Poised}(d)}$	$\overline{\text{send } d \ \{o\} \text{ Poised}(d)}$
POISED-CLOSE	POISED-WAIT	POISED-BIND	
$\overline{\text{close } d \text{ Poised}(d)}$	$\overline{\text{wait } d \text{ Poised}(d)}$	$\overline{m \text{ Poised}(d)}$	
		$\text{let } x \Leftarrow m \text{ in } n \text{ Poised}(d)$	

LEMMA 5.90 (THUNK EVALUATION CONTEXT). Given thunk τ , there exists an evaluation context $\mathcal{M}$ such that $\tau = \mathcal{M}[m]$ where m is one of the following forms:

$$\text{fork } (x : A) \text{ with } n \mid \text{recv } c \mid \text{recv } c \mid \text{send } c \ v \mid \text{send } c \ \{o\} \mid \text{close } c \mid \text{wait } c$$

PROOF. By induction on the structure of τ . $\square$

LEMMA 5.91 (POISED EVALUATION CONTEXT). *If m Poised(d) then there exists an evaluation context $\mathcal{M}$ such that $m = \mathcal{M}[m']$ where m' is one of the following forms:*

$$\mathbf{recv} \, d \mid \mathbf{recv} \, d \mid \mathbf{send} \, d \, v \mid \mathbf{send} \, d \, \{o\} \mid \mathbf{close} \, d \mid \mathbf{wait} \, d$$

PROOF. By induction on the derivation of m Poised(d). □

The final judgment characterizes a spawning tree that is poised to communicate on channel d connected to its parent.

$$\frac{\text{POISED-NODE} \quad m \text{ Poised}(d)}{\text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J}) \text{ Poised}(d)}$$

With these definitions, we can now prove the spawning tree progress lemma.

LEMMA 5.92 (SPAWNING TREE PROGRESS). *If $\Vdash \mathcal{P}$, then either $\mathcal{P}$ Terminal or there exists $\mathcal{P}'$ such that $\mathcal{P} \Rightarrow \mathcal{P}'$. If $A \Vdash \mathcal{P}$, then either $\mathcal{P}$ Poised(d) or there exists $\mathcal{P}'$ such that $\mathcal{P} \Rightarrow \mathcal{P}'$.*

PROOF. By mutual induction on the derivation of $\Vdash \mathcal{P}$ and $A \Vdash \mathcal{P}$.

Case (VALID-ROOT)

$$\frac{c_i :_{\mathbb{L}} \text{ch}^{k_i} \langle A_i \rangle; \epsilon; \epsilon \vdash m : C(\text{unit}) \quad \forall i \in I, \neg \text{ch}^{k_i} \langle A_i \rangle \Vdash \mathcal{P}_i \quad \forall j \in J, \Vdash Q_j}{\Vdash \text{Root}(m, \{(c_i, \mathcal{P}_i)\}_{i \in I}, \{Q_j\}_{j \in J})} \text{(VALID-ROOT)}$$

Apply the induction hypothesis on Q_j . If any Q_j is not terminal, then there exists Q_k such that $Q_k \Rightarrow Q'_k$. By the ROOT-SUBTREE rule, we are done. Thus we can assume that Q_j are all terminal.

Next, we analyze the expression m . By Theorem 5.81, either $m \rightsquigarrow m'$ or m is a value. If $m \rightsquigarrow m'$, then by the ROOT-EXPR rule, we are done. Thus we can assume that m is a value.

By Lemma 5.80 we know that either

- (1) exists value v such that $m = \mathbf{return} \, v$
- (2) m is a thunk

In sub-case (1), if $m = \mathbf{return} \, v$, then by Lemma 5.56 we have $\overline{c_i :_{\mathbb{L}} \text{ch}^{k_i} \langle A_i \rangle; \epsilon; \epsilon \vdash v : \text{unit}}$. By Lemma 5.78, we have $v = ()$. By Lemma 5.53, we have $c_i :_{\mathbb{L}} \text{ch}^{k_i} \langle A_i \rangle = \epsilon$. Thus $I = \emptyset$ and we have the terminal spawning tree $\text{Root}(\mathbf{return} \, (), \emptyset, \{Q_j\}_{j \in J})$.

In sub-case (2), if m is a thunk, then we can use Lemma 5.90 to decompose m into an evaluation context $\mathcal{M}$ and a sub-expression m' such that $m = \mathcal{M}[m']$ where m' is one of the following forms:

$$\mathbf{fork} \, (x : A) \mathbf{with} \, n \mid \mathbf{recv} \, c \mid \mathbf{recv} \, c \mid \mathbf{send} \, c \, v \mid \mathbf{send} \, c \, \{o\} \mid \mathbf{close} \, c \mid \mathbf{wait} \, c$$

We now perform case analysis on m' . Most cases are similar and straightforward, so we only show the following interesting cases in detail:

- If $m' = \mathbf{fork} \, (x : A) \mathbf{with} \, n$, then applying ROOT-FORK yields a reduction.
- If $m' = \mathbf{recv} \, c$, then by Lemma 5.83 and Lemma 5.61 we have $c \in \{c_i\}_{i \in I}$. Let k be such that $c_k = c$. Apply the induction hypothesis on all $\mathcal{P}_i$. If any $\mathcal{P}_i$ is not poised on d_i , then there exists $\mathcal{P}'_i$ such that $\mathcal{P}_i \Rightarrow \mathcal{P}'_i$. By the ROOT-CHILD rule, we are done. Thus we can assume that all $\mathcal{P}_i$ are poised on d_i . In particular, $\mathcal{P}_k$ is poised on d_k . By inversion of POISED-NODE and Lemma 5.91, we can decompose $\mathcal{P}_k$ into an evaluation context $\mathcal{N}$ and a sub-expression m'_k such that m'_k is one of the following forms:

$$\mathbf{recv} \, d_k \mid \mathbf{recv} \, d_k \mid \mathbf{send} \, d_k \, v \mid \mathbf{send} \, d_k \, \{o\} \mid \mathbf{close} \, d_k \mid \mathbf{wait} \, d_k$$

Since $\mathcal{P}_k$ is well-typed, the type of dual channel d_k is $\neg \text{ch}^{k_k} \langle A_k \rangle$. So m'_k must be $\mathbf{send} \, d_k \, v$. Applying ROOT-RECV yields a reduction.

This concludes the VALID-ROOT case.

Case (VALID-NODE):

$$\frac{c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle, d :_{\mathbb{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash m : C(\text{unit}) \quad \forall i \in \mathcal{I}, \neg \text{ch}^{K_i} \langle A_i \rangle \Vdash \mathcal{P}_i \quad \forall j \in \mathcal{J}, \Vdash Q_j}{\text{ch}^K \langle A \rangle \Vdash \text{Node}(d, m, \{(c_i, \mathcal{P}_i)\}_{i \in \mathcal{I}}, \{Q_j\}_{j \in \mathcal{J}})}_{(\text{VALID-NODE})}$$

Apply the induction hypothesis on Q_j . If any Q_j is not terminal, then there exists Q_k such that $Q_k \Rightarrow Q'_k$. By the NODE-SUBTREE rule, we are done. Thus we can assume that Q_j are all terminal.

Next, we analyze the expression m . By Theorem 5.81, either $m \rightsquigarrow m'$ or m is a value. If $m \rightsquigarrow m'$, then by the NODE-EXPR rule, we are done. Thus we can assume that m is a value.

By Lemma 5.80 we know that either

- (1) exists value v such that $m = \text{return } v$
- (2) m is a thunk

In sub-case (1), if $m = \text{return } v$, then applying Lemma 5.56 yields $c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle, d :_{\mathbb{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash v : \text{unit}$. By Lemma 5.78, we have $v = ()$. By Lemma 5.53, we have $c_i :_{\mathbb{L}} \text{ch}^{K_i} \langle A_i \rangle, d :_{\mathbb{L}} \text{ch}^K \langle A \rangle = \epsilon$, which is a contradiction since $d :_{\mathbb{L}} \text{ch}^K \langle A \rangle$ is not empty. Thus this sub-case is impossible.

In sub-case (2), if m is a thunk, then we can use Lemma 5.90 to decompose m into an evaluation context $\mathcal{M}$ and a sub-expression m' such that $m = \mathcal{M}[m']$ where m' is one of the following forms:

fork ($x : A$) **with** n | **recv** c | recv c | **send** c v | send c $\{o\}$ | **close** c | **wait** c

We now perform case analysis on m' . Most cases are similar and straightforward, so we only show the following interesting cases in detail:

- If $m' = \text{fork } (x : A) \text{ with } n$, then applying NODE-FORK yields a reduction.
- If $m' = \text{recv } c$, then by Lemma 5.83 and Lemma 5.61 we have either $c = d$ or $c \in \{c_i\}_{i \in \mathcal{I}}$. In the case that $c = d$, we are done by the POISED-NODE rule. In the case that $c \in \{c_i\}_{i \in \mathcal{I}}$, let k be such that $c_k = c$. Apply the induction hypothesis on all $\mathcal{P}_i$. If any $\mathcal{P}_i$ is not poised on d_i , then there exists $\mathcal{P}'_i$ such that $\mathcal{P}_i \Rightarrow \mathcal{P}'_i$. By the NODE-CHILD rule, we are done. Thus we can assume that all $\mathcal{P}_i$ are poised on d_i . In particular, $\mathcal{P}_k$ is poised on d_k . By inversion of POISED-NODE and Lemma 5.91, we can decompose $\mathcal{P}_k$ into an evaluation context $\mathcal{N}$ and a sub-expression m'_k such that m'_k is one of the following forms:

recv d_k | recv d_k | **send** d_k v | send d_k $\{o\}$ | **close** d_k | **wait** d_k

Since $\mathcal{P}_k$ is well-typed, the type of dual channel d_k is $\neg \text{ch}^{K_k} \langle A_k \rangle$. So m'_k must be **send** d_k v . Applying NODE-RECV yields a reduction.

- If $m' = \text{send } c$ v , then by Lemma 5.83 and Lemma 5.59 we have either $c = d$ or $c \in \{c_i\}_{i \in \mathcal{I}}$. In the case that $c = d$, we are done by the POISED-NODE rule. In the case that $c \in \{c_i\}_{i \in \mathcal{I}}$, let k be such that $c_k = c$. Apply the induction hypothesis on all $\mathcal{P}_i$. If any $\mathcal{P}_i$ is not poised on d_i , then there exists $\mathcal{P}'_i$ such that $\mathcal{P}_i \Rightarrow \mathcal{P}'_i$. By the NODE-CHILD rule, we are done. Thus we can assume that all $\mathcal{P}_i$ are poised on d_i . In particular, $\mathcal{P}_k$ is poised on d_k . By inversion of POISED-NODE and Lemma 5.91, we can decompose $\mathcal{P}_k$ into an evaluation context $\mathcal{N}$ and a sub-expression m'_k such that m'_k is one of the following forms:

recv d_k | recv d_k | **send** d_k v | send d_k $\{o\}$ | **close** d_k | **wait** d_k

Since $\mathcal{P}_k$ is well-typed, the type of dual channel d_k is $\neg \text{ch}^{K_k} \langle A_k \rangle$. So m'_k must be **recv** d_k . Now if $d \notin \text{FC}(v)$, then applying NODE-SEND yields a reduction. Otherwise if $d \in \text{FC}(v)$, then applying NODE-FORWARD yields a reduction.

This concludes the VALID-NODE case. □

Finally, we can prove the main global progress theorem.

THEOREM 5.93 (GLOBAL PROGRESS). *Given a reachable configuration P , either*

- $P \equiv \langle \text{return} () \rangle$, or
- *there exists Q such that $P \Rightarrow Q$ and Q is reachable.*

PROOF. Since P is reachable, there exists a spawning tree $\mathcal{P}$ such that $\Vdash \mathcal{P}$ and $|\mathcal{P}| = P$. By Lemma 5.92, either $\mathcal{P}$ Terminal or there exists $\mathcal{P}'$ such that $\mathcal{P} \Rightarrow \mathcal{P}'$.

In the case that $\mathcal{P}$ Terminal we have $P = |\mathcal{P}| \equiv \langle \text{return} () \rangle$ by Lemma 5.89.

In the case that there exists $\mathcal{P}'$ such that $\mathcal{P} \Rightarrow \mathcal{P}'$, by Lemma 5.87 we have $\Vdash \mathcal{P}'$. Let $Q = |\mathcal{P}'|$. By Lemma 5.86, we have $P \Rightarrow Q$ and Q is also reachable by definition. $\square$

5.4 Erasure Safety

To show that ghost messages can indeed be safely erased without affecting the behavior of processes, we define an erasure relation which replaces all ghost messages with the opaque value $\square$. Since $\square$ cannot be inspected, it cannot affect the behavior of processes. Thus if each step of an original process configuration can be simulated by its erased counterpart, then we can safely erase ghost messages. We define two kinds of erasure relations for programs and processes, respectively.

- $\Theta; \Gamma; \Delta \vdash m \sim m' : A$: the program m (of type A under context Θ, Γ, Δ) is erased to m' .
- $\Theta \Vdash P \sim P'$: the process P is erased to P' .

The formal rules for these relations are as follows.

Core Erasure. The erasure rules for the functional core of TLL_C .

$\frac{\text{VAR} \quad \epsilon; \Gamma, x : A; \Delta, x :_S A \vdash \quad \Delta \triangleright U}{\epsilon; \Gamma, x : A; \Delta, x :_S A \vdash x : A}$	$\frac{\text{CONVERSION} \quad \Gamma \vdash B : s \quad \Theta; \Gamma; \Delta \vdash m \sim m' : A \quad A \simeq B}{\Theta; \Gamma; \Delta \vdash m \sim m' : B}$
$\frac{\text{EXPLICIT-LAM} \quad \Theta; \Gamma, x : A; \Delta, x :_S A \vdash m \sim m' : B \quad \Theta \triangleright t \quad \Delta \triangleright t}{\Theta; \Gamma; \Delta \vdash \lambda_t(x : A).m \sim \lambda_t(x : \square).m' : \Pi_t(x : A).B}$	$\frac{\text{IMPLICIT-LAM} \quad \Theta; \Gamma, x : A; \Delta \vdash m \sim m' : B \quad \Theta \triangleright t \quad \Delta \triangleright t}{\Theta; \Gamma; \Delta \vdash \lambda_t\{x : A\}.m \sim \lambda_t\{x : \square\}.m' : \Pi_t\{x : A\}.B}$
$\frac{\text{EXPLICIT-APP} \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : \Pi_t(x : A).B \quad \Theta_2; \Gamma; \Delta_2 \vdash n \sim n' : A}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m n \sim m' n' : B[n/x]}$	$\frac{\text{IMPLICIT-APP} \quad \Theta; \Gamma; \Delta \vdash m \sim m' : \Pi_t\{x : A\}.B \quad \Gamma \vdash n : A}{\Theta; \Gamma; \Delta \vdash m \{n\} \sim m' \{\square\} : B[n/x]}$
$\frac{\text{EXPLICIT-PAIR} \quad \Gamma \vdash \Sigma_t(x : A).B : t \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : A \quad \Theta_2; \Gamma; \Delta_2 \vdash n \sim n' : B[m/x]}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash \langle m, n \rangle_t \sim \langle m', n' \rangle_t : \Sigma_t(x : A).B}$	$\frac{\text{IMPLICIT-PAIR} \quad \Gamma \vdash \Sigma_t\{x : A\}.B : t \quad \Gamma \vdash m : A \quad \Theta; \Gamma; \Delta \vdash n \sim n' : B[m/x]}{\Theta; \Gamma; \Delta \vdash \langle \{m\}, n \rangle_t \sim \langle \{\square\}, n' \rangle_t : \Sigma_t\{x : A\}.B}$
$\frac{\text{EXPLICIT-SUMELIM} \quad \Gamma, z : \Sigma_t(x : A).B \vdash C : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : \Sigma_t(x : A).B \quad \Theta_2; \Gamma, x : A, y : B; \Delta_2, x :_{r1} A, y :_{r2} B \vdash n \sim n' : C[\langle x, y \rangle_t / z]}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m, [x, y]n) \sim R_{[z]C}^\Sigma(m', [x, y]n') : C[m/z]}$	
$\frac{\text{IMPLICIT-SUMELIM} \quad \Gamma, z : \Sigma_t\{x : A\}.B \vdash C : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : \Sigma_t\{x : A\}.B \quad \Theta_2; \Gamma, x : A, y : B; \Delta_2, y :_r B \vdash n \sim n' : C[\langle \{x\}, y \rangle_t / z]}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]C}^\Sigma(m, [x, y]n) \sim R_{[z]C}^\Sigma(m', [x, y]n') : C[m/z]}$	

Data Erasure. The erasure rules for basic data types.

$$\begin{array}{c}
\text{UNITVAL} \\
\frac{\Gamma; \Delta \vdash \quad \Delta \triangleright U}{\epsilon; \Gamma; \Delta \vdash () \sim () : \text{unit}} \\
\\
\text{TRUE} \\
\frac{\Gamma; \Delta \vdash \quad \Delta \triangleright U}{\epsilon; \Gamma; \Delta \vdash \text{true} \sim \text{true} : \text{bool}} \\
\\
\text{FALSE} \\
\frac{\Gamma; \Delta \vdash \quad \Delta \triangleright U}{\epsilon; \Gamma; \Delta \vdash \text{false} \sim \text{false} : \text{bool}} \\
\\
\text{BOOLELIM} \\
\frac{\Gamma, z : \text{bool} \vdash A : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : \text{bool} \quad \Theta_2; \Gamma; \Delta_2 \vdash n_1 \sim n'_1 : A[\text{true}/z] \quad \Theta_2; \Gamma; \Delta_2 \vdash n_2 \sim n'_2 : A[\text{false}/z]}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash R_{[z]A}^{\text{bool}}(m, n_1, n_2) \sim R_{[z]A}^{\text{bool}}(m', n'_1, n'_2) : A[m/z]}
\end{array}$$

Monadic Erasure. The erasure rules for the monadic constructs.

$$\begin{array}{c}
\text{RETURN} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : A}{\Theta; \Gamma; \Delta \vdash \text{return } m \sim \text{return } m' : C(A)} \\
\\
\text{BIND} \\
\frac{\Gamma \vdash B : s \quad \Theta_1; \Gamma; \Delta_1 \vdash m \sim m' : C(A) \quad \Theta_2; \Gamma, x : A; \Delta_2, x :_r A \vdash n \sim n' : C(B)}{\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash \text{let } x \leftarrow m \text{ in } n \sim \text{let } x \leftarrow m' \text{ in } n' : C(B)}
\end{array}$$

Session Erasure. The erasure rules for concurrency primitives.

$$\begin{array}{c}
\text{CHANNEL-CH} \\
\frac{\Gamma; \Delta \vdash \quad \epsilon \vdash A : \text{proto} \quad \Delta \triangleright U}{c :_{\text{L}} \text{ch}\langle A \rangle; \Gamma; \Delta \vdash c \sim c : \text{ch}\langle A \rangle} \\
\\
\text{EXPLICIT-SEND-CH} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{ch}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{send } m \sim \text{send } m' : \Pi_{\text{L}}\{x : A\}. C(\text{ch}\langle B \rangle)} \\
\\
\text{IMPLICIT-SEND-CH} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{ch}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{send}} m \sim \underline{\text{send}} m' : \Pi_{\text{L}}\{x : A\}. C(\text{ch}\langle B \rangle)} \\
\\
\text{EXPLICIT-RECV-CH} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{ch}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{recv } m \sim \text{recv } m' : C(\Sigma_{\text{L}}\{x : A\}. \text{ch}\langle B \rangle)} \\
\\
\text{IMPLICIT-RECV-CH} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{ch}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{recv}} m \sim \underline{\text{recv}} m' : C(\Sigma_{\text{L}}\{x : A\}. \text{ch}\langle B \rangle)} \\
\\
\text{CHANNEL-HC} \\
\frac{\Gamma; \Delta \vdash \quad \epsilon \vdash A : \text{proto} \quad \Delta \triangleright U}{c :_{\text{L}} \text{hc}\langle A \rangle; \Gamma; \Delta \vdash c \sim c : \text{hc}\langle A \rangle} \\
\\
\text{EXPLICIT-SEND-HC} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{hc}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{send } m \sim \text{send } m' : \Pi_{\text{L}}\{x : A\}. C(\text{hc}\langle B \rangle)} \\
\\
\text{IMPLICIT-SEND-HC} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{hc}\langle ?\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{send}} m \sim \underline{\text{send}} m' : \Pi_{\text{L}}\{x : A\}. C(\text{hc}\langle B \rangle)} \\
\\
\text{EXPLICIT-RECV-HC} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{hc}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \text{recv } m \sim \text{recv } m' : C(\Sigma_{\text{L}}\{x : A\}. \text{hc}\langle B \rangle)} \\
\\
\text{IMPLICIT-RECV-HC} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{hc}\langle !\{x : A\}. B \rangle}{\Theta; \Gamma; \Delta \vdash \underline{\text{recv}} m \sim \underline{\text{recv}} m' : C(\Sigma_{\text{L}}\{x : A\}. \text{hc}\langle B \rangle)} \\
\\
\text{FORK} \\
\frac{\Theta; \Gamma, x : \text{ch}\langle A \rangle; \Delta, x :_{\text{L}} \text{ch}\langle A \rangle \vdash m \sim m' : C(\text{unit})}{\Theta; \Gamma; \Delta \vdash \text{fork } (x : \text{ch}\langle A \rangle) \text{ with } m \sim \text{fork } (x : \square) \text{ with } m' : C(\text{hc}\langle A \rangle)} \\
\\
\text{CLOSE} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{ch}\langle 1 \rangle}{\Theta; \Gamma; \Delta \vdash \text{close } m \sim \text{close } m' : C(\text{unit})} \\
\\
\text{WAIT} \\
\frac{\Theta; \Gamma; \Delta \vdash m \sim m' : \text{hc}\langle 1 \rangle}{\Theta; \Gamma; \Delta \vdash \text{wait } m \sim \text{wait } m' : C(\text{unit})}
\end{array}$$

Process Erasure. The erasure rules for processes.

$$\begin{array}{c}
\text{EXPR} \\
\frac{\Theta; \epsilon \vdash m \sim m' : C(\text{unit})}{\Theta \Vdash \langle m \rangle \sim \langle m' \rangle} \\
\\
\text{PAR} \\
\frac{\Theta_1 \vdash P \sim P' \quad \Theta_2 \vdash Q \sim Q'}{\Theta_1 \cup \Theta_2 \Vdash (P \mid Q) \sim (P' \mid Q')} \\
\\
\text{SCOPE} \\
\frac{\Theta, c :_{\text{L}} \text{ch}\langle A \rangle, d :_{\text{L}} \text{hc}\langle A \rangle \vdash P \sim P'}{\Theta \Vdash \text{vcd}.P \sim \text{vcd}.P'}
\end{array}$$

The steps to show erasure safety is similar to that of session fidelity Section 5.2. We begin by stating the weakening and substitution lemmas for the erasure relation. The proofs are routine inductions on the derivations of the erasure judgments.

LEMMA 5.94 (IMPLICIT WEAKENING FOR ERASURE). *If $\Theta; \Gamma; \Delta \vdash m \sim m' : A$, then for any $\Gamma \vdash B : s$ and $x \notin \Gamma$, we have $\Theta; \Gamma, x : B; \Delta \vdash m \sim m' : A$.*

LEMMA 5.95 (EXPLICIT WEAKENING FOR ERASURE). *If $\Theta; \Gamma; \Delta \vdash m \sim m' : A$, then for any $\Gamma \vdash B : U$ and $x \notin \Gamma$, we have $\Theta; \Gamma, x : B; \Delta, x :_U B \vdash m \sim m' : A$.*

LEMMA 5.96 (IMPLICIT SUBSTITUTION FOR ERASURE). *If $\Theta; \Gamma, x : A; \Delta \vdash m \sim m' : B$ and $\Gamma \vdash n : A$, then $\Theta; \Gamma; \Delta \vdash m[n/x] \sim m'[\square/x] : B[n/x]$.*

LEMMA 5.97 (EXPLICIT SUBSTITUTION FOR ERASURE). *If $\Theta_1; \Gamma, x : A; \Delta_1, x :_s A \vdash m \sim m' : B$ and $\Theta_2; \Gamma; \Delta_2 \vdash n \sim n' : A$, and $\Theta_2 \triangleright s$, $\Delta_2 \triangleright s$, then $\Theta_1 \cup \Theta_2; \Gamma; \Delta_1 \cup \Delta_2 \vdash m[n/x] \sim m'[n'/x] : B[n/x]$.*

The erasure relation implies program typing.

LEMMA 5.98 (ERASURE TYPING). *If $\Theta; \Gamma; \Delta \vdash m \sim m' : A$, then $\Theta; \Gamma; \Delta \vdash m : A$ and $\Gamma \vdash m : A$.*

PROOF. To show $\Theta; \Gamma; \Delta \vdash m : A$, we proceed by induction on the derivation of $\Theta; \Gamma; \Delta \vdash m \sim m' : A$. All cases are straightforward as they are compatible with the typing rules.

To show $\Gamma \vdash m : A$, we apply Theorem 5.66 to the judgment $\Theta; \Gamma; \Delta \vdash m : A$. $\square$

The types of the erasure relation are valid.

LEMMA 5.99 (ERASURE TYPE VALIDITY). *If $\Theta; \Gamma; \Delta \vdash m \sim m' : A$, then $\epsilon \vdash A : s$ for some s .*

PROOF. By Lemma 5.98, we have $\Theta; \Gamma; \Delta \vdash m : A$. The result then follows from Theorem 5.67. $\square$

Shape Preservation. Next, we prove “shape preservation” lemmas for the erasure relation. Intuitively, these lemmas state that the erasure relation preserve the overall shape of programs. The proofs are routine inductions on the derivations of the erasure judgments.

LEMMA 5.100. $\Theta; \Gamma; \Delta \vdash x \sim m' : B$, then $m' = x$.

LEMMA 5.101. *If $\Theta; \Gamma; \Delta \vdash \lambda_t(x : A).m \sim m' : B$, then $m' = \lambda_t(x : \square).n'$ for some n' .*

LEMMA 5.102. *If $\Theta; \Gamma; \Delta \vdash \lambda_t\{x : A\}.m \sim m' : B$, then $m' = \lambda_t\{x : \square\}.n'$ for some n' .*

LEMMA 5.103. *If $\Theta; \Gamma; \Delta \vdash m \ n \sim m' : B$, then $m' = n'_1 \ n'_2$ for some n'_1, n'_2 .*

LEMMA 5.104. *If $\Theta; \Gamma; \Delta \vdash m \{n\} \sim m' : B$, then $m' = n'_1 \{n'_2\}$ for some n'_1, n'_2 .*

LEMMA 5.105. *If $\Theta; \Gamma; \Delta \vdash \langle m, n \rangle_t \sim m' : B$, then $m' = \langle n'_1, n'_2 \rangle_t$ for some n'_1, n'_2 .*

LEMMA 5.106. *If $\Theta; \Gamma; \Delta \vdash \langle \{m\}, n \rangle_t \sim m' : B$, then $m' = \langle \{n'_1\}, n'_2 \rangle_t$ for some n'_1, n'_2 .*

LEMMA 5.107. *If $\Theta; \Gamma; \Delta \vdash R_{[z]C}^\Sigma(m, [x, y]n) \sim m' : B$, then $m' = R_{\square}^\Sigma(n'_1, [x, y]n'_2)$ for some n'_1, n'_2 .*

LEMMA 5.108. *If $\Theta; \Gamma; \Delta \vdash () \sim m' : B$, then $m' = ()$.*

LEMMA 5.109. *If $\Theta; \Gamma; \Delta \vdash \text{true} \sim m' : B$, then $m' = \text{true}$.*

LEMMA 5.110. *If $\Theta; \Gamma; \Delta \vdash \text{false} \sim m' : B$, then $m' = \text{false}$.*

LEMMA 5.111. *If $\Theta; \Gamma; \Delta \vdash R_{[z]A}^{\text{bool}}(m, n_1, n_2) \sim m' : B$, then $m' = R_{\square}^{\text{bool}}(n', n'_1, n'_2)$ for some n', n'_1, n'_2 .*

LEMMA 5.112. *If $\Theta; \Gamma; \Delta \vdash \text{return } m \sim m' : B$, then $m' = \text{return } n'$ for some n' .*

LEMMA 5.113. *If $\Theta; \Gamma; \Delta \vdash \text{let } x \Leftarrow m \text{ in } n \sim m' : B$, then $m' = \text{let } x \Leftarrow n'_1 \text{ in } n'_2$ for some n'_1, n'_2 .*

LEMMA 5.114. If $c :_L \mathbf{ch}\langle A \rangle; \Gamma; \Delta \vdash c \sim m' : B$, then $m' = c$.

LEMMA 5.115. If $\Theta; \Gamma; \Delta \vdash \mathbf{send} m \sim m' : B$, then $m' = \mathbf{send} n'$ for some n' .

LEMMA 5.116. If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} m \sim m' : B$, then $m' = \mathbf{recv} n'$ for some n' .

LEMMA 5.117. If $\Theta; \Gamma; \Delta \vdash \mathbf{send} m \sim m' : B$, then $m' = \mathbf{send} n'$ for some n' .

LEMMA 5.118. If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} m \sim m' : B$, then $m' = \mathbf{recv} n'$ for some n' .

LEMMA 5.119. If $\Theta; \Gamma; \Delta \vdash \mathbf{fork}(x : \mathbf{ch}\langle A \rangle) \mathbf{with} m \sim m' : B$, then $m' = \mathbf{fork}(x : \square) \mathbf{with} n'$ for some n' .

LEMMA 5.120. If $\Theta; \Gamma; \Delta \vdash \mathbf{close} m \sim m' : B$, then $m' = \mathbf{close} n'$ for some n' .

LEMMA 5.121. If $\Theta; \Gamma; \Delta \vdash \mathbf{wait} m \sim m' : B$, then $m' = \mathbf{wait} n'$ for some n' .

We also show that the erasure relation preserves the structure of thunks and values.

LEMMA 5.122. Given $\Theta; \Gamma; \Delta \vdash m \sim m' : A$, if m is a value, then m' is also a value, and if m is a thunk, then m' is also a thunk.

PROOF. By straightforward induction on the derivation of $\Theta; \Gamma; \Delta \vdash m \sim m' : A$ and applying the shape preservation lemmas. $\square$

Inversion Lemmas. We now define inversion lemmas for the erasure relation. When used in conjunction with the shape preservation lemmas, these lemmas allow us to invert an erasure judgment to obtain erasure judgments for its subterms. The proofs are routine inductions on the derivations of the erasure judgments.

LEMMA 5.123. If $\Theta; \Gamma; \Delta \vdash \lambda_t\{x : A_1\}.m_1 \sim \lambda_t\{x : A_2\}.m_2 : \Pi_s\{x : A_3\}.B$, then $\Theta; \Gamma, x : A_3; \Delta \vdash m_1 \sim m_2 : B$.

LEMMA 5.124. If $\Theta; \Gamma; \Delta \vdash \lambda_t(x : A_1).m_1 \sim \lambda_t(x : A_2).m_2 : \Pi_s(x : A_3).B$, then $\Theta; \Gamma, x : A_3; \Delta, x :_r A_3 \vdash m_1 \sim m_2 : B$ for some r .

LEMMA 5.125. If $\Theta; \Gamma; \Delta \vdash \langle \{m_1\}, n_1 \rangle_t \sim \langle \{m_2\}, n_2 \rangle_t : \Sigma_s\{x : A\}.B$, then $s = r$, $m_2 = \square$, $\Gamma \vdash m_1 : A$, and $\Theta; \Gamma; \Delta \vdash n_1 \sim n_2 : B.[m_1/x]$.

LEMMA 5.126. If $\Theta; \Gamma; \Delta \vdash \langle m_1, n_1 \rangle_t \sim \langle m_2, n_2 \rangle_t : \Sigma_s(x : A).B$, then there exist $\Theta_1, \Theta_2, \Delta_1, \Delta_2$ such that $s = r$, $\Theta_1 \cup \Theta_2 = \Theta$, $\Delta_1 \cup \Delta_2 = \Delta$, $\Theta_1; \Gamma; \Delta_1 \vdash m_1 \sim m_2 : A$, and $\Theta_2; \Gamma; \Delta_2 \vdash n_1 \sim n_2 : B[m_1/x]$.

LEMMA 5.127. If $\Theta; \Gamma; \Delta \vdash m_1 \{n_1\} \sim m_2 \{n_2\} : C$, then there exist A, B, s such that $n_2 = \square$, $\Theta; \Gamma; \Delta \vdash m_1 \sim m_2 : \Pi_s\{x : A\}.B$, $\Gamma \vdash n_1 : A$, and $C \simeq B[n_1/x]$.

LEMMA 5.128. If $\Theta; \Gamma; \Delta \vdash m_1 n_1 \sim m_2 n_2 : C$, then there exist $A, B, s, \Theta_1, \Theta_2, \Delta_1, \Delta_2$ such that $\Theta_1 \cup \Theta_2 = \Theta$, $\Delta_1 \cup \Delta_2 = \Delta$, $\Theta_1; \Gamma; \Delta_1 \vdash m_1 \sim m_2 : \Pi_s(x : A).B$, $\Theta_2; \Gamma; \Delta_2 \vdash n_1 \sim n_2 : A$, and $C \simeq B[n_1/x]$.

LEMMA 5.129. If $\Theta; \Gamma; \Delta \vdash \mathbf{return} m_1 \sim \mathbf{return} m_2 : C(A)$, then $\Theta; \Gamma; \Delta \vdash m_1 \sim m_2 : A$.

LEMMA 5.130. If $\Theta; \Gamma; \Delta \vdash \mathbf{let} x \leftarrow m_1 \mathbf{in} n_1 \sim \mathbf{let} x \leftarrow m_2 \mathbf{in} n_2 : C(B)$, then there exists $\Theta_1, \Theta_2, \Delta_1, \Delta_2, A, s$ such that $\Theta_1 \cup \Theta_2 = \Theta$, $\Delta_1 \cup \Delta_2 = \Delta$, $\Theta_1; \Gamma; \Delta_1 \vdash m_1 \sim m_2 : C(A)$, and $\Theta_2; \Gamma, x : A; \Delta_2, x :_r A \vdash n_1 \sim n_2 : C(B)$ and $x \notin FV(B)$.

LEMMA 5.131. If $\Theta; \Gamma; \Delta \vdash \mathbf{send} m \sim \mathbf{send} m' : C$, then there exists A, B such that either $C \simeq \Pi_L(x : A).C(\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{ch}\langle!(x : A). B\rangle$ or $C \simeq \Pi_L(x : A).C(\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{hc}\langle?(x : A). B\rangle$.

LEMMA 5.132. *If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} \, m \sim \mathbf{recv} \, m' : C$, then there exists A, B such that either $C \simeq C(\Sigma_L(x : A). \mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{ch}\langle ?(x : A). B \rangle$ or $C \simeq C(\Sigma_L(x : A). \mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{hc}\langle !(x : A). B \rangle$.*

LEMMA 5.133. *If $\Theta; \Gamma; \Delta \vdash \mathbf{send} \, m \sim \mathbf{send} \, m' : C$, then there exists A, B such that either $C \simeq \Pi_L\{x : A\}. C(\mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{ch}\langle !\{x : A\}. B \rangle$ or $C \simeq \Pi_L\{x : A\}. C(\mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{hc}\langle ?\{x : A\}. B \rangle$.*

LEMMA 5.134. *If $\Theta; \Gamma; \Delta \vdash \mathbf{recv} \, m \sim \mathbf{recv} \, m' : C$, then there exists A, B such that either $C \simeq C(\Sigma_L\{x : A\}. \mathbf{ch}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{ch}\langle ?\{x : A\}. B \rangle$ or $C \simeq C(\Sigma_L\{x : A\}. \mathbf{hc}\langle B \rangle)$ and $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{hc}\langle !\{x : A\}. B \rangle$.*

LEMMA 5.135. *If $\Theta; \Gamma; \Delta \vdash \mathbf{fork} \, (x : A_1) \mathbf{with} \, m_1 \sim \mathbf{fork} \, (x : A_2) \mathbf{with} \, m_2 : B$, then exists A such that $A_1 = \mathbf{ch}\langle A \rangle, A_2 = \square, B \simeq C(\mathbf{hc}\langle A \rangle), \Gamma \vdash A : \mathbf{proto}$, and $\Theta; \Gamma, x : \mathbf{ch}\langle A \rangle; \Delta, x :_L \mathbf{ch}\langle A \rangle \vdash m_1 \sim m_2 : C(\mathbf{unit})$.*

LEMMA 5.136. *If $\Theta; \Gamma; \Delta \vdash \mathbf{close} \, m \sim \mathbf{close} \, m' : A$, then $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{ch}\langle 1 \rangle$ and $A \simeq C(\mathbf{unit})$.*

LEMMA 5.137. *If $\Theta; \Gamma; \Delta \vdash \mathbf{wait} \, m \sim \mathbf{wait} \, m' : A$, then $\Theta; \Gamma; \Delta \vdash m \sim m' : \mathbf{hc}\langle 1 \rangle$ and $A \simeq C(\mathbf{unit})$.*

Erasure Congruence. Similarly to the proof of session fidelity, we prove that the erasure relation respects structural congruence.

LEMMA 5.138. *If $\Theta \Vdash P \sim P'$ and $P \equiv Q$, then there exists Q' such that $P' \equiv Q'$ and $\Theta \Vdash Q \sim Q'$.*

PROOF. By induction on the derivation of $\Theta \Vdash P \sim P'$. We have three cases:

Case EXPR:

$$\frac{\text{EXPR} \quad \Theta; \epsilon; \epsilon \vdash m \sim m' : C(\mathbf{unit})}{\Theta \Vdash \langle m \rangle \sim \langle m' \rangle}$$

It suffices to consider the case where $\langle m \rangle \equiv (\langle m \rangle \mid \langle \mathbf{return} \, () \rangle)$, i.e. $Q = \langle m \rangle \mid \langle \mathbf{return} \, () \rangle$. From erasure rules RETURN and UNITVAL, we have

$$\epsilon; \epsilon; \epsilon \vdash \mathbf{return} \, () \sim \mathbf{return} \, () : C(\mathbf{unit})$$

which gives us $\epsilon \Vdash \langle \mathbf{return} \, () \rangle \sim \langle \mathbf{return} \, () \rangle$ by rule EXPR. By rule PAR, we have

$$\Theta \Vdash (\langle m \rangle \mid \langle \mathbf{return} \, () \rangle) \sim (\langle m' \rangle \mid \langle \mathbf{return} \, () \rangle)$$

which concludes this case by taking $Q' = \langle m' \rangle \mid \langle \mathbf{return} \, () \rangle$.

Case PAR:

$$\frac{\text{PAR} \quad \frac{\Theta_1 \vdash P \sim P' \quad \Theta_2 \vdash Q_0 \sim Q'_0}{\Theta_1 \cup \Theta_2 \Vdash (P \mid Q_0) \sim (P' \mid Q'_0)}}{\Theta \Vdash (P \mid Q_0) \sim (P' \mid Q'_0)}$$

By case analysis on the congruence relation we have the following sub-cases:

- (1) $P \mid Q_1 \equiv Q_1 \mid P$
- (2) $P \mid (Q_1 \mid Q_2) \equiv (P \mid Q_1) \mid Q_2$
- (3) $P \mid \langle \mathbf{return} \, () \rangle \equiv P$
- (4) $\mathbf{vcd}.P \mid Q_1 \equiv \mathbf{vcd}.(P \mid Q_1)$

In sub-case (1), by PAR we have $\Theta_2 \cup \Theta_1 \Vdash (Q_1 \mid P) \sim (Q'_1 \mid P')$. By the commutativity of $\cup$, we conclude this case by taking $Q' = Q'_1 \mid P'$.

In sub-case (2), by PAR we have $(\Theta_1 \cup \Theta_2) \cup \Theta_3 \Vdash ((P \mid Q_1) \mid Q_2) \sim ((P' \mid Q'_1) \mid Q'_2)$. By the associativity of $\cup$, we conclude this case by taking $Q' = (P' \mid Q'_1) \mid Q'_2$.

In sub-case (3), we have $Q_0 = \langle \text{return } () \rangle$. By assumption we have $\Theta_2 \Vdash \langle \text{return } () \rangle \sim Q'_1$ for some Q'_1 . By inversion on the erasure judgment, we have $\Theta_2 = \epsilon$. Thus we have $\Theta \vdash P \sim P'$ and we can conclude this case by taking $Q' = P'$.

In sub-case (4), we have $\Theta_1 \Vdash \text{vcd}.P \sim P'$ and $\Theta_2 \Vdash Q_1 \sim Q'_1$. By inversion on $\Theta_1 \Vdash \text{vcd}.P \sim P'$ we know there exists P'' and A such that $P' = \text{vcd}.P''$ and $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \sim P''$. By PAR we have $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \cup \Theta_2 \Vdash P \mid Q_1 \sim P'' \mid Q'_1$. Since $c, d \notin \Theta_2$, we have $\Theta_1 \cup \Theta_2, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \mid Q_1 \sim P'' \mid Q'_1$. By SCOPE we have $\Theta_1 \cup \Theta_2 \Vdash \text{vcd}.(P \mid Q_1) \sim \text{vcd}.(P'' \mid Q'_1)$. We conclude this case by taking $Q' = \text{vcd}.(P'' \mid Q'_1)$.

Case SCOPE:

$$\frac{\text{SCOPE} \quad \Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \vdash P \sim P'}{\Theta \Vdash \text{vcd}.P \sim \text{vcd}.P'}$$

By case analysis on the congruence relation we have the following sub-cases:

- (1) $\text{vcd}.(P \mid Q_1) \equiv \text{vcd}.P \mid Q_1$
- (2) $\text{vcd}.P \equiv \text{vdc}.P$
- (3) $\text{vcd}.vc'd'.P \equiv vc'd' . \text{vcd}.P$

In sub-case (1), we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \vdash P \mid Q_1 \sim P' \mid Q'_1$ by assumption. By inversion on the erasure judgment, we know there exists Θ_1 and Θ_2 such that $\Theta_1 \cup \Theta_2 = \Theta$ and $\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \vdash P \sim P'$ and $\Theta_2 \vdash Q_1 \sim Q'_1$. Channels c and d must be distributed to the erasure relation of P as they are linear and do not appear in Q_1 . Applying SCOPE we have $\Theta_1 \Vdash \text{vcd}.P \sim \text{vcd}.P'$. By PAR we have $\Theta_1 \cup \Theta_2 \Vdash \text{vcd}.P \mid Q_1 \sim \text{vcd}.P' \mid Q'_1$. We conclude this case by taking $Q' = \text{vcd}.P' \mid Q'_1$.

In sub-case (2), we have $\Theta, d :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, c :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \vdash P \sim P'$ by assumption. By exchange of the context, we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \vdash P \sim P'$. By SCOPE we have $\Theta \Vdash \text{vcd}.P \sim \text{vdc}.P'$. We conclude this case by taking $Q' = \text{vdc}.P'$.

In sub-case (3), we have $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle, c' :_{\mathbb{L}} \mathbf{ch}\langle A' \rangle, d' :_{\mathbb{L}} \mathbf{hc}\langle A' \rangle \vdash P \sim P'$ by assumption. By exchange, we have $\Theta, c' :_{\mathbb{L}} \mathbf{ch}\langle A' \rangle, d' :_{\mathbb{L}} \mathbf{hc}\langle A' \rangle, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \sim P'$. Applying SCOPE twice we have $\Theta \Vdash vc'd' . \text{vcd}.P \sim vc'd' . \text{vcd}.P'$. We conclude this case by taking $Q' = vc'd' . \text{vcd}.P'$. $\square$

Erasure Simulation. We can now prove the simulation lemma for programs.

THEOREM 5.139 (TERM SIMULATION). *Given $\Theta; \epsilon; \epsilon \vdash m \sim m' : A$ and $m \rightsquigarrow n$, there exists n'_0 such that $m' \rightsquigarrow n'_0$ and $\Theta; \epsilon; \epsilon \vdash n \sim n'_0 : A$.*

PROOF. The proof is similar to the proof of program subject reduction (Theorem 5.73). We process by induction on the derivation of $\Theta; \epsilon; \epsilon \vdash m \sim m' : A$. We show the following representative cases:

Case (EXPLICIT-APP):

$$\frac{\text{EXPLICIT-APP} \quad \Theta_1; \epsilon; \epsilon \vdash m \sim m' : \Pi_t(x : A).B \quad \Theta_2; \epsilon; \epsilon \vdash n \sim n' : A}{\Theta_1 \cup \Theta_2; \epsilon; \epsilon \vdash m \ n \sim m' \ n' : B[n/x]}$$

From case analysis on the reduction we have three sub-cases: (1) STEP-EXPLICIT-APP₁, (2) STEP-EXPLICIT-APP₂, and (3) STEP-EXPLICIT- β .

In sub-case (1), we have $m \rightsquigarrow m_1$. By the induction hypothesis, there exists m'_1 such that $\Theta_1; \epsilon; \epsilon \vdash m_1 \sim m'_1 : \Pi_t(x : A).B$ and $m' \rightsquigarrow m'_1$. By EXPLICIT-APP we have $\Theta_1 \cup \Theta_2; \epsilon; \epsilon \vdash m_1 \ n \sim m'_1 \ n' : B[n/x]$. We conclude this case by taking $n'_0 = m'_1 \ n'$ since by STEP-EXPLICIT-APP₁ we have $m' \ n' \rightsquigarrow m'_1 \ n'$.

In sub-case (2), we have $n \rightsquigarrow n_1$. By the induction hypothesis, there exists n'_1 such that $\Theta_2; \epsilon; \epsilon \vdash n_1 \sim n'_1 : A$ and $n' \rightsquigarrow n'_1$. By EXPLICIT-APP we have $\Theta_1 \cup \Theta_2; \epsilon; \epsilon \vdash m \ n_1 \sim m' \ n'_1 : B[n_1/x]$. By Lemma 5.71, we have $B[n/x] \simeq B[n_1/x]$ which gives us $\Theta_1 \cup \Theta_2; \epsilon; \epsilon \vdash m \ n_1 \sim m' \ n'_1 : B[n/x]$ by CONVERSION. We conclude this case by taking $n'_0 = m' \ n'_1$ since by STEP-EXPLICIT-APP₂ we have $m' \ n' \rightsquigarrow m' \ n'_1$.

In sub-case (3), we have $m = \lambda_t(x : A').m_0$ and $n = v$ for some value v .

Applying Lemma 5.122 on the typing judgment

$$\Theta_2; \epsilon; \epsilon \vdash v \sim n' : A$$

we know that $n' = v'$ for some value v' . By Lemma 5.98 we have $\Theta_2; \epsilon; \epsilon \vdash v : A$. By Lemma 5.99 we have $\epsilon \vdash A : s$ for some s . By Lemma 5.72 we have $\Theta_2 \triangleright s$.

Applying Lemma 5.101 on the typing judgment

$$\Theta_1; \epsilon; \epsilon \vdash \lambda_t(x : A').m_0 \sim m' : \Pi_t(x : A).B$$

we know that $m' = \lambda_t(x : \square).m'_0$ for some m'_0 . By Lemma 5.124 there exists r such that

$$\Theta_1; x : A; x :_r A \vdash m_0 \sim m'_0 : B$$

By validity of context $x : A; x :_r A \vdash$ we have $\epsilon \vdash A : r$. By Theorem 5.37 we have $s = r$.

By Lemma 5.97 we have

$$\Theta_1 \cup \Theta_2; \epsilon; \epsilon \vdash m_0[v/x] \sim m'_0[v'/x] : B[v/x]$$

We conclude this case by taking $n'_0 = m'_0[v'/x]$ since we have $(\lambda_t(x : \square).m'_0) \ v' \rightsquigarrow m'_0[v'/x]$ by STEP-EXPLICIT- β .

Case (IMPLICIT-APP):

$$\frac{\text{IMPLICIT-APP} \quad \Theta; \epsilon; \epsilon \vdash m \sim m' : \Pi_t\{x : A\}.B \quad \epsilon \vdash n : A}{\Theta; \epsilon; \epsilon \vdash m \ \{n\} \sim m' \ \{\square\} : B[n/x]}$$

From case analysis on the reduction we have two sub-cases: (1) STEP-IMPLICIT-APP and (2) STEP-IMPLICIT- β .

In sub-case (1), we have $m \rightsquigarrow m_1$. By the induction hypothesis, there exists m'_1 such that $\Theta; \epsilon; \epsilon \vdash m_1 \sim m'_1 : \Pi_t\{x : A\}.B$ and $m' \rightsquigarrow m'_1$. By IMPLICIT-APP we have $\Theta; \epsilon; \epsilon \vdash m_1 \ \{n\} \sim m'_1 \ \{\square\} : B[n/x]$. We conclude this case by taking $n'_0 = m'_1 \ \{\square\}$ since by STEP-IMPLICIT-APP we have $m' \ \{\square\} \rightsquigarrow m'_1 \ \{\square\}$.

In sub-case (2), we have $m = \lambda_t\{x : A'\}.m_0$ for some m_0 .

Applying Lemma 5.102 on the typing judgment

$$\Theta; \epsilon; \epsilon \vdash \lambda_t\{x : A'\}.m_0 \sim m' : \Pi_t\{x : A\}.B$$

we know that $m' = \lambda_t\{x : \square\}.m'_0$ for some m'_0 . By Lemma 5.123 there is

$$\Theta; x : A; \epsilon \vdash m_0 \sim m'_0 : B$$

By Lemma 5.96 we have

$$\Theta; \epsilon; \epsilon \vdash m_0[n/x] \sim m'_0[\square/x] : B[n/x]$$

We conclude this case by taking $n'_0 = m'_0[\square/x]$ since we have $(\lambda_t\{x : \square\}.m'_0) \ \{\square\} \rightsquigarrow m'_0[\square/x]$ by STEP-IMPLICIT- β . $\square$

Lemma 5.140 allows us to reason about evaluation contexts in the erasure relation.

LEMMA 5.140. *If $\Theta; \epsilon; \epsilon \vdash \mathcal{M}[m] \sim m'_0 : C(B)$ then there exists $\Theta_1, \Theta_2, \mathcal{M}', m', A$ such that $\Theta = \Theta_1 \cup \Theta_2$ and $\Theta_1; \epsilon; \epsilon \vdash m \sim m' : C(A)$ and $m'_0 = \mathcal{M}'[m']$. For any Θ_3, n, n' such that $\Theta_3; \epsilon; \epsilon \vdash n \sim n' : C(A)$ and $\Theta_2 \cup \Theta_3$ is well-defined, we have $\Theta_2 \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{M}[n] \sim \mathcal{M}'[n'] : C(B)$.*

PROOF. By induction on the structure of the evaluation context $\mathcal{M}$.

Case ($\mathcal{M} = [\cdot]$): Trivial by taking $\mathcal{M}' = [\cdot]$, $m' = m'_0$, $\Theta_1 = \Theta$, $\Theta_2 = \epsilon$, and $A = B$.

Case ($\mathcal{M} = \text{let } x \Leftarrow \mathcal{N} \text{ in } n$): We have the typing judgment

$$\Theta; \epsilon; \epsilon \vdash \text{let } x \Leftarrow \mathcal{N}[m] \text{ in } n_1 \sim m'_0 : C(B)$$

By Lemma 5.113 we have $m'_0 = \text{let } x \Leftarrow m'_1 \text{ in } n'_1$ and

$$\Theta; \epsilon; \epsilon \vdash \text{let } x \Leftarrow \mathcal{N}[m] \text{ in } n_1 \sim \text{let } x \Leftarrow m'_1 \text{ in } n'_1 : C(B)$$

By Lemma 5.130 there exists Θ'_1, Θ'_2, A' such that

$$\Theta'_1; \epsilon; \epsilon \vdash \mathcal{N}[m] \sim m'_1 : C(A')$$

$$\Theta'_2; x : A'; x :_{\perp} A' \vdash n_1 \sim n'_1 : C(B)$$

$$\Theta = \Theta'_1 \cup \Theta'_2 \quad x \notin \text{FV}(B)$$

By the induction hypothesis, there exists $\Theta'_{11}, \Theta'_{12}, \mathcal{N}', m', A''$ such that

$$\Theta'_{11}; \epsilon; \epsilon \vdash m \sim m' : C(A'')$$

$$\Theta'_1 = \Theta'_{11} \cup \Theta'_{12}$$

$$m'_1 = \mathcal{N}'[m']$$

and for any Θ_3, n, n' such that $\Theta_3; \epsilon; \epsilon \vdash n \sim n' : C(A'')$ and $\Theta'_{12} \cup \Theta_3$ is well-defined, we have

$$\Theta'_{12} \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{N}[n] \sim \mathcal{N}'[n'] : C(A')$$

By choosing $\Theta_1 = \Theta'_{11}$, $\Theta_2 = \Theta'_{12} \cup \Theta'_2$, $\mathcal{M}' = \text{let } x \Leftarrow \mathcal{N}' \text{ in } n'_1$, and $A = A''$, we have

$$\Theta_1; \epsilon; \epsilon \vdash m \sim m' : C(A'')$$

$$\Theta = \Theta_1 \cup \Theta_2$$

$$m'_0 = \text{let } x \Leftarrow \mathcal{N}'[m'] \text{ in } n'_1 = \mathcal{M}'[m']$$

For any Θ_3, n, n' such that $\Theta_3; \epsilon; \epsilon \vdash n \sim n' : C(A'')$ and $\Theta_2 \cup \Theta_3$ is well-defined, we know that $\Theta'_{12} \cup \Theta_3$ is well-defined, which means that

$$\Theta'_{12} \cup \Theta_3; \epsilon; \epsilon \vdash \mathcal{N}[n] \sim \mathcal{N}'[n'] : C(A')$$

Now applying BIND we have

$$\Theta_2 \cup \Theta_3; \epsilon; \epsilon \vdash \text{let } x \Leftarrow \mathcal{N}[n] \text{ in } n_1 \sim \text{let } x \Leftarrow \mathcal{N}'[n'] \text{ in } n'_1 : C(B)$$

which concludes this case. $\square$

We can now prove the main simulation theorem for processes.

THEOREM 5.141 (PROCESS SIMULATION). *Given $\Theta \Vdash P \sim P'$ and reduction $P \Rightarrow Q_0$, there exists Q'_0 such that $P' \Rightarrow Q'_0$ and $\Theta \Vdash Q_0 \sim Q'_0$.*

PROOF. Similar to the proof of session fidelity (Theorem 5.84). We proceed by induction on the derivation of $P \Rightarrow Q$.

Case (PROC-FORK):

$$\langle \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with} m] \rangle \Rightarrow vcd.(\langle \mathcal{N}[\mathbf{return} c] \rangle \mid \langle m[d/x] \rangle)$$

By assumption, we have $\Theta \Vdash \langle \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with} m] \rangle \sim P'$. By inversion on the erasure judgment we know there exists n' such that $P' = \langle n' \rangle$ and

$$\Theta; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{fork}(x : A) \mathbf{with} m] \sim n' : C(\mathbf{unit})$$

By Lemma 5.140 there exists $\Theta_1, \Theta_2, \mathcal{N}', n'', B$ such that

$$\Theta_1; \epsilon; \epsilon \vdash \mathbf{fork}(x : A) \mathbf{with} m \sim n'' : C(B)$$

$$\Theta = \Theta_1 \cup \Theta_2$$

$$n' = \mathcal{N}'[n'']$$

By Lemma 5.119 we know that $n'' = \mathbf{fork}(x : A') \mathbf{with} m'$ for some A' and m' .

By Lemma 5.135 there exists A'' such that

$$\Theta_1; x : \mathbf{ch}\langle A'' \rangle; x :_{\mathbf{L}} \mathbf{ch}\langle A'' \rangle \vdash m \sim m' : C(\mathbf{unit})$$

$$\epsilon \vdash A'' : \mathbf{proto}$$

$$A' = \square \quad A = \mathbf{ch}\langle A'' \rangle \quad C(B) \simeq C(\mathbf{hc}\langle A'' \rangle)$$

By CHANNEL-HC we have $d :_{\mathbf{L}} \mathbf{hc}\langle A'' \rangle; \epsilon; \epsilon \vdash c \sim c : \mathbf{hc}\langle A'' \rangle$.

By CHANNEL-CH we have $c :_{\mathbf{L}} \mathbf{ch}\langle A'' \rangle; \epsilon; \epsilon \vdash d \sim d : \mathbf{ch}\langle A'' \rangle$.

By RETURN we have $d :_{\mathbf{L}} \mathbf{hc}\langle A'' \rangle; \epsilon; \epsilon \vdash \mathbf{return} c \sim \mathbf{return} c : C(\mathbf{hc}\langle A'' \rangle)$. Applying the evaluation contexts $\mathcal{N}$ and $\mathcal{N}'$ we have

$$\Theta_2, c :_{\mathbf{L}} \mathbf{hc}\langle A'' \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{return} c] \sim \mathcal{N}'[\mathbf{return} c] : C(\mathbf{unit})$$

By Lemma 5.97 we have

$$\Theta_1, d :_{\mathbf{L}} \mathbf{ch}\langle A'' \rangle; \epsilon; \epsilon \vdash m[d/x] \sim m'[d/x] : C(\mathbf{unit})$$

By EXPR and PAR we have

$$\Theta_1 \cup \Theta_2, d :_{\mathbf{L}} \mathbf{ch}\langle A'' \rangle, c :_{\mathbf{L}} \mathbf{hc}\langle A'' \rangle \Vdash (\langle \mathcal{N}[\mathbf{return} c] \rangle \mid \langle m[d/x] \rangle) \sim (\langle \mathcal{N}'[\mathbf{return} c] \rangle \mid \langle m'[d/x] \rangle)$$

By SCOPE we have

$$\Theta \Vdash vcd.(\langle \mathcal{N}[\mathbf{return} c] \rangle \mid \langle m[d/x] \rangle) \sim vcd.(\langle \mathcal{N}'[\mathbf{return} c] \rangle \mid \langle m'[d/x] \rangle)$$

which concludes this case by taking $Q'_0 = vcd.(\langle \mathcal{N}'[\mathbf{return} c] \rangle \mid \langle m'[d/x] \rangle)$ and noting that

$$\langle \mathcal{N}'[\mathbf{fork}(x : \square) \mathbf{with} m'] \rangle \Rightarrow vcd.(\langle \mathcal{N}'[\mathbf{return} c] \rangle \mid \langle m'[d/x] \rangle)$$

Case (PROC-END):

$$vcd.(\langle \mathcal{M}[\mathbf{close} c] \rangle \mid \langle \mathcal{N}[\mathbf{wait} d] \rangle) \Rightarrow \langle \mathcal{M}[\mathbf{return} ()] \rangle \mid \langle \mathcal{N}[\mathbf{return} ()] \rangle$$

By assumption, we have $\Theta \Vdash vcd.(\langle \mathcal{M}[\mathbf{close} c] \rangle \mid \langle \mathcal{N}[\mathbf{wait} d] \rangle) \sim P'$. By inversion on the erasure judgment we know there exists A and P'' such that $P' = vcd.P''$ and

$$\Theta, c :_{\mathbf{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbf{L}} \mathbf{hc}\langle A \rangle \Vdash (\langle \mathcal{M}[\mathbf{close} c] \rangle \mid \langle \mathcal{N}[\mathbf{wait} d] \rangle) \sim P''$$

By inversion on the erasure judgment we know there exists $\Theta_1, \Theta_2, m'_1, n'_1$ such that $\Theta = \Theta_1 \cup \Theta_2$ and $P'' = \langle m'_1 \rangle \mid \langle n'_1 \rangle$ and

$$\begin{aligned}\Theta_1, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{close} \ c] &\sim m'_1 : C(\text{unit}) \\ \Theta_2, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{wait} \ d] &\sim n'_1 : C(\text{unit})\end{aligned}$$

By Lemma 5.140 there exists $\Theta_{11}, \Theta_{12}, \mathcal{M}', m'_2, B$ such that

$$\begin{aligned}\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{close} \ c &\sim m'_2 : C(B) \\ \Theta_1 &= \Theta_{11} \cup \Theta_{12} \\ m'_1 &= \mathcal{M}'[m'_2]\end{aligned}$$

By Lemma 5.140 there exists $\Theta_{21}, \Theta_{22}, \mathcal{N}', n'_2, C$ such that

$$\begin{aligned}\Theta_{21}, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{wait} \ d &\sim n'_2 : C(C) \\ \Theta_2 &= \Theta_{21} \cup \Theta_{22} \\ n'_1 &= \mathcal{N}'[n'_2]\end{aligned}$$

By Lemma 5.120 we know that $m'_2 = \mathbf{close} \ c$ and by Lemma 5.121 we know that $n'_2 = \mathbf{wait} \ d$. Now following the same reasoning as in the proof of session fidelity (Theorem 5.84) for the Proc-END case, we have

$$\begin{aligned}\Theta_{12} \Vdash \langle \mathcal{M}[\mathbf{return} \ ()] \rangle &\sim \langle \mathcal{M}'[\mathbf{return} \ ()] \rangle : C(\text{unit}) \\ \Theta_{22} \Vdash \langle \mathcal{N}[\mathbf{return} \ ()] \rangle &\sim \langle \mathcal{N}'[\mathbf{return} \ ()] \rangle : C(\text{unit}) \\ \Theta_{11} = \Theta_{21} &= \epsilon\end{aligned}$$

By PAR we have

$$\Theta_{12} \cup \Theta_{22} \Vdash (\langle \mathcal{M}[\mathbf{return} \ ()] \rangle \mid \langle \mathcal{N}[\mathbf{return} \ ()] \rangle) \sim (\langle \mathcal{M}'[\mathbf{return} \ ()] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \ ()] \rangle)$$

which concludes this case by taking $Q'_0 = \langle \mathcal{M}'[\mathbf{return} \ ()] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \ ()] \rangle$ and noting that

$$vcd.(\langle \mathcal{M}'[\mathbf{close} \ c] \rangle \mid \langle \mathcal{N}'[\mathbf{wait} \ d] \rangle) \Rightarrow \langle \mathcal{M}'[\mathbf{return} \ ()] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \ ()] \rangle$$

Case (Proc-Com):

$$vcd.(\langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle) \Rightarrow vcd.(\langle \mathcal{M}[\mathbf{return} \ c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \ \langle v, d \rangle_{\mathbb{L}}] \rangle)$$

By assumption, we have $\Theta \Vdash vcd.(\langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle) \sim P'$. By inversion on the erasure judgment we know there exists κ, A and P'' such that $P' = vcd.P''$ and

$$\Theta, c :_{\mathbb{L}} \mathbf{ch}^{\kappa}\langle A \rangle, d :_{\mathbb{L}} \neg \mathbf{ch}^{\kappa}\langle A \rangle \Vdash (\langle \mathcal{M}[\mathbf{send} \ c \ v] \rangle \mid \langle \mathcal{N}[\mathbf{recv} \ d] \rangle) \sim P''$$

By inversion on the erasure judgment we know there exists $\Theta_1, \Theta_2, m'_1, n'_1$ such that $\Theta = \Theta_1 \cup \Theta_2$ and $P'' = \langle m'_1 \rangle \mid \langle n'_1 \rangle$ and

$$\begin{aligned}\Theta_1, c :_{\mathbb{L}} \mathbf{ch}^{\kappa}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{M}[\mathbf{send} \ c \ v] &\sim m'_1 : C(\text{unit}) \\ \Theta_2, d :_{\mathbb{L}} \neg \mathbf{ch}^{\kappa}\langle A \rangle; \epsilon; \epsilon \vdash \mathcal{N}[\mathbf{recv} \ d] &\sim n'_1 : C(A)\end{aligned}$$

By Lemma 5.140 there exists $\Theta_{11}, \Theta_{12}, \mathcal{M}', m'_2, B$ such that

$$\begin{aligned}\Theta_{11}, c :_{\mathbb{L}} \mathbf{ch}^{\kappa}\langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \ c \ v &\sim m'_2 : C(B) \\ \Theta_1 &= \Theta_{11} \cup \Theta_{12} \\ m'_1 &= \mathcal{M}'[m'_2]\end{aligned}$$

By Lemma 5.140 there exists $\Theta_{21}, \Theta_{22}, \mathcal{N}', n'_2, C$ such that

$$\begin{aligned}\Theta_{21}, d :_{\mathbb{L}} \neg \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash \mathbf{recv} \, d \sim n'_2 : C(C) \\ \Theta_2 = \Theta_{21} \cup \Theta_{22} \\ n'_1 = \mathcal{N}'[n'_2]\end{aligned}$$

By Lemma 5.103, Lemma 5.115 and Lemma 5.114 we know that $m'_2 = \mathbf{send} \, c \, m'_3$ for some m'_3 .

By Lemma 5.128 we know there exists $\Theta_{111}, \Theta_{112}, A'$ and r such that

$$\begin{aligned}\Theta_{111}, c :_{\mathbb{L}} \text{ch}^K \langle A \rangle; \epsilon; \epsilon \vdash \mathbf{send} \, c \sim \mathbf{send} \, c : \Pi_t(x : A').C(B) \\ \Theta_{112}; \epsilon; \epsilon \vdash v \sim m'_3 : A'\end{aligned}$$

By Lemma 5.122 we know that $m'_3 = v'$ for some value v' .

By Lemma 5.116 we know that $n'_2 = \mathbf{recv} \, d$.

Now following the same reasoning as in the proof of session fidelity (Theorem 5.84) for the PROC-COM case, we know there exists B' such that

$$\begin{aligned}\Theta_{12}, c :_{\mathbb{L}} \text{ch}^K \langle B' \rangle \Vdash \langle \mathcal{M}[\mathbf{return} \, c] \rangle \sim \langle \mathcal{M}'[\mathbf{return} \, c] \rangle \\ \Theta_{22} \cup \Theta_{112}, d :_{\mathbb{L}} \neg \text{ch}^K \langle B' \rangle \Vdash \langle \mathcal{N}[\mathbf{return} \, \langle v, d \rangle_{\mathbb{L}}] \rangle \sim \langle \mathcal{N}'[\mathbf{return} \, \langle v', d \rangle_{\mathbb{L}}] \rangle \\ \Theta_{111} = \Theta_{21} = \epsilon\end{aligned}$$

By PAR and SCOPE we have

$$\Theta_{12} \cup \Theta_{22} \cup \Theta_{112} \Vdash \text{vcd}.(\langle \mathcal{M}[\mathbf{return} \, c] \rangle \mid \langle \mathcal{N}[\mathbf{return} \, \langle v, d \rangle_{\mathbb{L}}] \rangle) \sim \text{vcd}.(\langle \mathcal{M}'[\mathbf{return} \, c] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \, \langle v', d \rangle_{\mathbb{L}}] \rangle)$$

which concludes this case by taking $Q'_0 = \text{vcd}.(\langle \mathcal{M}'[\mathbf{return} \, c] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \, \langle v', d \rangle_{\mathbb{L}}] \rangle)$ and noting

$$\text{vcd}.(\langle \mathcal{M}'[\mathbf{send} \, c \, v'] \rangle \mid \langle \mathcal{N}'[\mathbf{recv} \, d] \rangle) \Rightarrow \text{vcd}.(\langle \mathcal{M}'[\mathbf{return} \, c] \rangle \mid \langle \mathcal{N}'[\mathbf{return} \, \langle v', d \rangle_{\mathbb{L}}] \rangle)$$

Case (PROC-COM): Similar to the PROC-COM case.

Case (PROC-EXPR):

$$\frac{m \rightsquigarrow n}{\langle m \rangle \Rightarrow \langle n \rangle} (\text{PROC-EXPR})$$

By assumption, we have $\Theta \Vdash \langle m \rangle \sim P'$. By inversion on the erasure judgment we know there exists m' such that $\Theta; \epsilon; \epsilon \vdash m \sim m' : C(A)$ and $P' = \langle m' \rangle$.

By Theorem 5.139 there exists n' such that $\Theta; \epsilon; \epsilon \vdash n \sim n' : C(A)$ and $m' \rightsquigarrow n'$.

By EXPR we have $\Theta \Vdash \langle n \rangle \sim \langle n' \rangle$. We conclude this case by taking $Q'_0 = \langle n' \rangle$ and noting that $\langle m' \rangle \Rightarrow \langle n' \rangle$ by PROC-EXPR.

Case (PROC-PAR):

$$\frac{P \Rightarrow Q}{O \mid P \Rightarrow O \mid Q} (\text{PROC-PAR})$$

By assumption, we have $\Theta \Vdash (O \mid P) \sim P'$. By inversion on the erasure judgment we know there exists $\Theta_1, \Theta_2, O', P''$ such that $\Theta = \Theta_1 \cup \Theta_2$, $P' = O' \mid P''$ and

$$\begin{aligned}\Theta_1 \Vdash O \sim O' \\ \Theta_2 \Vdash P \sim P''\end{aligned}$$

By the induction hypothesis, there exists Q' such that $P'' \Rightarrow Q'$ and $\Theta_2 \Vdash Q \sim Q'$. By PAR we have $\Theta \Vdash (O \mid Q) \sim (O' \mid Q')$. We conclude this case by taking $Q'_0 = O' \mid Q'$ and noting that $O' \mid P'' \Rightarrow O' \mid Q'$ by PROC-PAR.

Case (PROC-SCOPE):

$$\frac{P \Rightarrow Q}{vcd.P \Rightarrow vcd.Q} (\text{PROC-SCOPE})$$

By assumption, we have $\Theta \Vdash vcd.P \sim P'$. By inversion on the erasure judgment we know there exists P'' such that $P' = vcd.P''$ and $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash P \sim P''$ for some A .

By the induction hypothesis, there exists Q' such that $P'' \Rightarrow Q'$ and $\Theta, c :_{\mathbb{L}} \mathbf{ch}\langle A \rangle, d :_{\mathbb{L}} \mathbf{hc}\langle A \rangle \Vdash Q \sim Q'$. By SCOPE we have $\Theta \Vdash vcd.Q \sim vcd.Q'$. We conclude this case by taking $Q'_0 = vcd.Q'$ and noting that $vcd.P'' \Rightarrow vcd.Q'$ by PROC-SCOPE.

Case (PROC-CONGR):

$$\frac{P \equiv P_1 \quad P_1 \Rightarrow Q_1 \quad Q_1 \equiv Q}{P \Rightarrow Q}$$

By assumption, we have $\Theta \Vdash P \sim P'$. By Lemma 5.138 there exists P'_1 such that $\Theta \Vdash P_1 \sim P'_1$ and $P' \equiv P'_1$. By the induction hypothesis, there exists Q'_1 such that $P'_1 \Rightarrow Q'_1$ and $\Theta \Vdash Q_1 \sim Q'_1$. By Lemma 5.138 there exists Q'_0 such that $\Theta \Vdash Q \sim Q'_0$ and $Q'_1 \equiv Q'_0$. We conclude this case by noting that $P' \Rightarrow Q'_0$ by PROC-CONGR.

□

References

- [1] Ankush Das, Di Wang, and Jan Hoffmann. 2023. Probabilistic Resource-Aware Session Types. *Proceedings of the ACM on Programming Languages* 7, POPL (Jan. 2023), 1925–1956. doi:10.1145/3571259
- [2] Qiancheng Fu, Ankush Das, and Marco Gaboardi. 2025. Probabilistic Refinement Session Types. *Proceedings of the ACM on Programming Languages* 9, PLDI (June 2025), 1666–1691. doi:10.1145/3729317